CS52 · 5TH SEMESTER · CIE-1

Artificial Intelligence & Machine Learning

The complete CIE-1 study guide — from zero to exam-ready. Every topic in plain language, every class example solved step by step.

Unit I

What is AI · Intelligent agents · Problem solving by search (uninformed & informed) · Heuristics · Logical agents · Wumpus world · Propositional logic

Unit II

First-order logic · Inference in FOL (unification, forward/backward chaining, resolution) · Knowledge graphs & ontologies · Uncertainty & Bayes · Intro to ML · Find-S & Candidate-Elimination

Unit III (CIE-1 part)

Classification vs regression · Linear regression with examples · Naive Bayes classifier · Safety net: logistic regression, SVM, KNN

Course coordinator: Dr. Rajarajeswari S · Text books: Russell & Norvig, *AI: A Modern Approach* (4th ed.); Tom M. Mitchell, *Machine Learning* · CIE: 50 marks (avg. of two tests for 30 + lab 10 + case study 10)

0 How to use this guide

What's inside

This guide covers **everything for CIE-1**: all of Unit I, all of Unit II, and the three Unit III topics your CR shared (classification vs regression, Naive Bayes, linear regression with example). It merges your class notes, slides and the textbook, and solves every worked example from class step by step.

Each chapter follows the same pattern:

1. **Plain-English idea** first — what it is and why it exists.
2. **Formal definition** — the line the examiner expects in your answer.
3. **Diagram / table** — draw these in the exam.
4. **Worked example** — numbers checked.
5. **Likely questions + 60-second recap** at the end.

Box legend

DEFINITION
Write this word-for-word-ish.

KEY POINT
The idea that makes it click.

EXAM TIP
How to score full marks.

COMMON MISTAKE
Where students lose marks.

WORKED EXAMPLE
Solved problems.

MEMORY TRICK
Mnemonics.

★ Very likely = high chance in the exam
★ Likely = good chance
Yellow highlight = the key phrase to remember.

Study plan (fast track)

Session	What to study	Goal
Day 1	Ch 1–2 (AI, agents, PEAS, environment types, agent types) + Ch 3 search (BFS, DFS, UCS, IDS, A*)	Draw the 5 agent diagrams; trace A* on the Romania map
Day 2	Ch 4 logical agents & propositional logic + Ch 5 FOL	Truth table, CNF, resolution in PL; translate 10 English sentences to FOL
Day 3	Ch 6 inference in FOL + Ch 7 knowledge graphs	Unification table; crime example (FC + BC); "John is happy" by resolution
Day 4	Ch 8 uncertainty & Bayesian networks + Ch 9 intro to ML	Cancer MAP, PlayTennis NB, P(W) in a Bayes net; Find-S and Candidate-Elimination tables
Day 5	Ch 10–12 (Unit III) + Ch 13 skim	Comparison table; hours→marks regression; email NB with Laplace
Last night	Revision pages at the end: formula sheet, top-25 questions, checklist	Re-solve 1 numerical per topic without looking

HOW TO WRITE ANSWERS IN AN AIML PAPER

- **Definition → diagram → explanation → example → advantages/limitations.** Even a 5-mark answer should have a diagram or table.
- For numericals, write the **formula first**, then substitution, then the boxed answer with units/meaning.
- For proofs (chaining, resolution), number every clause and write the substitution on every step.
- Underline keywords: rational, admissible, entailment, unification, posterior, conditional independence.

ABOUT THE SYLLABUS

Two syllabus copies exist. The revised 2026 copy (from the class Drive) lists **First-Order Logic under Unit II** and marks **Uninformed Search** (Unit I) and **Knowledge Graphs & Ontologies** (Unit II) as self-study (SS) topics. Self-study topics are still examinable — they're fully covered here. For Unit III, CIE-1 covers only the three topics your CR shared; if the teacher announces more, Ch 13 gives you a head start.

Contents

Unit I — Agents, Search & Logic

1. What is AI?
2. Intelligent agents: PEAS, environments, agent types
3. Problem solving by search: BFS, UCS, DFS, DLS, IDS, bidirectional, greedy, A*, heuristics
4. Logical agents, Wumpus world, propositional logic, resolution, chaining

Unit II — Logic, Knowledge, Uncertainty, Intro to ML

5. First-order logic
6. Inference in FOL: UI/EI, unification, lifting, forward & backward chaining, resolution
7. Knowledge graphs & ontologies
8. Reasoning under uncertainty: Bayes, MAP, Bayesian networks, Naive Bayes
9. Introduction to ML: definitions, tasks, well-posed problems, types, Find-S, Candidate-Elimination

Unit III — CIE-1 portion

10. Classification vs regression
11. Linear regression (3 worked examples, R^2, polynomial)
12. Naive Bayes classifier (4 worked examples, Laplace)
13. Safety net: logistic regression, SVM, KNN

Revision

14. Formula sheet, top questions, last-night checklist

UNIT I

Intelligent Agents, Search & Logic

- **Ch 1 · What is AI?** four approaches, the Turing test, why we build rational agents
- **Ch 2 · Intelligent Agents** agents & environments, rationality, PEAS, environment types, the five agent structures
- **Ch 3 · Problem Solving by Searching** problem formulation, toy & real problems, search trees, uninformed search (BFS, UCS, DFS, DLS, IDS, bidirectional), informed search (greedy, A*), heuristic functions
- **Ch 4 · Logical Agents** knowledge-based agents, the Wumpus world, logic & entailment, propositional logic, equivalences, resolution, Horn clauses, forward & backward chaining, agents based on propositional logic

1 What is Artificial Intelligence?

Start here. This chapter gives you the vocabulary of the whole course: what "intelligent" means for a machine, the four ways people have defined AI, and why this textbook builds **rational agents**.

HOW TO STUDY UNIT I (PLAN)

- ☐ **Session 1:** Ch 1 + Ch 2 (definitions, PEAS, environment types, agent diagrams). These are easy marks.
- ☐ **Session 2:** Ch 3 up to uninformed search. Practise the traces on paper (frontier at every step).
- ☐ **Session 3:** Ch 3 informed search (A* on the Romania map, h_1/h_2).
- ☐ **Session 4:** Ch 4 (Wumpus world, truth table, equivalences, CNF, resolution, forward/backward chaining).
- ☐ **Before the test:** read every *60-second recap* box and attempt the *Likely exam questions*.

Tags: ★ Very likely = appears in almost every CIE/SEE paper; ★ Likely = asked often.

1.1 Intelligence and AI in plain words

Intelligence is the ability to *perceive* a situation, *understand / reason* about it, *decide*, and *act* so as to reach a goal — and to *learn* from experience. **Artificial Intelligence (AI)** is the field that tries to build machines (programs, robots) that show this kind of intelligent behaviour.

DEFINITION (WRITE THIS IN THE EXAM)

Artificial Intelligence is the study and design of intelligent agents: systems that perceive their environment through sensors and act upon it through actuators so as to achieve the best possible (expected) outcome. AI is concerned not only with *understanding* intelligence but also with *building* intelligent entities.

Different researchers defined AI differently. The textbook groups eight famous definitions along **two dimensions**:

- **Thinking vs Acting** — do we care about the *thought process / reasoning* inside, or only about the outside *behaviour*?
- **Human vs Rational** — do we measure success by *how closely it matches humans* (fidelity to human performance), or against an *ideal* standard of doing the right thing (**rationality**)?

Thinking Humanly	Thinking Rationally
"The exciting new effort to make computers think … *machines with minds*, in the full and literal sense." (Haugeland, 1985) "The automation of activities that we associate with human thinking — decision-making, problem solving, learning." (Bellman, 1978)	"The study of mental faculties through the use of computational models." (Charniak & McDermott, 1985) "The study of the computations that make it possible to perceive, reason, and act." (Winston, 1992)
Acting Humanly	**Acting Rationally**
"The art of creating machines that perform functions that require intelligence when performed by people." (Kurzweil, 1990) "The study of how to make computers do things at which, at the moment, people are better." (Rich & Knight, 1991)	"Computational Intelligence is the study of the design of intelligent agents." (Poole et al., 1998) "AI … is concerned with intelligent behaviour in artifacts." (Nilsson, 1998)

1.2 The four approaches to AI ★ Very likely

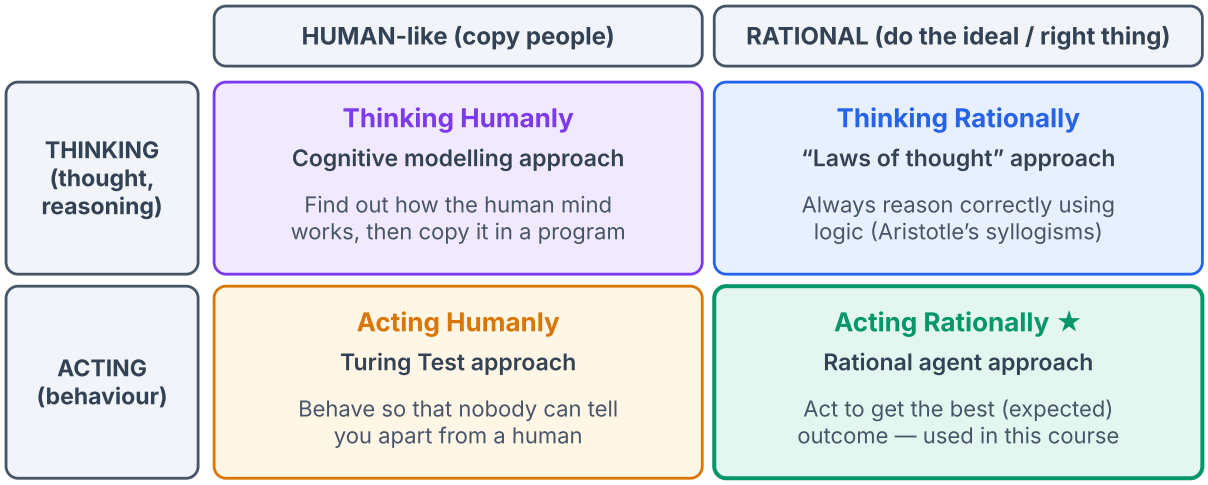


Fig 1.1 The 2 × 2 grid: Thinking/Acting × Human/Rational gives four approaches.

(a) Acting humanly — the Turing Test approach

Plain idea: "If it behaves like a human and you can't tell the difference, call it intelligent." Alan Turing (1950, paper *"Computing Machinery and Intelligence"*) proposed an operational test instead of a philosophical definition.

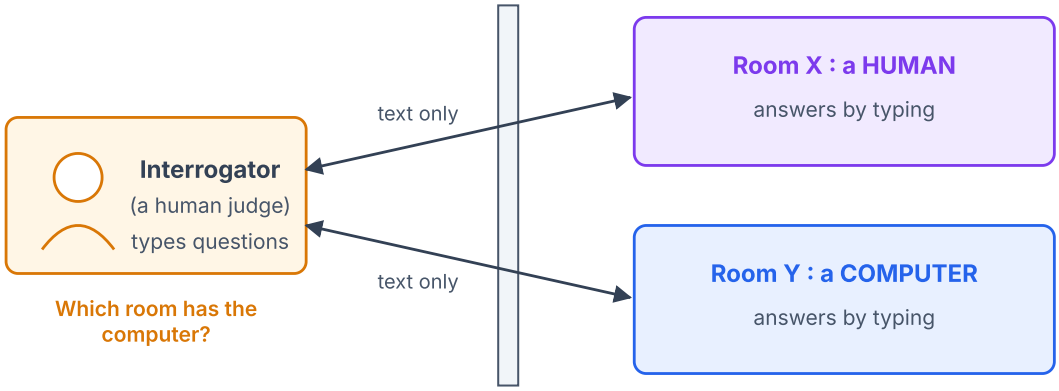


Fig 1.2 The Turing test (imitation game).

1. A human **interrogator** types questions to two hidden parties: a human and a computer.
2. Both reply only through text (a teletype / chat), so voice and looks give nothing away.
3. The computer passes the test if the interrogator cannot reliably tell which reply came from the computer.

To pass the (standard) Turing test, the computer needs **four capabilities**:

Capability	Why it is needed
1. Natural language processing (NLP)	to communicate successfully in English (or any human language)
2. Knowledge representation	to store what it knows or hears
3. Automated reasoning	to use the stored information to answer questions and draw new conclusions
4. Machine learning	to adapt to new circumstances and to detect and extrapolate patterns

The **Total Turing Test** also includes a video signal (so the interrogator can test perceptual abilities) and a hatch to pass physical objects. It needs two more capabilities: **5. Computer vision** (to perceive objects) and **6. Robotics** (to manipulate objects and move about).

MEMORY TRICK — THE 6 CAPABILITIES

"Nice Kids Always Learn … Very Rapidly" → **N**LP, **K**nowledge representation, **A**utomated reasoning, **L**earning (ML) … + **V**ision, **R**obotics for the *Total* Turing test.

Why AI researchers do not focus on passing it: it is more important to study the underlying principles of intelligence than to duplicate a human. ("Aeronautical engineers did not build aeroplanes by copying pigeons so exactly that they fool other pigeons.")

(b) Thinking humanly — the cognitive modelling approach

Plain idea: first find out *how humans actually think*, then write a program that thinks the same way. We can learn how the mind works in three ways: **introspection** (catching our own thoughts as they go by), **psychological experiments** (observing a person in action) and **brain imaging** (observing the brain in action). If the program's input–output behaviour *and timing* match human behaviour, the program's mechanisms may also be operating in humans.

Example: Newell and Simon's **GPS (General Problem Solver)** — they cared more about comparing its reasoning steps with those of human subjects than about simply getting correct answers. This approach led to the field of **cognitive science** (computer models from AI + experimental methods from psychology).

(c) Thinking rationally — the "laws of thought" approach

Plain idea: always reason *correctly*, using the rules of logic. The Greek philosopher **Aristotle** gave **syllogisms** — patterns of argument that always give correct conclusions from correct premises:

> "Socrates is a man; all men are mortal; therefore Socrates is mortal."

The **logicist** tradition in AI hopes to build intelligent systems from such logical programs. **Two main obstacles:**

1. It is not easy to take *informal* knowledge and state it in formal logical notation, especially when the knowledge is less than 100% certain.
2. There is a big difference between solving a problem "in principle" and solving it in practice: even a few hundred facts can exhaust a computer's resources unless it knows which reasoning steps to try first.

(d) Acting rationally — the rational agent approach ★

Plain idea: do the right thing — whatever is expected to give the best result. An **agent** is just something that acts (Latin *agere*, "to do"). A computer agent is expected to do more: operate autonomously, perceive its environment, persist over a long time, adapt to change, and create and pursue goals.

DEFINITION

A **rational agent** is one that acts so as to achieve the best outcome or, when there is uncertainty, the best expected outcome.

Why the rational-agent approach is preferred (textbook reasons):

1. **More general than "laws of thought".** Correct inference is only *one* way of acting rationally. Some rational actions involve no inference at all — e.g. pulling your hand back from a hot stove is a *reflex*, faster and better than slow deliberation.
2. **More open to scientific development.** Rationality is *mathematically well defined* and completely general, so we can "unpack" it to design agents that provably achieve it. Human behaviour, in

contrast, is tuned for one specific environment and is the product of a complex, largely unknown evolutionary process.

3. All the skills needed for the Turing test (language, reasoning, knowledge, learning) are still useful — they help an agent act rationally.

Limited rationality: perfect rationality (always doing exactly the right thing) is not feasible in complicated environments because the computation needed is too high; real agents must act appropriately when there is not enough time to do all the computations.

EXAM TIP — "EXPLAIN THE FOUR APPROACHES TO AI" (10M)

- Draw the **2 × 2 grid** (Fig 1.1) first — it earns marks by itself.
- For each approach write: **name of the approach** (cognitive modelling / laws of thought / Turing test / rational agent), **one-line idea**, **one example** (GPS, Aristotle's syllogism, Turing test, reflex vs. deliberation), and a **limitation**.
- End with "The textbook adopts the rational agent approach because it is more general and mathematically well-defined."

COMMON MISTAKES

- Mixing the axes: the Turing test is **acting humanly** (it only checks behaviour), *not* thinking humanly.
- Writing only four capabilities for the **Total** Turing test — it needs **six** (add vision and robotics).
- "Rational = perfect / all-knowing." No: rational means best *expected* outcome given what the agent knows (more in Ch 2).

1.3 Foundations of AI (where the ideas come from)

Discipline	Question it answered for AI	Contribution (examples)
Philosophy	Can formal rules be used to draw valid conclusions? Where does knowledge come from? How does knowledge lead to action?	Aristotle's syllogisms, rationalism, mind as a physical system
Mathematics	What are the formal rules to draw valid conclusions? What can be computed? How do we reason with uncertain information?	Logic (Boole, Frege), algorithms, computability (Turing), intractability / NP-completeness, probability (Bayes)
Economics	How should we make decisions to maximise payoff?	Utility, decision theory, game theory, MDPs
Neuroscience	How do brains process information?	Neurons; the brain causes the mind
Psychology	How do humans and animals think and act?	Behaviourism, cognitive psychology
Computer engineering	How can we build an efficient computer?	The hardware that runs AI programs
Control theory & cybernetics	How can artifacts operate under their own control?	Feedback, minimising an objective function over time
Linguistics	How does language relate to thought?	Computational linguistics / NLP

1.4 A very short history

Year	Milestone
1943	McCulloch & Pitts: first model of artificial neurons.
1950	Turing's paper "Computing Machinery and Intelligence" — the Turing test, machine learning, reinforcement learning ideas.
1956	**Dartmouth workshop** — John McCarthy coins the term **"Artificial Intelligence"**. Birth of AI.
1952–69	Early enthusiasm: Logic Theorist, GPS, LISP (1958), checkers program that learned.
1966–73	A dose of reality: early systems failed to scale (combinatorial explosion).
1969–86	Knowledge-based / **expert systems** (DENDRAL, MYCIN); AI becomes an industry (1980s).
1986–	Return of neural networks (back-propagation); probabilistic reasoning and machine learning grow.
1997	IBM **Deep Blue** beats world chess champion Garry Kasparov.
2011–today	Big data and **deep learning** (image recognition 2012), AlphaGo beats Lee Sedol (2016), large language models.

1.5 What can AI do today? (applications)

Self-driving / robotic vehicles · Speech recognition · Autonomous planning & scheduling · Game playing (chess, Go) · Spam fighting · Logistics planning · Robotics · Machine translation · Medical diagnosis · Recommendation systems · Face / image recognition

EXAM TIP — SHORT ANSWERS

For a 4–5 mark "What is AI?" question: give the **definition**, the **two dimensions**, name the **four approaches** in one line each, and finish with **3–4 applications**. Do not write a long history unless asked.

LIKELY EXAM QUESTIONS — CHAPTER 1

1. Define AI. Explain the four approaches to AI with a neat diagram. **(10M)**
2. What is the Turing test? List the capabilities a computer needs to pass it. What is the Total Turing test? **(6M)**
3. Explain the "laws of thought" approach and its two obstacles. **(4M)**
4. Why is the rational-agent approach preferred over the other approaches? **(5M)**
5. Explain the cognitive modelling approach. **(4M)**
6. List the foundations of AI and the question each discipline answers. **(5M)**
7. List any five applications of AI. **(3M)**

60-SECOND RECAP

- AI = building **intelligent agents** that perceive and act to achieve the best expected outcome.
- Two axes: **thinking vs acting**, **human vs rational** → four approaches.
- Thinking humanly = **cognitive modelling**; thinking rationally = **laws of thought** (logic, syllogisms); acting humanly = **Turing test**; acting rationally = **rational agent** (used in this course).
- Turing test needs **NLP, knowledge representation, automated reasoning, ML**; Total Turing test adds **vision, robotics**.
- Rational-agent approach wins: more general (not only inference) and mathematically well-defined.

2 Intelligent Agents

An agent is anything that senses and acts. This chapter defines a *good* (rational) agent, shows how to describe its task (PEAS), how to classify environments, and the five standard agent designs — all very frequently examined.

2.1 Agents and environments

DEFINITIONS

An **agent** is anything that can be viewed as perceiving its environment through sensors and acting upon that environment through actuators.

- **Sensor** — the part that takes input from the environment (eyes, camera, keyboard).
- **Actuator** — the part that produces an effect on the environment (hands, motors, screen display).
- **Percept** — the agent's perceptual input *at any given instant*.
- **Percept sequence** — the *complete history* of everything the agent has ever perceived.

Key fact: an agent's choice of action at any instant can depend on its entire percept sequence observed to date, but not on anything it hasn't perceived.

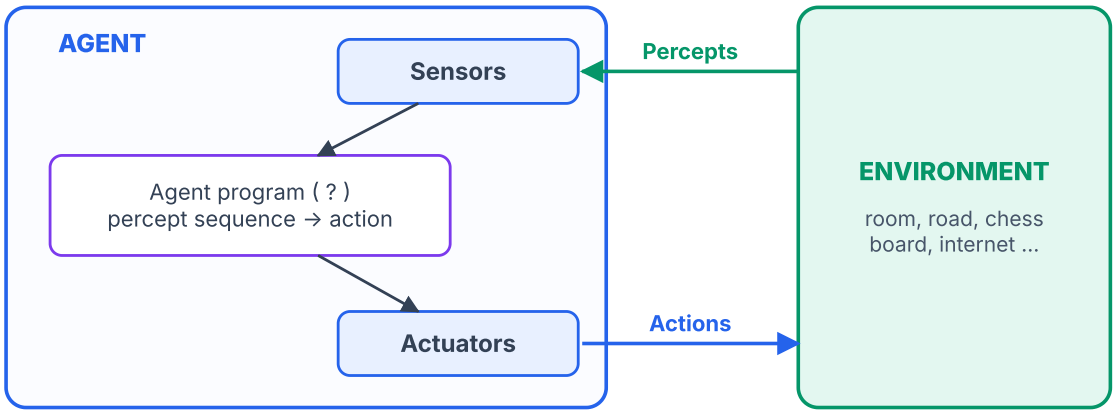


Fig 2.1 Agents interact with environments through sensors and actuators. The "?" is the agent program we must design.

Agent	Sensors	Actuators
Human agent	eyes, ears, nose, skin (touch)	hands, legs, vocal tract (mouth)
Robotic agent	cameras, infrared range finders	various motors (wheels, arms)
Software agent	keystrokes, file contents, received network packets	displaying on screen, writing files, sending network packets

Agent function vs agent program

AGENT FUNCTION (ABSTRACT)

A **mathematical mapping** from *every possible percept sequence* to an action:

$$f : P^* \to A$$

It describes the agent's behaviour completely; in principle it could be written as a (huge) table.

AGENT PROGRAM (CONCRETE)

The **actual code** that implements the agent function and runs on the agent's physical **architecture** (computer + sensors + actuators).

It takes only the **current percept** as input (if it needs history, it must remember it).

MEMORY TRICK

Function = WHAT the agent does (a specification on paper). **Program = HOW** it does it (code in a machine). Agent = Architecture + Program.

2.2 Worked example: the vacuum-cleaner world

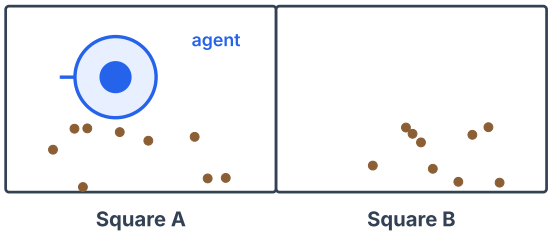


Fig 2.2 A vacuum-cleaner world with just two squares.

- **Squares:** A and B.
- **Percepts:** [location, status], e.g. `[A, Dirty]`.
- **Actions:** *Left*, *Right*, *Suck*, *NoOp* (do nothing).
- **Simple agent function:** "if the current square is dirty, suck; otherwise move to the other square."

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [A, Dirty]	Suck
...	...
[A, Clean], [A, Clean], [A, Dirty]	Suck

Partial tabulation of the agent function.

```
function REFLEX-VACUUM-AGENT([location, status])
          returns an action
  if status = Dirty then return Suck
  else if location = A then return Right
  else if location = B then return Left
```

The table on the left is the **agent function**; this 3-line code is an **agent program** implementing it. Notice how much smaller the program is.

2.3 Good behaviour: the concept of rationality ★ Very likely

A **rational agent** is one that *does the right thing*. But what is "right"? We judge an agent by the *consequences* of its behaviour: the actions change the environment through a sequence of states; if that sequence is desirable, the agent performed well. Desirability is captured by a **performance measure**.

DESIGNING A PERFORMANCE MEASURE

Design performance measures according to what one actually wants in the environment, rather than according to how one thinks the agent should behave.

Example: "amount of dirt cleaned in 8 hours" is a bad measure — a clever agent could clean, dump the dirt back, and clean again! Better: "reward the agent for having a clean floor", e.g. +1 point for each clean square at each time step.

What is rational at any given time depends on FOUR things

1. The **performance measure** that defines the criterion of success.
2. The agent's **prior knowledge** of the environment.
3. The **actions** that the agent can perform.
4. The agent's **percept sequence** to date.

MEMORY TRICK

"PPAP" → **P**erformance measure, **P**rior knowledge, **A**ctions, **P**ercept sequence.

DEFINITION OF A RATIONAL AGENT (LEARN WORD-FOR-WORD)

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

WORKED EXAMPLE — IS THE SIMPLE VACUUM AGENT RATIONAL?

Yes, **under these assumptions**:

- Performance measure: +1 for each clean square at each time step, over 1000 time steps.
- The geography is known in advance, but the dirt distribution and the agent's initial location are not. Clean squares stay clean; *Suck* cleans the current square; *Left*/*Right* move the agent (no effect at the edge).
- The only actions are *Left*, *Right*, *Suck*; the agent correctly perceives its location and whether it contains dirt.

Under different circumstances the **same agent becomes irrational**: e.g. if each movement costs −1 point, then once both squares are clean it keeps oscillating needlessly; a better agent would do *NoOp*. If clean squares can become dirty again, it should check them occasionally.

Rationality is NOT omniscience

An **omniscient** agent knows the *actual* outcome of its actions and can act accordingly — impossible in reality. *Example:* you look both ways and cross the road; a cargo door falls from a passing aeroplane and flattens you. You were still **rational** (given what you could perceive), just not omniscient.

Rational	Omniscient	Perfect
maximizes **expected** performance, given percepts so far	knows the **actual** outcome of every action in advance	maximizes **actual** performance (needs omniscience)

Because rationality depends only on the percept sequence *to date*, a rational agent must also do:

- **Information gathering** — doing actions to modify future percepts (e.g. *looking* both ways before crossing). **Exploration** of an unknown environment is one kind of information gathering.

- **Learning** — learn as much as possible from what it perceives. Prior knowledge can be modified and augmented with experience.
- **Autonomy** — an agent is autonomous to the extent that it relies on its own percepts (and learning) rather than on the prior knowledge of its designer. Counter-examples from nature: the *dung beetle* keeps "plugging" its nest even after the dung ball is removed; the *sphex wasp* repeats its fixed routine endlessly when the caterpillar is moved. Their fixed behaviour fails when assumptions break. A truly rational agent should become independent of its prior knowledge after sufficient experience.

COMMON MISTAKES

- "A rational agent never makes mistakes." — Wrong. Rational ≠ omniscient ≠ perfect. It maximises **expected**, not actual, performance.
- Forgetting "*given the percept sequence and built-in knowledge*" in the definition — examiners look for these words.
- Confusing agent *function* (a mapping) with agent *program* (code).

2.4 The nature of environments

A **task environment** is the "problem" to which a rational agent is the "solution". Before designing an agent we must specify it as fully as possible.

PEAS description ★ Very likely

DEFINITION

PEAS = **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors — the four things that specify a task environment. Writing PEAS is always the first step in designing an agent.

WORKED EXAMPLE — PEAS FOR AN AUTOMATED TAXI DRIVER (THE TEXTBOOK'S MAIN EXAMPLE)

PEAS	Automated taxi driver	Why (plain words)
Performance measure	Safe, fast, legal, comfortable trip; maximize profits; minimize fuel consumption and wear; minimize impact on other road users	Goals often **conflict** (fast vs safe) → trade-offs needed
Environment	Roads (lanes, highways), other traffic, pedestrians, stray animals, road works, police cars, puddles, potholes, customers, weather	Everything the taxi deals with
Actuators	Steering, accelerator, brake, signal (indicator), horn, display / speech to talk to passengers	How it changes the world
Sensors	Cameras, sonar / radar / LIDAR, speedometer, GPS, odometer, accelerometer, engine sensors, keyboard / microphone	How it observes the world

Agent type	Performance measure	Environment	Actuators	Sensors
Medical diagnosis system	Healthy patient, reduced costs	Patient, hospital, staff	Display of questions, tests, diagnoses, treatments, referrals	Keyboard entry of symptoms, findings, patient's answers
Satellite image analysis system	Correct image categorization	Downlink from orbiting satellite	Display of scene categorization	Colour pixel arrays
Part-picking robot	Percentage of parts in correct bins	Conveyor belt with parts; bins	Jointed arm and hand	Camera, joint angle sensors
Refinery controller	Purity, yield, safety	Refinery, operators	Valves, pumps, heaters, displays	Temperature, pressure, chemical sensors
Interactive English tutor	Student's score on test	Set of students, testing agency	Display of exercises, suggestions, corrections	Keyboard entry
Vacuum-cleaner robot	Cleanliness, efficiency (time, energy), battery life, safety	Rooms, floor (wood/carpet), furniture, obstacles, dirt	Wheels, brushes, vacuum extractor	Camera, dirt-detection sensor, bump / cliff sensors, infrared wall sensor

EXAM TIP — PEAS QUESTIONS (5M)

- Draw a 4-row table with headings **P, E, A, S**; give **3–5 items** in each row.
- Be ready for *new* agents too (e.g. a robot soccer player, an online shopping assistant, a drone, a mathematician's theorem-proving assistant). Ask yourself: *How is success scored? Where does it live? What can it move/output? What can it read/sense?*
- Don't mix A and S: actuators **do**, sensors **sense**. "Camera" is never an actuator; "display" is never a sensor.

Properties (dimensions) of task environments ★ Very likely

Use the "test question" in the second column to decide each property quickly.

Property	Test question (plain English)	Meaning & examples
Fully observable vs **partially observable**	Can the sensors see *everything relevant* to choosing an action, at every moment?	**Fully:** sensors give the complete state (chess — the whole board is visible). No need to keep internal state. **Partially:** noisy/inaccurate sensors or parts of the state missing (vacuum agent with only a local dirt sensor can't tell whether the other square is dirty; a taxi can't see what other drivers are thinking; poker — opponents' cards hidden). **Unobservable:** no sensors at all.
Single agent vs **multiagent**	Is there another agent whose performance depends on my behaviour?	**Single:** crossword puzzle. **Multiagent:** chess (**competitive** — opponent maximises its score by minimising mine); taxi driving (**partially cooperative** — avoiding collisions helps everyone; **partially competitive** — only one car can take a parking space). Communication and randomized behaviour often matter in multiagent settings.
Deterministic vs **stochastic** (nondeterministic)	Is the next state *completely determined* by the current state and my action?	**Deterministic:** crossword, chess (ignoring the opponent). **Stochastic:** outcome uncertain, quantified by probabilities (taxi — tyres blow out, traffic unpredictable; backgammon — dice). **Nondeterministic:** actions have possible outcomes but *no probabilities* attached. An environment is **uncertain** if it is not fully observable or not deterministic.

Episodic vs **sequential**	Does my current decision affect future decisions?	**Episodic:** experience is divided into atomic episodes (perceive, then one action); the next episode does not depend on earlier actions (part-picking robot judging each part on an assembly line; image classification). **Sequential:** current decision can affect all future decisions (chess, taxi driving). Episodic is much simpler — no need to think ahead.
Static vs **dynamic** (vs **semidynamic**)	Can the world change *while I am thinking*?	**Static:** world waits for the agent (crossword). **Dynamic:** world keeps changing (taxi driving — other cars move while the agent decides). **Semidynamic:** the environment itself doesn't change with time, but the agent's *performance score* does (chess played with a clock).
Discrete vs **continuous**	Are states, time, percepts and actions *countable steps* or smooth values?	**Discrete:** chess — finite number of distinct states, discrete set of percepts and actions. **Continuous:** taxi driving — speed and location sweep through a range of continuous values smoothly over time; steering angles are continuous.
Known vs **unknown**	Does the agent (designer) know the "laws of physics" of the environment?	**Known:** the outcomes (or outcome probabilities) of all actions are given. **Unknown:** the agent must learn how the world works. Note: known ≠ fully observable — solitaire card games are known but partially observable; a new video game can be fully observable (the whole screen is visible) but unknown (you don't know what the buttons do).

KEY POINT

The **hardest** case is: partially observable, multiagent, stochastic, sequential, dynamic, continuous and unknown — e.g. taxi driving (except that the environment is mostly known).

MEMORY TRICK — THE 7 DIMENSIONS

"Oh, All Dogs Eat Some Dog Kibble" → **O**bservable? · **A**gents (single/multi) · **D**eterministic? · **E**pisodic? · **S**tatic? · **D**iscrete? · **K**nown?

Classification of standard task environments

Task environment	Observable	Agents	Deterministic	Episodic	Static	Discrete
Crossword puzzle	Fully	Single	Deterministic	Sequential	Static	Discrete
Chess with a clock	Fully	Multi	Deterministic	Sequential	Semi	Discrete
Poker	Partially	Multi	Stochastic	Sequential	Static	Discrete
Backgammon	Fully	Multi	Stochastic	Sequential	Static	Discrete
Taxi driving	Partially	Multi	Stochastic	Sequential	Dynamic	Continuous
Medical diagnosis	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Image analysis	Fully	Single	Deterministic	Episodic	Semi	Continuous
Part-picking robot	Partially	Single	Stochastic	Episodic	Dynamic	Continuous
Refinery controller	Partially	Single	Stochastic	Sequential	Dynamic	Continuous
Interactive English tutor	Partially	Multi	Stochastic	Sequential	Dynamic	Discrete

COMMON MISTAKES

- **Chess with a clock** is **semi**dynamic (not dynamic): the board doesn't change while you think, but your remaining time does.
- **Image analysis** is **episodic** (each image is independent) and semidynamic; **backgammon** is fully observable but **stochastic** (dice).
- "Partially observable" is caused by *sensors*; "stochastic" is caused by *actions/world dynamics*. Keep them separate.

EXAM TIP — "CLASSIFY THE ENVIRONMENT OF X"

Write **all 7 properties** as a list with **a one-line reason for each** (e.g. "Partially observable — the taxi cannot see around corners"). Reasons carry the marks; a bare table of words gets half credit.

2.5 The structure of agents ★ Very likely

$$\text{Agent} = \text{Architecture} + \text{Program}$$

The **architecture** is the computing device with physical sensors and actuators; the **program** implements the agent function. All agent programs here have the same skeleton: *take the current percept from the sensors, return an action to the actuators*.

Table-driven agent (and why it fails)

```
function TABLE-DRIVEN-AGENT(percept) returns an action
  persistent: percepts, a sequence, initially empty
              table, a table of actions, indexed by percept
                     sequences, initially fully specified
  append percept to the end of percepts
  action ← LOOKUP(percepts, table)
  return action
```

It literally stores the agent function as a table. If P is the set of possible percepts and T the lifetime, the table has $\sum_{t=1}^{T} |P|^t$ entries — for chess **at least 10^{150}** entries (the observable universe has fewer than 10^{80} atoms). It is doomed because:

1. no physical agent has the space to store the table;
2. the designer would not have time to create it;
3. no agent could ever learn all the right entries from experience;
4. even for a simple environment, there is no guidance about how to fill in the entries.

The key challenge of AI: write programs that produce rational behaviour from a small program rather than a vast table. Four basic kinds of agent programs do this, plus the learning agent:

MEMORY TRICK — AGENT TYPES IN INCREASING POWER

"Some Monkeys Grab Useful Leaves" → **S**imple reflex → **M**odel-based reflex → **G**oal-based → **U**tility-based → **L**earning agent.

(1) Simple reflex agent

Plain idea: reacts to *what it sees right now* using **condition–action rules** (if–then rules), ignoring all history. Like blinking when something flies at your eye.

if *car-in-front-is-braking* then *initiate-braking*

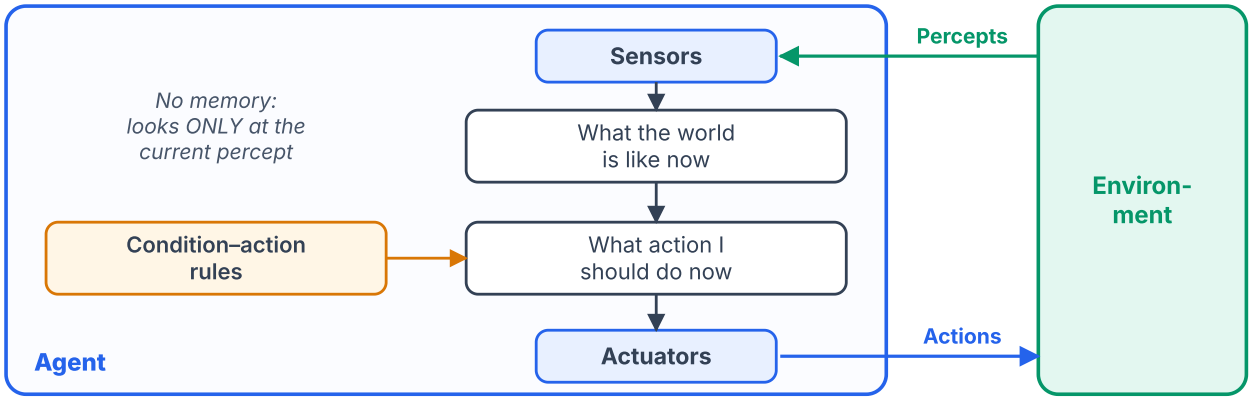


Fig 2.3 Schematic diagram of a simple reflex agent (white boxes = current internal state of the decision process; amber box = background knowledge used).

```
function SIMPLE-REFLEX-AGENT(percept) returns an action
  persistent: rules, a set of condition–action rules
  state  ← INTERPRET-INPUT(percept)
  rule   ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

- **Example:** REFLEX-VACUUM-AGENT (Section 2.2).
- **Advantage:** very simple, fast, small.
- **Limitation:** works only if the correct decision can be made on the basis of the current percept alone, i.e. only if the environment is fully observable. Even a little unobservability causes trouble.
- **Infinite loops:** a vacuum agent with no location sensor, in a clean square, may keep moving *Left* forever. Escape: **randomize** the action (e.g. flip a coin to choose Left/Right).

(2) Model-based reflex agent

Plain idea: keeps an **internal state** (memory) of the parts of the world it can't see now. Like a driver who remembers that a car is in the blind spot even when it is not visible in the mirror.

To update its internal state it needs two kinds of knowledge (together called a **model** of the world):

- **Transition model** — how the world changes over time: (a) how the world evolves independently of the agent (an overtaking car gets closer), and (b) how the agent's own actions affect the world (turning the wheel clockwise turns the car right).
- **Sensor model** — how the state of the world is reflected in the agent's percepts (e.g. when it is dark, car headlights appear in the camera image).

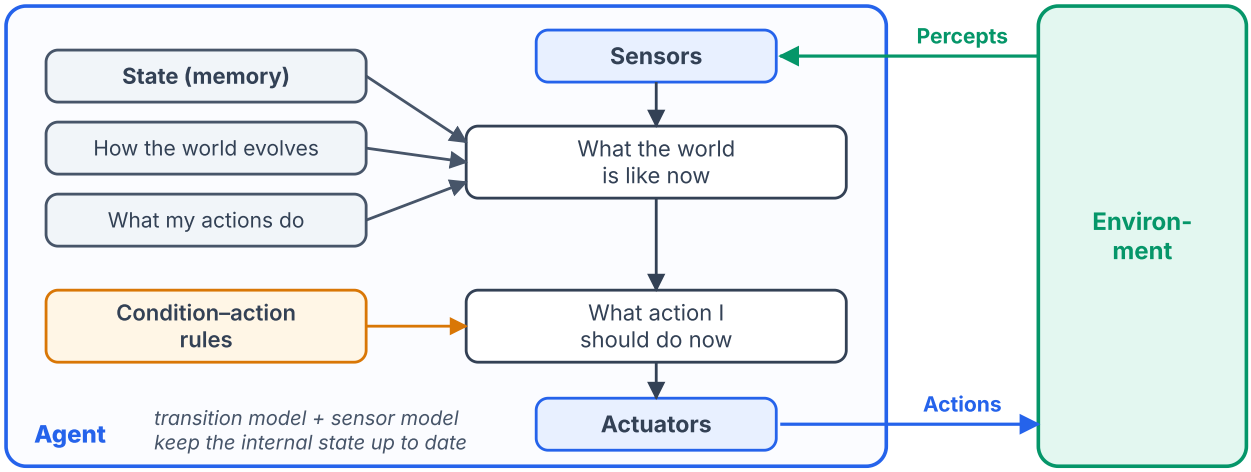


Fig 2.4 A model-based reflex agent: it combines the current percept with its old internal state to estimate the current world.

```
function MODEL-BASED-REFLEX-AGENT(percept) returns an action
  persistent: state, the agent's current conception of the world state
              transition_model, how the next state depends on the
                                current state and action
              sensor_model, how the current world state is
                            reflected in the agent's percepts
              rules, a set of condition–action rules
              action, the most recent action, initially none
  state  ← UPDATE-STATE(state, action, percept,
                        transition_model, sensor_model)
  rule   ← RULE-MATCH(state, rules)
  action ← rule.ACTION
  return action
```

The internal state is usually a *best guess* — in a partially observable world the agent can rarely know the exact state.

(3) Goal-based agent

Plain idea: knowing the current state is not always enough — at a road junction the taxi can turn left, right or go straight. The correct decision depends on **where it is trying to get to**, i.e. its **goal**. The agent combines its model with goal information and asks “what will happen if I do action A?” and “will that bring me closer to my goal?”

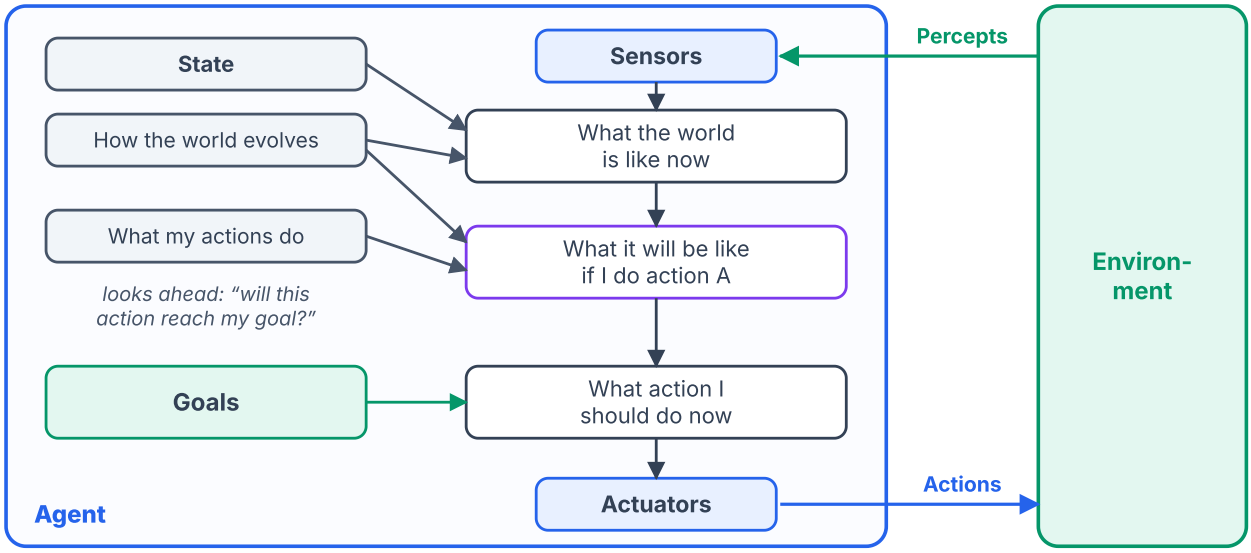


Fig 2.5 A model-based, goal-based agent: it keeps track of the world state and a set of goals, and chooses an action that will (eventually) achieve them.

- Involves **looking into the future** — **search** (Ch 3) and **planning** find action sequences that reach the goal.
- **More flexible** than a reflex agent: the knowledge behind decisions is explicit and can be changed. E.g. if it starts to rain, the agent updates its knowledge of how effectively its brakes work and automatically changes its behaviour; to go to a new destination we just change the goal — no need to rewrite all the rules.
- **Less efficient** than a reflex agent (it has to think ahead).

(4) Utility-based agent

Plain idea: goals only give a crude binary distinction "happy / unhappy". Many routes reach the destination, but some are quicker, safer, more reliable, or cheaper. A **utility function** gives a *number* saying **how happy (desirable)** a state is.

> **DEFINITION**
>
> A **utility function** is an internalization of the performance measure: $U(\text{state}) \rightarrow \text{real number}$. A rational utility-based agent chooses the action that maximizes the expected utility of the action outcomes (the utility it expects to get, on average, given the probabilities of each outcome).

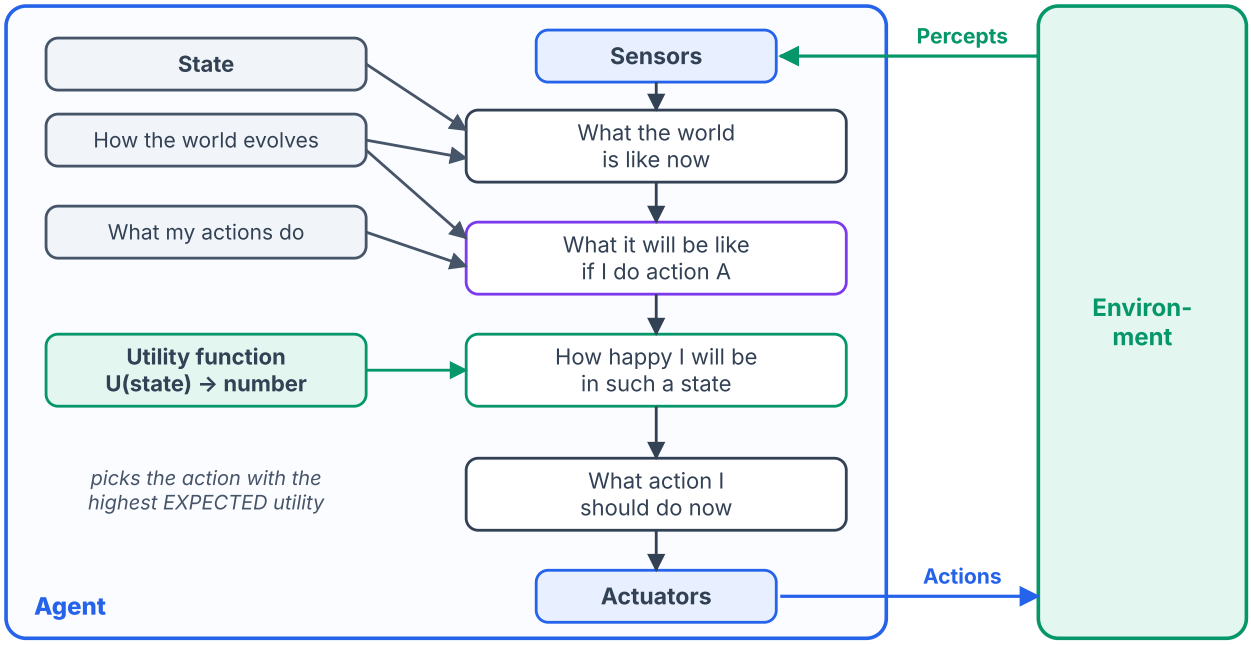


Fig 2.6 A model-based, utility-based agent.

Utility helps where goals are inadequate: (1) when there are **conflicting goals** (speed vs safety) — utility specifies the right trade-off; (2) when there are **several goals, none achievable with certainty** — utility weighs the likelihood of success against the importance of each goal.

(5) Learning agent

Plain idea: any of the above agents can be made to **improve itself with experience**. Learning allows the agent to operate in initially unknown environments and become more competent than its initial knowledge alone would allow. (Turing's proposal: build learning machines and then teach them.)

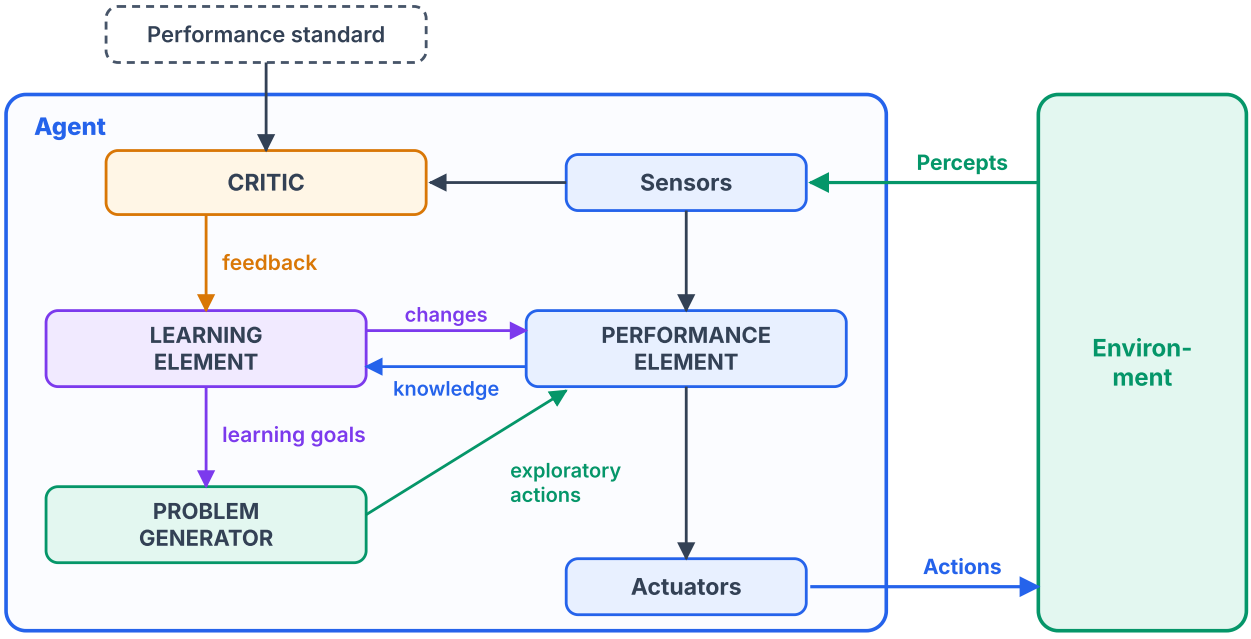


Fig 2.7 A general learning agent with its four conceptual components.

Component	Role	Taxi example
Performance element	Selects external actions (this is the "whole agent" we discussed earlier: takes percepts, decides actions)	Knowledge & procedures for driving
Critic	Tells the learning element how well the agent is doing **with respect to a fixed performance standard** (percepts alone don't say whether the agent is doing well)	Observes other drivers' shocked reactions after a quick left turn across lanes
Learning element	Makes improvements: uses feedback from the critic to decide how the performance element should be modified	Forms a rule "quick left turns across traffic are bad"
Problem generator	Suggests **exploratory actions** that lead to new and informative experiences (may be sub-optimal in the short run, but better in the long run)	"Try the brakes on a different road surface"

Comparison of agent types

Agent type	Uses memory (internal state)?	Decides using	Strength	Weakness / limitation
Table-driven	Whole percept sequence	Table lookup	Trivially correct in theory	Table astronomically large
Simple reflex	No	Condition–action rules on current percept	Fast, simple	Needs fully observable env.; infinite loops
Model-based reflex	Yes (state + model)	Rules on estimated current state	Handles partial observability	No explicit goals; still rule-bound
Goal-based	Yes	Goals + prediction (search/planning)	Flexible; goals easy to change	Slower; goals are only yes/no
Utility-based	Yes	Maximize expected utility	Trade-offs, uncertainty, conflicting goals	Utility hard to design; heavy computation
Learning	Yes	Any of the above + improvement from feedback	Works in unknown environments; improves	Needs experience; exploration costs

How states are represented inside an agent

Representation	What a state looks like	Used by
Atomic	an indivisible "black box" with no internal structure (e.g. just the city name "Arad")	search & game playing (Ch 3), HMMs, MDPs
Factored	a fixed set of variables/attributes, each with a value (fuel = 40%, GPS = ..., light = on)	CSPs, **propositional logic** (Ch 4), planning, Bayesian networks
Structured	objects and the relationships between them (truck *is backing into* driveway, cow *blocking* it)	first-order logic, relational databases, NLP

EXAM TIP — "EXPLAIN THE TYPES OF AGENTS WITH NEAT DIAGRAMS" (10M)

- This is one of the most common 10-mark questions. For **each** type give: one-line idea, **diagram**, pseudocode (at least for simple reflex and model-based), an example, and a limitation.
- Practise drawing the diagrams quickly: **Agent box on the left, Environment box on the right, Percepts arrow in at the top (Sensors), Actions arrow out at the bottom (Actuators)**. Then fill the inside: simple reflex (2 boxes + rules), model-based (+ State, How the world evolves, What my actions do), goal-based (+ What it will be like if I do A, Goals), utility-based (+ How happy I will be, Utility).
- For the learning agent, label all four components and the arrows *feedback, changes, knowledge, learning goals*.

COMMON MISTAKES

- Saying a simple reflex agent has "memory" — it does not; that is the model-based agent.
- Saying the *critic* changes the agent — the critic only gives feedback; the *learning element* makes changes.
- Calling the utility function a goal. Goal = yes/no; utility = degree of happiness (a number).

LIKELY EXAM QUESTIONS — CHAPTER 2

1. Define agent, percept, percept sequence, agent function and agent program. Explain with the vacuum-cleaner world. **(6M)**
2. What is a rational agent? What does rationality depend on? Differentiate rationality and omniscience. **(6M)**
3. Explain autonomy, information gathering and learning with respect to rational agents. **(5M)**
4. Give the PEAS description for an automated taxi driver / medical diagnosis system / part-picking robot / English tutor. **(5M)**
5. Explain the properties of task environments with examples. **(10M)**
6. Classify crossword, chess with a clock, poker, taxi driving and medical diagnosis along all dimensions. **(5M)**
7. Why does the table-driven agent fail? **(4M)**
8. Explain simple reflex and model-based reflex agents with diagrams and pseudocode. **(10M)**
9. Explain goal-based and utility-based agents with diagrams. How do they differ? **(8M)**
10. Explain the learning agent with a neat diagram. **(6M)**

60-SECOND RECAP

- Agent = perceives via **sensors**, acts via **actuators**; agent function (percept sequence → action) vs agent program (code). Agent = architecture + program.
- Rationality depends on **performance measure, prior knowledge, actions, percept sequence**; rational maximizes **expected** performance; rational ≠ omniscient. Needs information gathering, learning, autonomy.
- **PEAS** = Performance, Environment, Actuators, Sensors (taxi example).
- 7 environment dimensions: observable, agents, deterministic, episodic, static, discrete, known.
- Agent types: **simple reflex** (rules on current percept), **model-based** (internal state, transition + sensor model), **goal-based** (look ahead to goal), **utility-based** (maximize expected utility), **learning** (performance element, critic, learning element, problem generator).

3 Problem Solving by Searching

How can an agent find a sequence of actions that reaches its goal? By formulating the problem precisely and then **searching** through possible action sequences. This chapter carries the most marks in Unit I: problem formulation, the six uninformed strategies, A* and heuristics.

3.1 Problem-solving agents

A **problem-solving agent** is a *goal-based agent* that uses **atomic** state representations (a state is just a name such as "Arad"). **Story used throughout:** an agent is on holiday in **Arad** (Romania) and has a non-refundable flight leaving from **Bucharest** tomorrow. It must find a route.

1. **Goal formulation** — decide what you want, based on the current situation and the performance measure: *be in Bucharest*. Goals limit the objectives the agent tries to achieve.
2. **Problem formulation** — decide what *actions* and *states* to consider, given the goal (e.g. "drive from one city to a neighbouring city", not "move the left foot 1 cm").
3. **Search** — simulate sequences of actions in the model until one reaching the goal is found; the result is a **solution** (an action sequence).
4. **Execution** — carry out the actions in the solution one by one.

"**Formulate, search, execute**." Here we assume the environment is fully observable, discrete, known and deterministic, so the solution is a fixed sequence of actions and the agent can execute it with its "eyes closed" (an **open-loop** system: percepts are ignored while executing).

```
function SIMPLE-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq, an action sequence, initially empty
              state, some description of the current world state
              goal, a goal, initially null
              problem, a problem formulation
  state ← UPDATE-STATE(state, percept)
  if seq is empty then
      goal    ← FORMULATE-GOAL(state)
      problem ← FORMULATE-PROBLEM(state, goal)
      seq     ← SEARCH(problem)
      if seq = failure then return a null action
  action ← FIRST(seq)
  seq    ← REST(seq)
  return action
```

Well-defined problems and solutions ★ Very likely

THE 5 COMPONENTS OF A PROBLEM (LEARN THIS LIST)

1. **Initial state** — the state the agent starts in. *In(Arad)*.
2. **Actions** — ACTIONS(*s*) returns the set of actions applicable in state *s*. From *In(Arad)*: {*Go(Sibiu)*, *Go(Timisoara)*, *Go(Zerind)*}.
3. **Transition model** — RESULT(*s*, *a*) returns the state that results from doing action *a* in state *s* (its **successor**). RESULT(*In(Arad)*, *Go(Zerind)*) = *In(Zerind)*.
4. **Goal test** — decides whether a given state is a goal. Either an explicit set ({*In(Bucharest)*}) or an abstract property ("checkmate" in chess).
5. **Path cost** — a function that assigns a numeric cost to each path; it is the sum of **step costs** *c*(*s*, *a*, *s'*) (assumed ≥ 0). In Romania: distance in km.

State space	the set of all states reachable from the initial state by any sequence of actions; it forms a **directed graph** (nodes = states, links = actions). Initial state + actions + transition model together define it implicitly.
Path	a sequence of states connected by a sequence of actions.
Solution	an action sequence that leads from the initial state to a goal state.
Optimal solution	the solution with the lowest path cost among all solutions.
Abstraction	removing irrelevant detail from the representation (weather, radio, scenery). Valid if every abstract solution can be expanded into a real one; useful if the abstract actions are easier than the original problem.

MEMORY TRICK — 5 COMPONENTS

"I Act To Get Paid" → **I**nitial state, **A**ctions, **T**ransition model, **G**oal test, **P**ath cost.

3.2 Example problems

Toy problems illustrate or exercise methods; they have concise, exact descriptions so researchers can compare algorithms. **Real-world problems** are ones people actually care about; they have no single agreed description.

(1) Vacuum world ★ Likely

Component	Formulation
States	agent location × dirt status of each square: $2 \times 2^2 = 8$ states (with n squares: $n \cdot 2^n$)
Initial state	any state can be the initial state
Actions	*Left*, *Right*, *Suck*
Transition model	actions have their expected effects, except: *Left* in the leftmost square, *Right* in the rightmost square and *Suck* in a clean square have no effect
Goal test	all squares are clean
Path cost	each step costs 1

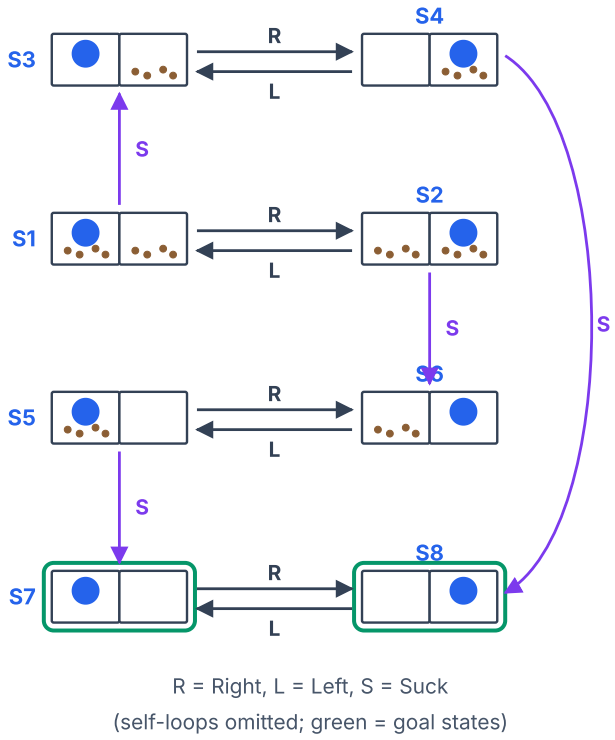


Fig 3.1 State space of the vacuum world (S7, S8 are goals).

(2) The 8-puzzle ★ Likely

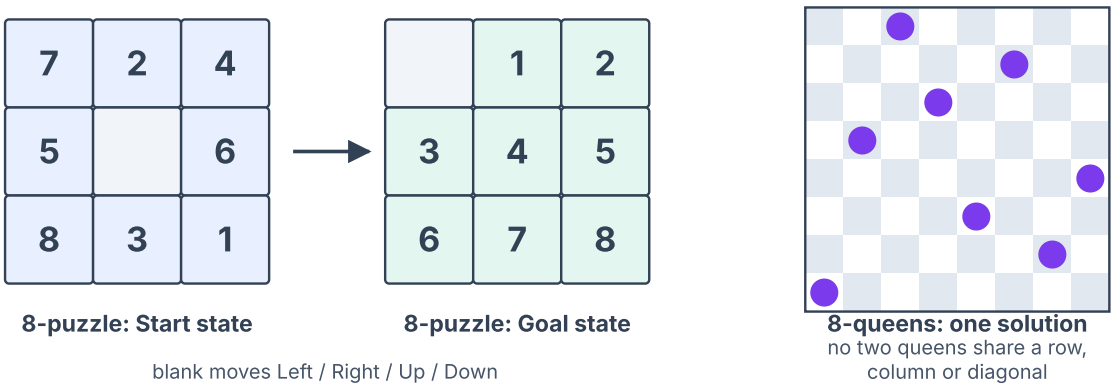


Fig 3.2 A typical 8-puzzle instance (start and goal) and one solution of the 8-queens problem.

A 3×3 board with 8 numbered tiles and one blank. A tile next to the blank can slide into it. Goal: reach the goal configuration.

Component	Formulation
States	location of each of the 8 tiles and the blank in the 9 squares
Initial state	any state (any given goal can be reached from exactly **half** of the possible initial states)
Actions	movements of the **blank**: *Left, Right, Up, Down* (fewer when the blank is at an edge or corner)
Transition model	returns the new board; e.g. applying *Left* to the start state above swaps the 5 and the blank
Goal test	does the state match the goal configuration?
Path cost	each step costs 1 (path cost = number of moves)

Size: the 8-puzzle has 9!/2 = 181,440 reachable states; the 15-puzzle (4×4) about 1.3 trillion; the 24-puzzle (5×5) about 10^{25}. The sliding-block family is **NP-complete**, a standard test bed for heuristic search.

(3) The 8-queens problem

Place 8 queens on a chessboard so that **no queen attacks another** (no two in the same row, column or diagonal). Two kinds of formulation:

- **Incremental formulation** — operators *add* a queen to the state, starting from an empty board.
- **Complete-state formulation** — start with all 8 queens on the board and *move* them around (used by local search).

	Naive incremental formulation	Better incremental formulation
States	any arrangement of 0–8 queens on the board	arrangements of n queens ($0 \le n \le 8$), one per column in the leftmost n columns, none attacking another
Initial state	no queens on the board	no queens on the board
Actions	add a queen to any empty square	add a queen to any square in the leftmost empty column such that it is not attacked
Transition / goal	returns the board with the queen added / 8 queens on the board, none attacked. (Path cost is irrelevant: only the final state matters.)	
Size	64 × 63 × ... × 57 ≈ **1.8×10^{14}** sequences	only **2,057** states — good formulation matters!

(4) Route finding in Romania ★ Very likely

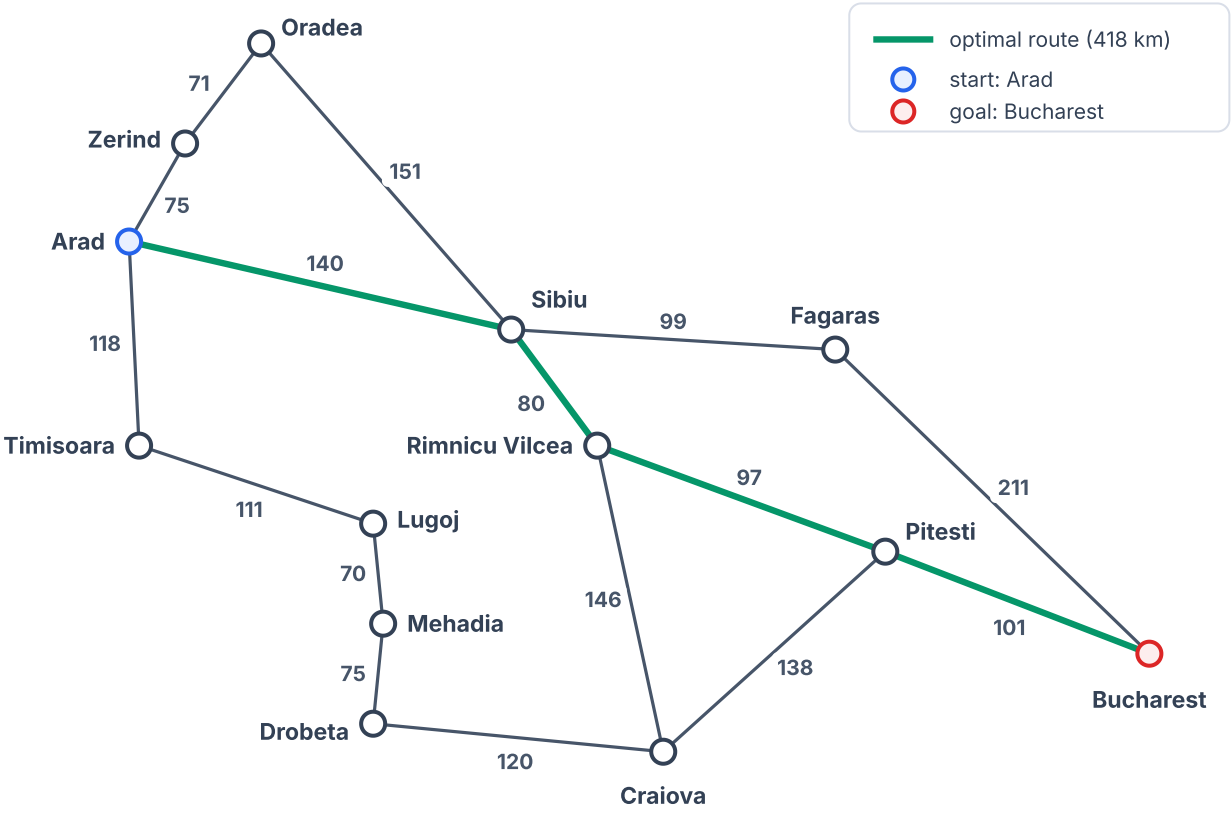


Fig 3.3 Part of the road map of Romania (distances in km). Used for greedy best-first search and A.*

Component	Romania formulation
Initial state	*In(Arad)*
Actions	*Go(city)* for each neighbouring city
Transition	RESULT(*In(x)*, *Go(y)*) = *In(y)*
Goal test	state = *In(Bucharest)*
Path cost	sum of road distances (km)

Straight-line distance to Bucharest, $h_{SLD}(n)$			
Arad	**366**	Mehadia	**241**
Bucharest	**0**	Oradea	**380**
Craiova	**160**	Pitesti	**100**
Drobeta	**242**	Rimnicu Vilcea	**193**
Fagaras	**176**	Sibiu	**253**
Lugoj	**244**	Timisoara	**329**
		Zerind	**374**

Real-world problems (list with one line each)

Route finding	car navigation, routing video streams in networks, military operations planning, **airline travel planning** (state = location + current time + fare bases + domestic/international; cost = money, waiting time, flight time, seat quality ...)
Touring problems	visit every city at least once and return; the **travelling salesperson problem (TSP)**: visit each city exactly once with the shortest tour (NP-hard)
VLSI layout	position millions of components and connections on a chip (cell layout, channel routing) to minimise area, delays, stray capacitances
Robot navigation	route finding in continuous space with many degrees of freedom
Automatic assembly sequencing	find an order in which to assemble the parts of an object; related: protein design (sequence of amino acids that folds correctly)
Internet searching	software agents looking for answers to questions or related information

EXAM TIP — "FORMULATE THE PROBLEM" QUESTIONS (5M)

Always answer with the **5 components** as a table: **States, Initial state, Actions, Transition model, Goal test, Path cost** (states first, as the textbook does). Add the **state-space size** if you can (8 for vacuum, 181,440 for 8-puzzle, 2,057 for good 8-queens). New problems (water jug, missionaries & cannibals, tic-tac-toe) are answered the same way.

3.3 Searching for solutions

A solution is an action sequence, so search algorithms consider many possible action sequences. These form a **search tree**:

- **Root** = initial state; **branches** = actions; **nodes** = states in the state space.
- **Expanding** a node = applying every legal action to its state, thereby **generating** a new set of states (its **child** nodes; it is their **parent**).
- **Leaf node** = a node with no children (yet). The **frontier** (open list) = the set of all leaf nodes available for expansion.
- **Search strategy** = the rule that decides *which frontier node to expand next*. This is the only thing that differs between BFS, DFS, UCS, A* ...

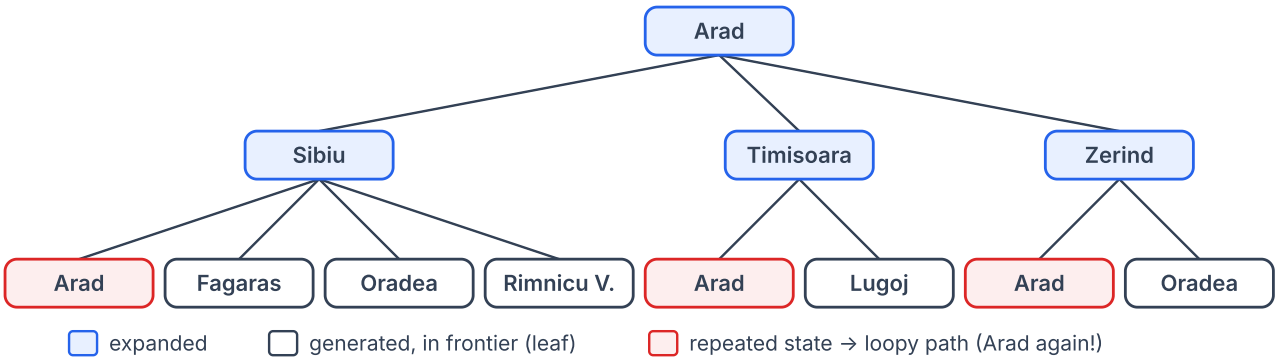


Fig 3.4 Partial search tree for Arad → Bucharest after expanding Arad, then Sibiu, Timisoara and Zerind. Notice Arad appearing again: a loopy path.

Repeated states. The tree contains **Arad → Sibiu → Arad**, a **loopy path**; because of loops the search tree can be *infinite* even though the state space has only 20 cities. Loopy paths are a special case of **redundant paths** (more than one way to reach the same state, e.g. Arad–Sibiu (140 km) vs Arad–Zerind–Oradea–Sibiu (297 km)). "Algorithms that forget their history are doomed to repeat it." The fix is an **explored set** (closed list) that remembers every expanded node.

```
function TREE-SEARCH(problem) returns a
solution, or failure
  initialize the frontier using the initial
     state of problem
  loop do
    if the frontier is empty then return
failure
    choose a leaf node and remove it from
       the frontier
    if the node contains a goal state then
       return the corresponding solution
    expand the chosen node, adding the
       resulting nodes to the frontier
```

```
function GRAPH-SEARCH(problem) returns a
solution, or failure
  initialize the frontier using the initial
     state of problem
  initialize the explored set to be empty
  loop do
    if the frontier is empty then return
failure
    choose a leaf node and remove it from the
frontier
    if the node contains a goal state then
       return the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the
resulting
       nodes to the frontier only if not in
the
       frontier or explored set
```

Tree search may revisit states (simple, little memory, but may loop); **graph search** never expands a state twice. The frontier *separates* the explored region of the state space from the unexplored region.

Infrastructure: nodes and queues

Field of node *n*	Meaning
`n.STATE`	the state this node corresponds to
`n.PARENT`	the node that generated it
`n.ACTION`	the action applied to the parent
`n.PATH-COST`	$g(n)$, cost of the path from the root to n

Following PARENT pointers back from a goal node gives the solution path. A **node** is a bookkeeping data structure; a **state** is a world configuration — two different nodes can contain the same state.

```
function CHILD-NODE(problem, parent, action)
         returns a node
  return a node with
    STATE  = problem.RESULT(parent.STATE,
action),
    PARENT = parent, ACTION = action,
    PATH-COST = parent.PATH-COST +
        problem.STEP-COST(parent.STATE,
action)
```

Frontier = a **queue** with EMPTY?, POP, INSERT. Types: **FIFO** queue (pops oldest), **LIFO** queue / stack (pops newest), **priority queue** (pops the element with the best priority).

Measuring problem-solving performance ★ Very likely

- **Completeness** — is the algorithm guaranteed to find a solution when there is one?
- **Optimality** — does it find the optimal (lowest path cost) solution?
- **Time complexity** — how long does it take? (number of nodes generated)
- **Space complexity** — how much memory? (max nodes stored)
- ***b*** = **branching factor**: maximum number of successors of any node.
- ***d*** = **depth** of the *shallowest* goal node (number of steps from the root).
- ***m*** = **maximum length** of any path in the state space.
- **Search cost** (time, memory) + **path cost** of the solution = **total cost**.

MEMORY TRICK

"COST" → **C**ompleteness, **O**ptimality, **S**pace, **T**ime. And **b** = **b**readth of the tree, **d** = **d**epth of the goal, **m** = **m**aximum depth.

3.4 Uninformed (blind) search strategies ★ Very likely

Uninformed strategies have no information about states beyond the problem definition: all they can do is generate successors and tell a goal state from a non-goal state. They differ only in the **order** in which nodes are expanded. (This section is a self-study topic in your syllabus, so it is explained slowly with one example for all six strategies.)

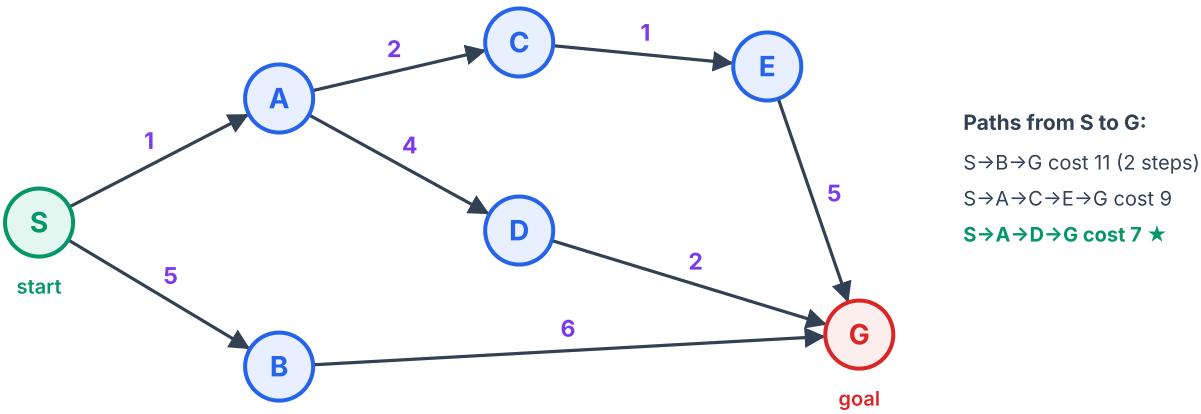


Fig 3.5 Our example graph for all uninformed searches: start S, goal G, one-way roads with step costs.

CONVENTIONS FOR EVERY TRACE BELOW

- Children are generated in **alphabetical order**; ties in a priority queue are broken alphabetically.
- We use **graph search** (a state already expanded or already in the frontier is not added again, except UCS may *replace* a frontier entry with a cheaper one).
- **BFS** tests for the goal when a node is **generated** (textbook version); all other strategies test when a node is **popped** for expansion.

(1) Breadth-first search (BFS)

Idea: expand the shallowest unexpanded node first — the root, then all its children, then all their children … level by level, like ripples on water. **Frontier:** a **FIFO queue** (new nodes go to the back).

```
function BREADTH-FIRST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  frontier ← a FIFO queue with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)              /* the shallowest node in frontier */
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        if problem.GOAL-TEST(child.STATE) then return SOLUTION(child)
        frontier ← INSERT(child, frontier)
```

Step	Pop & expand	Children generated (goal-tested now)	Frontier after (front → back)	Explored
0	—	S is not the goal	[S]	{ }
1	S	A, B	[A, B]	{S}
2	A	C, D	[B, C, D]	{S, A}
3	B	**G — goal test succeeds ✔**	stop	{S, A, B}

Result: path **S → B → G**, cost 5 + 6 = **11**, depth 2. BFS found the *shallowest* goal (fewest steps) but not the cheapest (cost 7 exists).

Properties. Complete if b is finite (it reaches every depth). **Optimal** only if path cost is a non-decreasing function of depth, e.g. **all step costs equal**. **Time:** nodes generated $b + b^2 + \ldots + b^d = O(b^d)$ (it would be $O(b^{d+1})$ if the goal test were applied on expansion). **Space:** $O(b^d)$ — every generated node stays in memory ($O(b^{d-1})$ explored + $O(b^d)$ frontier).

Depth d	Nodes	Time	Memory
2	110	0.11 ms	107 KB
6	10^6	1.1 s	1 GB
10	10^{10}	3 hours	10 TB
14	10^{14}	3.5 years	99 PB
16	10^{16}	350 years	10 EB

BFS cost with b = 10, 1 million nodes/second, 1000 bytes/node (textbook figures). Two lessons: **memory is a bigger problem than time**, and exponential problems can't be solved by uninformed search except for small instances.

(2) Uniform-cost search (UCS)

Idea: expand the node with the lowest path cost $g(n)$ first — like BFS, but measuring distance in cost instead of steps (Dijkstra's idea). **Frontier:** a **priority queue ordered by g**. Two changes from BFS:

1. The goal test is applied when a node is **selected for expansion**, not when generated (the first goal generated may be on a sub-optimal path).
2. If a **cheaper path** to a node already in the frontier is found, the frontier node is **replaced**.

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
  node ← a node with STATE = problem.INITIAL-STATE, PATH-COST = 0
  frontier ← a priority queue ordered by PATH-COST, with node as the only element
  explored ← an empty set
  loop do
    if EMPTY?(frontier) then return failure
    node ← POP(frontier)                /* the lowest-cost node in frontier */
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    add node.STATE to explored
    for each action in problem.ACTIONS(node.STATE) do
      child ← CHILD-NODE(problem, node, action)
      if child.STATE is not in explored or frontier then
        frontier ← INSERT(child, frontier)
      else if child.STATE is in frontier with higher PATH-COST then
        replace that frontier node with child
```

Step	Pop (*g*)	Goal?	Children added (*g* = cost so far)	Frontier after (lowest *g* first)
0	—			{S:0}
1	S:0	no	A:1, B:5	{A:1, B:5}
2	A:1	no	C:1+2=3, D:1+4=5	{C:3, B:5, D:5}
3	C:3	no	E:3+1=4	{E:4, B:5, D:5}
4	E:4	no	G:4+5=9 (generated, **not** returned yet)	{B:5, D:5, G:9}
5	B:5	no	G:5+6=11 — worse than G:9 already in frontier ⇒ ignore	{D:5, G:9}
6	D:5	no	G:5+2=7 — **cheaper than G:9 ⇒ replace**	{G:7}
7	G:7	**YES ✔**	stop	

Result: S → A → D → G, cost 1 + 4 + 2 = **7** — the optimal path. Had we tested G when it was first generated (step 4), we would have returned the cost-9 path.

Properties. Complete if every step cost ≥ ε for some small positive ε (zero-cost loops such as repeated *NoOp* could otherwise trap it). **Optimal**: yes — nodes are expanded in order of their optimal path cost. **Time and space:** $O(b^{1 + \lfloor C^*/\varepsilon \rfloor})$, where C^* = cost of the optimal solution; this can be much larger than b^d because UCS may explore large trees of cheap steps first. With all step costs equal it becomes $O(b^{d+1})$.

(3) Depth-first search (DFS)

Idea: expand the deepest unexpanded node first — go down one branch as far as possible, then back up (*backtrack*) to the next branch. Like exploring a maze by always taking the first unexplored corridor. **Frontier:** a **LIFO queue (stack)**: the most recently generated node is chosen first.

Step	Pop	Goal?	Push	Stack after (top first)
0				[S]
1	S	no	A, B	[A, B]
2	A	no	C, D	[C, D, B]
3	C	no	E	[E, D, B]
4	E	no	G	[G, D, B]
5	G	**YES ✔**		stop

Result: S → A → C → E → G, cost 1+2+1+5 = **9**, depth 4. Neither the cheapest (7) nor the shallowest (2) — DFS simply returns the *first* goal it bumps into.

Implementation: GRAPH-SEARCH with a LIFO queue, or (more commonly) a recursive function that calls itself on each child in turn.

Properties. Complete? The graph-search version is complete in *finite* state spaces; the tree-search version is **not** (it can loop forever, e.g. Arad–Sibiu–Arad–Sibiu …); both fail in infinite state spaces (they can go down an infinite path that never reaches the goal). **Optimal?** No. **Time:** $O(b^m)$ — terrible if $m >> d$. **Space:** only $O(bm)$ for tree search — it stores just the current path plus the unexpanded siblings of each node on it. This is DFS's big advantage: at d = 16 it needs about 156 KB instead of BFS's 10 exabytes. (**Backtracking search**, which generates one successor at a time, needs only $O(m)$.)

(4) Depth-limited search (DLS)

Idea: DFS with a **depth limit *l***: nodes at depth *l* are treated as if they have no successors. This removes the infinite-path problem. DFS is the special case $l = \infty$.

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
  return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
  if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
  else if limit = 0 then return cutoff
  else
    cutoff_occurred? ← false
    for each action in problem.ACTIONS(node.STATE) do
      child  ← CHILD-NODE(problem, node, action)
      result ← RECURSIVE-DLS(child, problem, limit − 1)
      if result = cutoff then cutoff_occurred? ← true
      else if result ≠ failure then return result
    if cutoff_occurred? then return cutoff else return failure
```

```
l = 1:  S(d0) → A(d1, limit reached: cutoff) → B(d1, cutoff)              ⇒ returns CUTOFF
(no answer within depth 1)
l = 2:  S(d0) → A(d1) → C(d2, cutoff) → D(d2, cutoff) → B(d1) → G(d2) ✔  ⇒ returns S→B→G
(cost 11)
```

Two kinds of failure: *failure* = no solution at all; *cutoff* = no solution *within the depth limit*. **Properties: complete only if *l* ≥ *d*** (incomplete if $l < d$); **not optimal** (if $l > d$ it may return a deeper, costlier goal); time $O(b^l)$; space $O(bl)$. **Choosing *l*:** use problem knowledge — Romania has 20 cities, so l = 19 is safe; better, any city is reachable from any other in at most 9 steps (the **diameter** of the state space), so l = 9.

(5) Iterative deepening search (IDS)

Idea: run DLS with l = 0, 1, 2, 3, … until a goal is found. It combines the small memory of DFS with the completeness (and shallowest-goal optimality) of BFS.

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
  for depth = 0 to ∞ do
    result ← DEPTH-LIMITED-SEARCH(problem, depth)
    if result ≠ cutoff then return result
```

Limit *l*	Nodes visited, in order	Result of this iteration
0	S	cutoff
1	S, A, B	cutoff
2	S, A, C, D, B, **G**	**found S → B → G (depth 2, cost 11)**

"Isn't repeating the top levels wasteful?" Not much: most nodes are at the bottom level. Nodes at depth d are generated once, depth d−1 twice, …, depth 1 d times:

$$N(\text{IDS}) = (d)b + (d-1)b^2 + \ldots + (1)b^d = O(b^d)$$

b = 10, d = 5: N(IDS) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450 vs N(BFS) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110 (only about 11% more)

Properties: complete (if b finite); optimal if all step costs are equal; time $O(b^d)$; space $O(bd)$. IDS is the preferred uninformed method when the search space is large and the depth of the solution is not known.

(6) Bidirectional search

Idea: run **two simultaneous searches** — one **forward** from the initial state and one **backward** from the goal — and stop when the two frontiers **meet**. Like two tunnel crews digging from both sides of a mountain. Motivation: $b^{d/2} + b^{d/2}$ is much less than b^d. E.g. b = 10, d = 6: about 2,220 nodes instead of 1,111,110 for BFS.

Round	Forward search from S	Backward search from G (follow arrows in reverse)	Frontiers intersect?
0	{S}	{G}	no
1	expand S → {A, B}	expand G → predecessors {B, D, E}	**yes, at B ✔**

Result: S → B (forward half) + B → G (backward half) = **S → B → G**. Each search went only 1 level instead of 2. **Properties:** complete and optimal (for equal step costs) if both directions use BFS; time and space $O(b^{d/2})$. **Limitations:** (i) we must be able to compute **predecessors** (reverse actions); (ii) the goal must be an explicit state — hard for abstract goals like "checkmate"; (iii) one frontier must be kept in memory to check intersections, so space is the weak point. The goal test is replaced by "do the frontiers intersect?".

Comparison of uninformed strategies ★ Very likely

Criterion	BFS	Uniform-cost	DFS	Depth-limited	Iterative deepening	Bidirectional
Frontier	FIFO queue	priority queue (g)	LIFO stack	LIFO stack	LIFO stack	two queues
Complete?	Yes[a]	Yes[a,b]	No	No (yes if $l \geq d$)	Yes[a]	Yes[a,d]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\varepsilon \rfloor})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes[c]	Yes	No	No	Yes[c]	Yes[c,d]
On our graph	S-B-G (11)	S-A-D-G (7)	S-A-C-E-G (9)	l=2: S-B-G (11)	S-B-G (11)	S-B-G (11)

[a] complete if b is finite; [b] complete if step costs ≥ ε for positive ε; [c] optimal if step costs are all identical; [d] if both directions use breadth-first search. (Tree-search versions; b = branching factor, d = depth of shallowest solution, m = maximum depth, l = depth limit, C^* = optimal cost.)

EXAM TIP — HOW TO WRITE A SEARCH TRACE FOR FULL MARKS

- Draw the graph first, then a table with columns **Step | Node expanded | Frontier | Explored** (add *g* or *f* values for UCS/A*).
- State the data structure (**FIFO / LIFO / priority queue**) and your tie-breaking rule (alphabetical).
- Finish with the **path**, its **cost**, and a one-line comment ("BFS found the shallowest, not the cheapest, path").
- For a "compare" question, reproduce the comparison table with footnotes — examiners check the complexities.

COMMON MISTAKES

- **"BFS is always optimal."** Only when all step costs are equal; otherwise use UCS.
- **"DFS is complete."** Not in infinite spaces (or tree search with loops). DFS is never optimal.
- Doing the goal test on **generation** in UCS/A* — it must be done when the node is **popped**.
- Writing DFS space as $O(b^m)$ — that is its *time*; space is $O(bm)$.
- Confusing *d* (depth of the shallowest goal) with *m* (maximum depth of the space).

3.5 Informed (heuristic) search strategies

Informed search uses problem-specific knowledge beyond the problem definition to find solutions more efficiently. The general approach is **best-first search**: a graph/tree search in which the node selected for expansion is the one with the lowest value of an **evaluation function *f*(*n*)** (a cost estimate). It is identical to UCS except that *f* replaces *g* in the priority queue. Most best-first algorithms include a heuristic function in *f*.

DEFINITION — HEURISTIC FUNCTION

$h(n)$ = estimated cost of the cheapest path from the state at node *n* to a goal state. It depends only on the state, is non-negative, and $h(n) = 0$ if *n* is a goal. *Example:* h_{SLD} = straight-line distance to Bucharest (you can't compute it from the problem description; it is extra knowledge).

Greedy best-first search

Idea: expand the node that appears closest to the goal: $f(n) = h(n)$. "Always walk towards the city that looks nearest to Bucharest as the crow flies."

Step	Expand (*h*)	Children added (*h*)	Frontier after (lowest *h* first)
1	Arad 366	Sibiu 253, Timisoara 329, Zerind 374	Sibiu 253, Timisoara 329, Zerind 374
2	Sibiu 253	Arad 366, Fagaras 176, Oradea 380, Rimnicu Vilcea 193	Fagaras 176, R.Vilcea 193, Timisoara 329, Arad 366, Zerind 374, Oradea 380
3	Fagaras 176	Sibiu 253, Bucharest 0	Bucharest 0, R.Vilcea 193, Sibiu 253, Timisoara 329, …
4	Bucharest 0	**goal ✔** Path Arad → Sibiu → Fagaras → Bucharest = 140 + 99 + 211 = **450 km**	

Very fast (it never expanded a node off the solution path) but not optimal: the route via Rimnicu Vilcea and Pitesti is 418 km, 32 km shorter. **Properties: not optimal**; the tree-search version is **incomplete** even in a finite space (from Iasi to Fagaras it goes to the dead end Neamt, back to Iasi, Neamt again … forever); the graph-search version is complete in finite spaces. Worst-case time and space $O(b^m)$, but a good heuristic reduces this dramatically.

A* search: minimizing the total estimated solution cost ★ Very likely

$$f(n) = g(n) + h(n)$$

$g(n)$ = cost so far to reach n · $h(n)$ = estimated cost from n to the goal · $f(n)$ = estimated cost of the cheapest solution **through** n

A* is identical to uniform-cost search except that it orders the priority queue by $g + h$ instead of g. It combines UCS (exact cost so far, keeps it honest) with greedy search (estimate of the rest, keeps it focused).

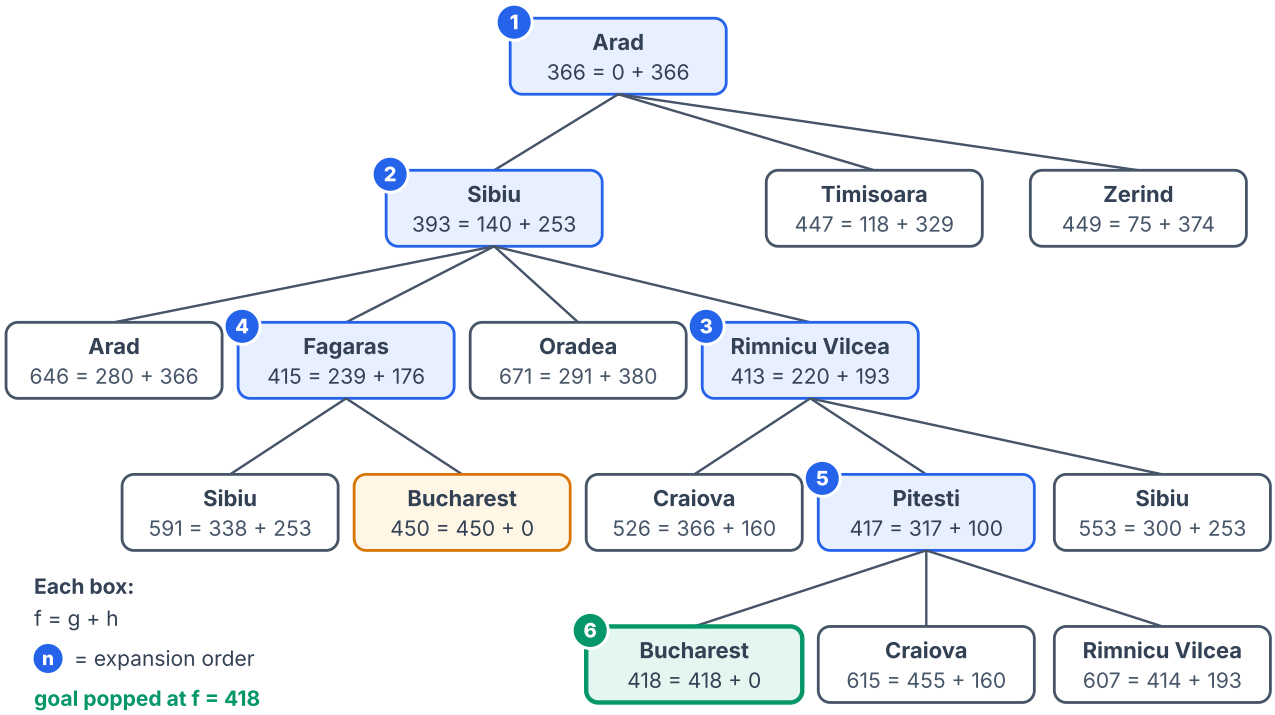


Fig 3.6 A search tree for Arad → Bucharest with h_{SLD}. Numbers in circles give the order of expansion.*

Step	Expand (f)	New nodes: $f = g + h$	Frontier after (lowest f first)
1	Arad 366	Sibiu 393 = 140+253; Timisoara 447 = 118+329; Zerind 449 = 75+374	Sibiu 393, Timisoara 447, Zerind 449
2	Sibiu 393	Arad 646 = 280+366; Fagaras 415 = 239+176; Oradea 671 = 291+380; R.Vilcea 413 = 220+193	R.Vilcea 413, Fagaras 415, Timisoara 447, Zerind 449, Arad 646, Oradea 671
3	R.Vilcea 413	Craiova 526 = 366+160; Pitesti 417 = 317+100; Sibiu 553 = 300+253	Fagaras 415, Pitesti 417, Timisoara 447, Zerind 449, Craiova 526, Sibiu 553, Arad 646, Oradea 671
4	Fagaras 415	Sibiu 591 = 338+253; **Bucharest 450 = 450+0** (goal generated, but not popped)	Pitesti 417, Timisoara 447, Zerind 449, Bucharest 450, Craiova 526, Sibiu 553, Sibiu 591, …
5	Pitesti 417	**Bucharest 418 = 418+0**; Craiova 615 = 455+160; R.Vilcea 607 = 414+193	Bucharest 418, Timisoara 447, Zerind 449, Bucharest 450, Craiova 526, …
6	**Pop Bucharest (418) — goal ✔** Path: Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest = 140 + 80 + 97 + 101 = 418 km (optimal)		

Why didn't A* stop at step 4? Bucharest (450) was only *generated*; Pitesti had a lower f (417), meaning a cheaper route through Pitesti might exist — and it did. (This trace is the textbook's tree-search version; graph search would simply skip the repeated states Arad, Sibiu, R.Vilcea and give the same answer.) Note also that Timisoara (447) and Zerind (449) are **never expanded**: their f exceeds 418, so their whole subtrees are **pruned**.

Conditions for optimality: admissibility and consistency

ADMISSIBLE HEURISTIC

$h(n)$ is **admissible** if it never overestimates the true cost to reach the goal:

$$0 \le h(n) \le h^*(n) \text{ for every } n$$

(h^* = true optimal cost.) Admissible heuristics are **optimistic**. h_{SLD} is admissible because the straight line is the shortest path between two points, so it can never be an overestimate.

CONSISTENT (MONOTONE) HEURISTIC

For every node n and every successor n' generated by action a:

$$h(n) \le c(n, a, n') + h(n')$$

This is the **triangle inequality**: one side of the triangle ($n \to$ G) can't be longer than the sum of the other two. E.g. h(Sibiu) = 253 ≤ 80 + h(R.Vilcea) = 80 + 193 = 273 ✔.

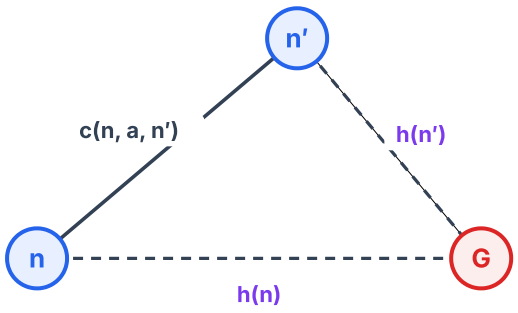


Fig 3.7 Triangle inequality behind consistency.

- Every consistent heuristic is also admissible (but a few admissible heuristics are not consistent). h_{SLD} is consistent.
- **Theorem:** the tree-search version of A* is optimal if h is admissible; the graph-search version is optimal if h is consistent.
- If h is consistent, the values of f along any path are **non-decreasing**: $f(n') = g(n) + c(n,a,n') + h(n') \ge g(n) + h(n) = f(n)$. So the sequence of nodes expanded by A* is in non-decreasing order of f, and the first goal selected for expansion must be optimal.

PROOF SKETCH (10M ANSWERS) — A* WITH AN ADMISSIBLE H IS OPTIMAL (TREE SEARCH)

Let C^* be the optimal cost. Suppose a sub-optimal goal G_2 (cost $C_2 > C^*$) is in the frontier. Let n be a frontier node on an optimal path (one always exists until the goal is found).

1. $f(G_2) = g(G_2) + 0 = C_2 > C^*$ (since h(goal) = 0).
2. $f(n) = g(n) + h(n) \le g(n) + h^*(n) = C^*$ (admissibility).
3. So $f(n) \le C^* < f(G_2)$: A* always expands n before G_2, so it never returns the sub-optimal goal. ∎

Other properties of A*. A* expands **all** nodes with $f(n) < C^*$, **some** nodes on the "goal contour" $f(n) = C^*$, and **no** nodes with $f(n) > C^*$ (think of contour lines on a map spreading out from the start). It is **complete** if there are finitely many nodes with $f \le C^*$ (true if step costs ≥ ε and b is finite), and **optimally efficient**: no other optimal algorithm using the same heuristic is guaranteed to expand fewer nodes. **Drawback:** the number of nodes is still exponential in the worst case, and because A* keeps all generated nodes in memory, it usually runs out of space long before it runs out of time.

Memory-bounded variant	Idea in one line
IDA* (iterative-deepening A*)	like IDS, but the cutoff is the *f*-cost (*g* + *h*) instead of depth; each iteration's cutoff = smallest *f* that exceeded the previous cutoff.
RBFS (recursive best-first search)	recursive, linear-space best-first search; keeps the *f*-value of the best alternative path (*f_limit*) and backtracks when the current node exceeds it, backing up the best child's *f*-value.
MA* / SMA* (simplified memory-bounded A*)	runs A* until memory is full, then drops the **worst** leaf (highest *f*) and backs its value up to its parent, so it can be regenerated later if needed.

COMMON MISTAKES

- **"Greedy best-first is optimal."** No — it ignores $g(n)$ (Arad→Bucharest: 450 km instead of 418).
- **An admissible *h* must never OVERestimate** (underestimating is fine). $h(n) = 0$ for all n is admissible (A* becomes UCS).
- Stopping A* when the goal is *generated*; it stops only when the goal is *popped* (step 6, not step 4).
- Arithmetic slips in *g*: always add the road length to the **parent's** *g*, e.g. *g*(Pitesti) = 220 + 97 = 317.

3.6 Heuristic functions ★ Very likely

The 8-puzzle facts: average solution cost of a random instance is about **22 steps**, branching factor about **3** (blank in the middle: 4 moves; on an edge: 3; in a corner: 2). Exhaustive tree search to depth 22 would examine about $3^{22} \approx 3.1 \times 10^{10}$ states; graph search cuts this to 181,440 distinct states (the 15-puzzle has about 10^{13}), so we need a good heuristic that **never overestimates**.

H_1 = NUMBER OF MISPLACED TILES

Count the tiles (not the blank) that are not in their goal position. **Admissible** because every misplaced tile must be moved at least once.

H_2 = SUM OF MANHATTAN DISTANCES

For each tile, add its horizontal + vertical distance from its goal square (tiles can't move diagonally) — also called **city-block distance**. **Admissible** because one move can bring only one tile one step closer.

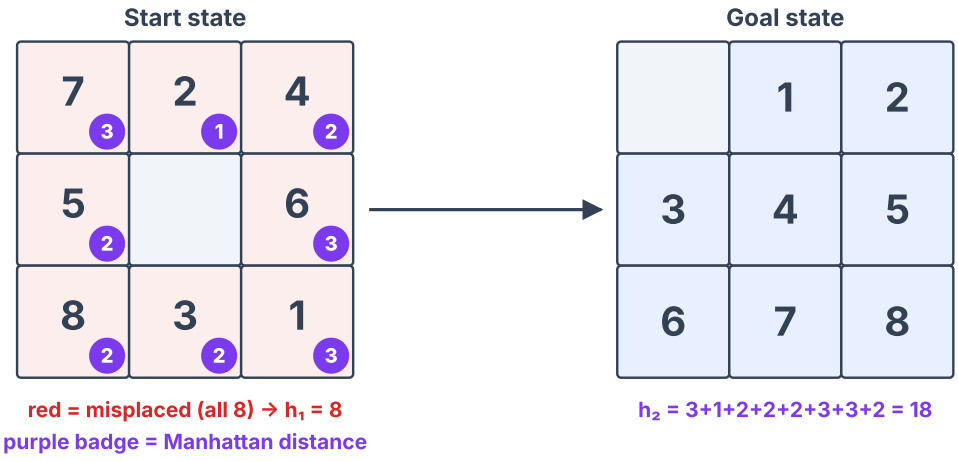


Fig 3.8 Computing h_1 and h_2 for the standard start state.

WORKED EXAMPLE — H_1 AND H_2 FOR START 7 2 4 / 5 _ 6 / 8 3 1 → GOAL _ 1 2 / 3 4 5 / 6 7 8

Tile	1	2	3	4	5	6	7	8	Total
Misplaced?	yes	yes	yes	yes	yes	yes	yes	yes	$h_1 = 8$
Manhattan distance	3	1	2	2	2	3	3	2	$h_2 = 18$

E.g. tile 1 is at row 3, column 3 and belongs at row 1, column 2: $|3-1| + |3-2| = 2 + 1 = 3$. Tile 7 is at row 1, column 1 and belongs at row 3, column 2: $2 + 1 = 3$. The true optimal solution cost is **26**, so both $8 \le 26$ and $18 \le 26$ are underestimates, as admissibility requires.

Effective branching factor b^* (quality of a heuristic)

If A* generates N nodes in total and the solution depth is d, then b^* is the branching factor that a **uniform tree** of depth d would need in order to contain $N + 1$ nodes:

$$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$$

Easy example: $d = 2$, $N = 6 \Rightarrow 7 = 1 + b^* + (b^*)^2 \Rightarrow (b^*)^2 + b^* - 6 = 0 \Rightarrow (b^* + 3)(b^* - 2) = 0 \Rightarrow$ **$b^* = 2$**

Textbook example: solution at depth 5 using 52 nodes $\Rightarrow 53 = 1 + b^* + \ldots + (b^*)^5 \Rightarrow$ **$b^* \approx 1.92$** (check: $1 + 1.92 + 3.69 + 7.08 + 13.59 + 26.09 \approx 53.4$)

A well-designed heuristic has b^* close to 1, so fairly large problems can be solved. In the textbook's experiment at solution depth 12, IDS generated about 3.6 million nodes, A* with h_1 about 227 and A* with h_2 only about 73.

Dominance

If $h_2(n) \ge h_1(n)$ for every node n (both admissible), we say **h_2 dominates h_1**. Then A* using h_2 will **never expand more nodes** than A* using h_1 (except possibly some nodes with $f = C^*$). Reason: A* surely expands every node with $f(n) < C^*$, i.e. with $h(n) < C^* - g(n)$; every node that satisfies this for the larger h_2 also satisfies it for h_1. So **it is always better to use the larger admissible heuristic**, provided it is cheap to compute. Manhattan distance dominates misplaced tiles (a misplaced tile is at distance ≥ 1).

Generating admissible heuristics from relaxed problems

A **relaxed problem** is the problem with **fewer restrictions on the actions**. Its state-space graph is a supergraph of the original (extra edges = shortcuts), so the cost of an optimal solution to a relaxed problem is an admissible (and consistent) heuristic for the original problem. 8-puzzle rule: "A tile can move from square A to square B if A is horizontally or vertically adjacent to B **and** B is blank." Removing conditions gives:

Relaxed rule	Exact cost of the relaxed problem
(a) A tile can move from A to B if A is adjacent to B	h_2 (Manhattan distance)
(b) A tile can move from A to B if B is blank	Gaschnig's heuristic
(c) A tile can move from A to B	h_1 (misplaced tiles)

If we have several admissible heuristics and none dominates, use $h(n) = \max\{h_1(n), \ldots, h_m(n)\}$ — still admissible, and it dominates all of them.

- **Pattern databases (subproblems):** store the *exact* solution cost of every configuration of a subproblem, e.g. getting tiles 1, 2, 3, 4 (and the blank) into place while ignoring the others. The cost is

a lower bound on the whole problem, so it is admissible. Built once by searching backward from the goal; lookups are cheap. **Disjoint** pattern databases (1-2-3-4 and 5-6-7-8, counting only moves of their own tiles) can be **added** and stay admissible.

- **Learning heuristics from experience:** solve many 8-puzzles optimally; each state on a solution path is an example (state, true cost). Learn h from **features**, e.g. $x_1(n)$ = number of misplaced tiles, $x_2(n)$ = number of pairs of adjacent tiles that are not adjacent in the goal: $h(n) = c_1 x_1(n) + c_2 x_2(n)$, with constants fitted to the data (not guaranteed admissible).

Uninformed vs informed search

	Uninformed (blind) search	Informed (heuristic) search
Knowledge	only the problem definition	problem definition + heuristic $h(n)$
Chooses node by	depth (BFS/DFS) or path cost $g(n)$ (UCS)	$f(n) = h(n)$ (greedy) or $g(n) + h(n)$ (A*)
Efficiency	explores many useless nodes; slow for big problems	focuses on promising nodes; far fewer nodes
Optimal algorithms	UCS; BFS/IDS for equal step costs	A* with admissible (tree) / consistent (graph) h
Examples	BFS, UCS, DFS, DLS, IDS, bidirectional	greedy best-first, A*, IDA*, RBFS, SMA*
Extra cost	none	designing and computing h

EXAM TIP — A* / HEURISTIC QUESTIONS (10M)

- Write $f(n) = g(n) + h(n)$ and define each term. Then do the Romania trace as a table with "$f = g + h$" for every node (show the additions!) and draw the tree (Fig 3.6).
- Always end with the **path and cost (418 km)** and the comparison with greedy (450 km).
- For "admissible vs consistent": two definitions + formula + triangle figure + the optimality theorem.
- For 8-puzzle heuristics: draw both boards, compute h_1 and h_2 tile by tile, then mention dominance and b^*.

LIKELY EXAM QUESTIONS — CHAPTER 3

1. What are the components of a well-defined problem? Formulate the vacuum world / 8-puzzle / 8-queens / Romania route-finding problem. **(6–8M)**
2. Differentiate tree search and graph search. Explain frontier, explored set and the node data structure. **(6M)**
3. Explain BFS with algorithm and example; analyse its time and space complexity. **(8M)**
4. Explain uniform-cost search. Why is the goal test applied on expansion? Trace it on a given graph. **(8M)**
5. Explain DFS, depth-limited search and iterative deepening search with examples. **(10M)**
6. Compare all uninformed search strategies (completeness, optimality, time, space). **(6M)**
7. Explain greedy best-first search and A* on the Romania map. Why is greedy not optimal? **(10M)**
8. Define admissible and consistent heuristics. Prove that A* is optimal when h is admissible. **(8M)**
9. For the given 8-puzzle state compute h_1 and h_2. What is dominance? What is the effective branching factor? **(6M)**
10. How can admissible heuristics be generated from relaxed problems? **(4M)**

60-SECOND RECAP

- Problem = **initial state, actions, transition model, goal test, path cost**. Optimal solution = lowest path cost.
- Frontier = open list of leaf nodes; explored set = closed list (graph search avoids repeated states). Measure with completeness, optimality, time, space using b, d, m.
- **BFS** FIFO, $O(b^d)$ time & space, optimal for equal costs · **UCS** lowest g, optimal · **DFS** LIFO, $O(bm)$ space, not complete/optimal · **DLS** depth limit l · **IDS** best of BFS + DFS, $O(bd)$ space · **Bidirectional** $O(b^{d/2})$.
- **Greedy**: $f = h$, fast, not optimal (450 km). **A***: $f = g + h$, optimal with admissible (tree) / consistent (graph) h (418 km).
- 8-puzzle: h_1 = misplaced tiles (8), h_2 = Manhattan (18); h_2 dominates h_1; good heuristics have b^* near 1; relaxed problems give admissible heuristics.

4 Logical Agents & Propositional Logic

Humans know things and reason with that knowledge. A **knowledge-based agent** keeps facts as sentences in a logic and *derives* new facts to decide what to do. We use the Wumpus world as the running example and propositional logic as the language.

4.1 Knowledge-based agents

DEFINITIONS

- **Knowledge base (KB)** — a set of sentences that represent what the agent knows about the world. (Think: the agent's notebook of facts and rules.)
- **Sentence** — an assertion about the world, expressed in a **knowledge representation language**. A sentence taken as given without being derived from other sentences is an **axiom**.
- **TELL** — adds a new sentence to the KB. **ASK** — queries what is known.
- **Inference** — deriving new sentences from old ones. The answer to an ASK must *follow* from what has been TELLed; inference should not make things up.
- **Background knowledge** — what the KB contains initially, before any percepts.

```
function KB-AGENT(percept) returns an action
  persistent: KB, a knowledge base
              t, a counter, initially 0, indicating time
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  action ← ASK(KB, MAKE-ACTION-QUERY(t))
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action
```

Each time it is called the agent does three things: (1) **TELLs** the KB what it perceives, (2) **ASKs** the KB what action to perform (extensive reasoning about the current state, outcomes of actions, etc. happens here), (3) **TELLs** the KB which action was chosen, and executes it.

Function	What it does
`MAKE-PERCEPT-SENTENCE`	builds a sentence asserting that the agent perceived the given percept at time *t*
`MAKE-ACTION-QUERY`	builds a sentence that asks what action should be done at time *t*
`MAKE-ACTION-SENTENCE`	builds a sentence asserting that the chosen action was executed

Knowledge level vs implementation level

At the **knowledge level** we describe an agent only by *what it knows and what its goals are* (e.g. a taxi knows that a bridge links two parts of the city and its goal is to take a passenger across). At the **implementation level** we describe the *data structures and algorithms* that store and manipulate that knowledge. We can design at the knowledge level without worrying about implementation.

Declarative vs procedural approach

Declarative: build the agent by TELLing it sentences one by one until it knows how to operate (state *what* is true). **Procedural:** encode the desired behaviours directly as program code (state *how* to act). Successful agents usually combine both, and add learning.

4.2 The Wumpus world ★ Very likely

A cave of rooms (a 4×4 grid) connected by passageways. Somewhere lurks the terrible **wumpus**, a beast that eats anyone who enters its room. The agent has **one arrow** to shoot it. Some rooms contain bottomless **pits** that trap anyone who wanders in (the wumpus is too big to fall in). One room contains a heap of **gold**. The agent must find the gold and get out alive.

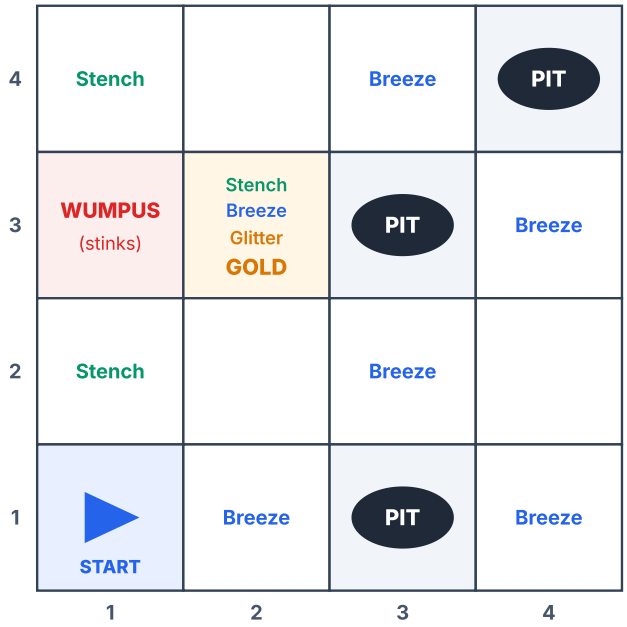


Fig 4.1 A typical Wumpus world. The agent starts at [1,1] facing right.

PEAS description	
P	**+1000** for climbing out of the cave with the gold; **−1000** for falling into a pit or being eaten by the wumpus; **−1** for each action taken; **−10** for using up the arrow. The game ends when the agent dies or climbs out.
E	4×4 grid of rooms. Agent starts in [1,1] facing east (right). Gold and wumpus are placed randomly in squares other than [1,1]; every other square is a pit with probability 0.2.
A	*Forward*, *TurnLeft*, *TurnRight* (by 90°), *Grab* (pick up gold in the same square), *Shoot* (arrow flies straight ahead until it kills the wumpus or hits a wall; only the first Shoot has an effect), *Climb* (leave the cave, only from [1,1]).
S	5 sensors, each giving one bit: **Stench** (in the wumpus's square and directly adjacent ones, not diagonal), **Breeze** (adjacent to a pit), **Glitter** (in the gold's square), **Bump** (walked into a wall), **Scream** (wumpus killed; heard anywhere).

The percept is a **5-tuple** [*Stench, Breeze, Glitter, Bump, Scream*]. E.g. stench and breeze but nothing else: `[Stench, Breeze, None, None, None]`. Entering a square with a pit or a live wumpus kills the agent (a dead wumpus is safe but still smells).

Environment properties:

- **Partially observable** — only local perception (you smell/feel only neighbours).
- **Deterministic** — outcomes are exactly specified.
- **Sequential** — rewards may come only after many actions.
- **Static** — the wumpus and pits don't move.
- **Discrete** — squares, actions and percepts are discrete.
- **Single-agent** — the wumpus is a natural feature, not an agent.
- Rules ("physics") **known**; the layout is **unknown** to the agent.

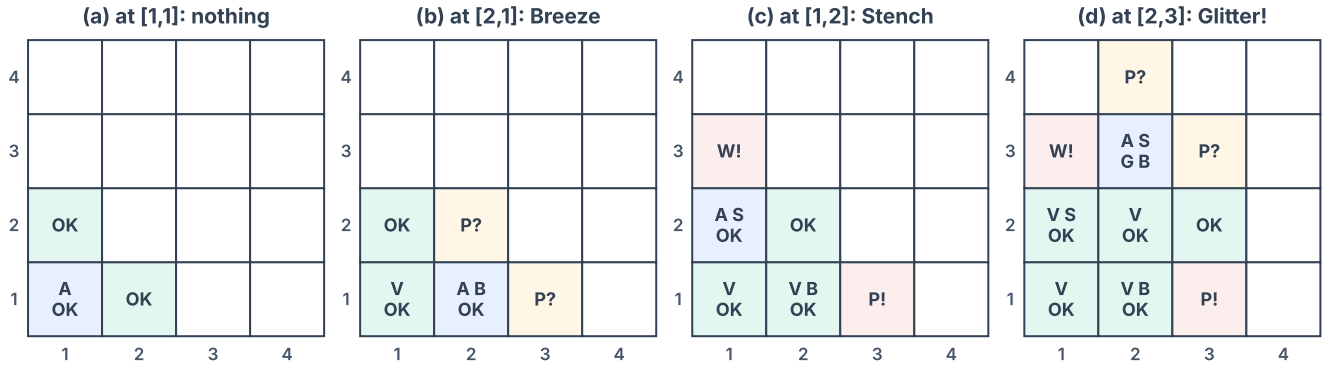
MEMORY TRICKS

Sensors: "**S**ome **B**rave **G**uys **B**ump into **S**creams" → Stench, Breeze, Glitter, Bump, Scream (in tuple order).

Scores: "gold out +1000, death −1000, arrow −10, every step −1".

Clues: **S**tench = **S**omething (wumpus) next door; **B**reeze = **B**ottomless pit next door.

Step-by-step reasoning of the agent



A = agent · B = breeze · G = glitter · OK = safe · P = pit · S = stench · V = visited · W = wumpus · ? = maybe · ! = certain

Fig 4.2 What the agent knows after each move (for the world in Fig 4.1).

1. **At [1,1]:** percept `[None, None, None, None, None]`. No breeze ⇒ no pit in the neighbours; no stench ⇒ no wumpus in the neighbours. So [1,2] and [2,1] are OK (safe). The agent moves to [2,1].
2. **At [2,1]:** percept `[None, Breeze, None, None, None]`. Breeze ⇒ a pit in [2,2] or [3,1] (marked P?). [1,1] is already known safe. Being cautious, the agent turns back to [1,1] and then goes to the other safe square [1,2].
3. **At [1,2]:** percept `[Stench, None, None, None, None]`. Stench ⇒ the wumpus is in [1,3] or [2,2]. But there was **no stench in [2,1]**, so the wumpus is not in [2,2] ⇒ wumpus in [1,3] (W!). No breeze in [1,2] ⇒ no pit in [2,2] (or [1,3]); combined with the breeze in [2,1] ⇒ pit in [3,1] (P!). Now [2,2] is OK.
4. **At [2,2]:** nothing perceived ⇒ [3,2] and [2,3] are OK. The agent moves to [2,3].
5. **At [2,3]:** `[Stench, Breeze, Glitter, None, None]` ⇒ **Grab** the gold, return along safe squares to [1,1] and **Climb** out. (The breeze here means a pit in [2,4] or [3,3] — not yet certain.)

KEY POINT

In each case where the agent draws a conclusion from the available information, that conclusion is guaranteed to be correct if the available information is correct. This is the fundamental property of logical reasoning.

4.3 Logic: syntax, semantics, entailment ★ Very likely

Syntax the rules for **well-formed sentences**. "$x + y = 4$" is well formed; "$x4y+ =$" is not.

Semantics the **meaning** of sentences: defines the **truth** of each sentence with respect to each **possible world**. "$x + y = 4$" is true in a world where $x = 2$, $y = 2$, false where $x = 1$, $y = 1$.

Model a mathematical abstraction of a possible world: it fixes the truth or falsehood of every relevant sentence. If sentence α is true in model m, we say ***m* satisfies α** (or m is a model of α). **M(α)** = the set of all models of α.

ENTAILMENT

α ⊨ β ("α entails β", "β logically follows from α") means: in every model in which α is true, β is also true.

$$\alpha \models \beta \text{ if and only if } M(\alpha) \subseteq M(\beta)$$

Example: ($x = 0$) ⊨ ($xy = 0$). Note: if α ⊨ β, α is the *stronger* assertion — it rules out more possible worlds.

WORKED EXAMPLE — ENTAILMENT BY MODEL CHECKING IN THE WUMPUS WORLD

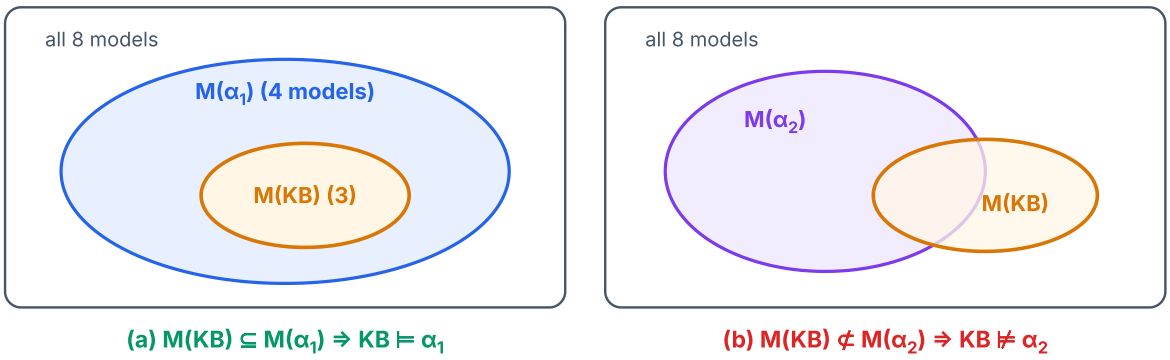
The agent has felt nothing in [1,1] and a breeze in [2,1]. KB = these percepts + the rules of the world. Are there pits in [1,2], [2,2], [3,1]? Three yes/no symbols ⇒ 2^3 = 8 possible models.

$P_{1,2}$	$P_{2,2}$	$P_{3,1}$	KB?	$\alpha_1 = \neg P_{1,2}$	$\alpha_2 = \neg P_{2,2}$
F	F	F	F	T	T
F	F	T	T	T	T
F	T	F	T	T	F
F	T	T	T	T	F
T	F	F	F	F	T
T	F	T	F	F	T
T	T	F	F	F	F
T	T	T	F	F	F

KB is false when $P_{1,2}$ is true (there would be a breeze in [1,1]) and when neither [2,2] nor [3,1] has a pit (no breeze in [2,1]). So KB is true in exactly **3 models**.

α_1 "no pit in [1,2]" is true in all 3 ⇒ $KB \models \alpha_1$.

α_2 "no pit in [2,2]" is false in 2 of them ⇒ **$KB \not\models \alpha_2$**: the agent cannot conclude that [2,2] is pit-free (or that it has one).



(a) $M(KB) \subseteq M(\alpha_1) \Rightarrow KB \models \alpha_1$ (b) $M(KB) \not\subseteq M(\alpha_2) \Rightarrow KB \not\models \alpha_2$

Fig 4.3 Entailment as a subset relation between sets of models.

Inference, soundness, completeness, grounding

Logical inference derives conclusions: $KB \vdash_i \alpha$ means "α is derived from KB by inference algorithm *i*". **Model checking** is one inference algorithm: enumerate all possible models and check that α is true in every model in which KB is true. *Analogy:* the set of all consequences of KB is a haystack and α a needle; **entailment** = the needle is in the haystack; **inference** = finding it.

SOUND (TRUTH-PRESERVING)

An inference algorithm is **sound** if it derives only entailed sentences: whenever $KB \vdash_i \alpha$, also $KB \models \alpha$. An unsound procedure "makes things up" (announces needles that aren't there).

COMPLETE

An inference algorithm is **complete** if it can derive any sentence that is entailed: whenever $KB \models \alpha$, also $KB \vdash_i \alpha$. It never misses a needle.

With a sound inference procedure, **if KB is true in the real world, then any sentence α derived from KB is also true in the real world**. **Grounding** asks how we know KB is true in the real world at all: the connection is the agent's **sensors** (percept sentences are true because sensors produced them) and **learning** (general rules such as "squares next to pits are breezy" come from experience; learning can be wrong, so this is not guaranteed).

MEMORY TRICK

⊨ (double bar) = **semantics** — "is *truly* implied in all models". ⊢ (single bar) = **syntax** — "can be *derived* by the algorithm". **Sound** = "says nothing false"; **Complete** = "misses nothing true".

4.4 Propositional logic: a very simple logic ★ Very likely

Syntax

- **Atomic sentence** = a single **proposition symbol** standing for a proposition that can be true or false: *P*, *Q*, *R*, $W_{1,3}$ ("the wumpus is in [1,3]"), *North*. Two symbols have fixed meanings: **True** (always true) and **False** (always false).
- **Complex sentences** are built from simpler ones with parentheses and the five **logical connectives**:

Symbol	Name	Read as	Example and parts
¬	**Negation**	not	$\neg W_{1,3}$. A **literal** is an atomic sentence (*positive literal*) or a negated atomic sentence (*negative literal*).
∧	**Conjunction**	and	$W_{1,3} \wedge P_{3,1}$; the parts are **conjuncts**.
∨	**Disjunction**	or	$(W_{1,3} \wedge P_{3,1}) \vee W_{2,2}$; the parts are **disjuncts**.
⇒	**Implication** (conditional)	implies, if…then	$(W_{1,3} \wedge P_{3,1}) \Rightarrow \neg W_{2,2}$. Left part = **premise / antecedent**, right part = **conclusion / consequent**. Also called *rules* or if–then statements (sometimes written ⊃ or →).
⇔	**Biconditional**	if and only if	$W_{1,3} \Leftrightarrow \neg W_{2,2}$ (sometimes written ≡).

```
Sentence        → AtomicSentence |
ComplexSentence
AtomicSentence  → True | False | P | Q | R |
...
ComplexSentence → ( Sentence ) | [ Sentence ]
                |  ¬ Sentence
                |  Sentence ∧ Sentence
                |  Sentence ∨ Sentence
                |  Sentence ⇒ Sentence
                |  Sentence ⇔ Sentence
OPERATOR PRECEDENCE : ¬, ∧, ∨, ⇒, ⇔
```

BNF grammar of propositional logic (left). **Precedence** from highest to lowest: ¬ , ∧ , ∨ , ⇒ , ⇔ . So

$$\neg A \wedge B \Rightarrow C \text{ means } ((\neg A) \wedge B) \Rightarrow C$$

MEMORY TRICK — PRECEDENCE

"**N**aughty **A**nts **O**ften **I**nvade **B**akeries" → **N**ot, **A**nd, **O**r, **I**mplies, **B**iconditional.

Semantics and the truth table

A **model** in propositional logic simply fixes the truth value (true/false) of every proposition symbol, e.g. $m_1 = \{P_{1,2} = false, P_{2,2} = false, P_{3,1} = true\}$. With *n* symbols there are 2^n models. The truth of a complex sentence is computed recursively with these rules:

P	*Q*	¬*P*	*P* ∧ *Q*	*P* ∨ *Q*	*P* ⇒ *Q*	*P* ⇔ *Q*
false	false	true	false	false	**true**	true
false	true	true	false	true	**true**	false
true	false	false	false	true	**false**	false
true	true	false	true	true	**true**	true

- ¬*P* is true iff *P* is false.
- *P* ∧ *Q* is true iff both are true.
- *P* ∨ *Q* is true iff at least one is true.
- *P* ⇒ *Q* is true **unless *P* is true and *Q* is false**.
- *P* ⇔ *Q* is true iff *P* and *Q* are both true or both false.

COMMON MISTAKE — IMPLICATION

$P \Rightarrow Q$ is TRUE whenever P is false, whatever Q is. Read it as "if P is true, then I claim Q is true; otherwise I make no claim." So "5 is odd ⇒ Tokyo is the capital of Japan" is true, and "5 is even ⇒ Sam is smart" is true regardless of Sam. Implication does not need any causal relation between P and Q.

Evaluating a sentence: in m_1, $\neg P_{1,2} \wedge (P_{2,2} \vee P_{3,1}) = true \wedge (false \vee true) = true \wedge true = true$.

A simple knowledge base for the Wumpus world

Symbols for each [x,y]: $P_{x,y}$ = pit, $W_{x,y}$ = wumpus, $B_{x,y}$ = breeze, $S_{x,y}$ = stench (true if present).

Rule	Sentence	Meaning
R_1	$\neg P_{1,1}$	There is no pit in [1,1].
R_2	$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$	A square is breezy **if and only if** there is a pit in a neighbouring square.
R_3	$B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$	Same rule for [2,1].
R_4	$\neg B_{1,1}$	Percept: no breeze in [1,1].
R_5	$B_{2,1}$	Percept: breeze in [2,1].

Why ⇔ and not ⇒? $B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})$ is true but incomplete: it does not say that a pit next door *causes* a breeze. We need both directions.

Inference by truth-table enumeration (model checking)

```
function TT-ENTAILS?(KB, α) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
          α, the query, a sentence in propositional logic
  symbols ← a list of the proposition symbols in KB and α
  return TT-CHECK-ALL(KB, α, symbols, { })

function TT-CHECK-ALL(KB, α, symbols, model) returns true or false
  if EMPTY?(symbols) then
     if PL-TRUE?(KB, model) then return PL-TRUE?(α, model)
     else return true        // when KB is false, always return true
  else
     P ← FIRST(symbols);  rest ← REST(symbols)
     return (TT-CHECK-ALL(KB, α, rest, model ∪ {P = true})
             and TT-CHECK-ALL(KB, α, rest, model ∪ {P = false}))
```

The KB R_1–R_5 has 7 symbols ($B_{1,1}$, $B_{2,1}$, $P_{1,1}$, $P_{1,2}$, $P_{2,1}$, $P_{2,2}$, $P_{3,1}$) ⇒ 2^7 = **128 models**. KB is true in only **three** of them:

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	R_1…R_5	KB
F	T	F	**F**	F	F	T	all true	**true**
F	T	F	**F**	F	T	F	all true	**true**
F	T	F	**F**	F	T	T	all true	**true**

$P_{1,2}$ is false in all three ⇒ $KB \models \neg P_{1,2}$ (no pit in [1,2]). $P_{2,2}$ is true in two and false in one ⇒ we cannot yet tell. The algorithm is **sound and complete**, but with n symbols its time is $O(2^n)$ (space $O(n)$, since it enumerates depth-first). Propositional entailment is co-NP-complete, so every known algorithm is exponential in the worst case.

4.5 Reasoning patterns: propositional theorem proving ★ Very likely

Instead of enumerating models, we can apply **inference rules** directly to sentences to build a **proof**. First, some concepts:

- **Logical equivalence:** $\alpha \equiv \beta$ if they are true in the same set of models. Equivalently, $\alpha \equiv \beta$ iff $\alpha \models \beta$ and $\beta \models \alpha$.
- **Validity:** a sentence is **valid** (a **tautology**) if it is true in *all* models, e.g. $P \vee \neg P$. **Deduction theorem:** $\alpha \models \beta$ iff $(\alpha \Rightarrow \beta)$ is valid.
- **Satisfiability:** a sentence is **satisfiable** if it is true in *some* model (the **SAT** problem, the first problem proved NP-complete). α is valid iff $\neg\alpha$ is unsatisfiable.
- **Proof by contradiction (refutation, reductio ad absurdum):** $\alpha \models \beta$ iff $(\alpha \wedge \neg\beta)$ is unsatisfiable. Assume β is false and show this leads to a contradiction. This is the basis of resolution.

Equivalence	Name
$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$	commutativity of $\wedge$
$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$	commutativity of $\vee$
$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$	associativity of $\wedge$
$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$	associativity of $\vee$
$\neg(\neg\alpha) \equiv \alpha$	double-negation elimination
$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$	contraposition
$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$	**implication elimination**
$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$	**biconditional elimination**
$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$	**De Morgan**
$\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$	**De Morgan**
$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$	distributivity of $\wedge$ over $\vee$
$(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$	**distributivity of $\vee$ over $\wedge$**

Inference rules

Modus Ponens $\dfrac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$

If (WumpusAhead ∧ WumpusAlive) ⇒ Shoot and (WumpusAhead ∧ WumpusAlive) are given, infer Shoot.

And-Elimination $\dfrac{\alpha \wedge \beta}{\alpha}$

From (WumpusAhead ∧ WumpusAlive), infer WumpusAlive.

All the equivalences above can also be used as inference rules, e.g. biconditional elimination gives $\dfrac{\alpha \Leftrightarrow \beta}{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}$ and $\dfrac{(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)}{\alpha \Leftrightarrow \beta}$. These rules are **sound** (a line under premises means: whenever the premises are true, the conclusion is true).

WORKED PROOF — PROVE $\neg P_{1,2}$ (NO PIT IN [1,2]) FROM R_1–R_5

Step	Sentence derived	Rule used
R_6	$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$	biconditional elimination on R_2
R_7	$(P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1}$	And-Elimination on R_6
R_8	$\neg B_{1,1} \Rightarrow \neg(P_{1,2} \vee P_{2,1})$	contraposition of R_7
R_9	$\neg(P_{1,2} \vee P_{2,1})$	Modus Ponens with R_8 and R_4 ($\neg B_{1,1}$)
R_{10}	$\mathbf{\neg P_{1,2} \wedge \neg P_{2,1}}$	De Morgan on R_9 ⇒ neither [1,2] nor [2,1] has a pit ✔

Finding a proof can be framed as a **search problem**: initial state = the KB; actions = applying inference rules; result = KB plus the new sentence; goal = a state containing the sentence to prove. It can be much faster than model checking because irrelevant symbols ($B_{2,1}$, $P_{1,1}$, $P_{2,2}$, $P_{3,1}$) are simply ignored.

Monotonicity: the set of entailed sentences can only *increase* as information is added: if $\text{KB} \models \alpha$ then $\text{KB} \wedge \beta \models \alpha$. New knowledge can help draw more conclusions but cannot invalidate a conclusion already drawn.

4.6 Proof by resolution ★ Very likely

The rules above are sound, but are they *complete*? **Resolution** is a single inference rule that gives a complete inference algorithm when coupled with any complete search. Continue the Wumpus story: the agent returns from [2,1] to [1,1] and goes to [1,2], where it perceives a stench but no breeze. We add R_{11}: $\neg B_{1,2}$ and R_{12}: $B_{1,2} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{1,3})$. As before we derive R_{13}: $\neg P_{2,2}$ and R_{14}: $\neg P_{1,3}$, and from R_3 and R_5 (biconditional elimination + Modus Ponens) R_{15}: $P_{1,1} \vee P_{2,2} \vee P_{3,1}$. Now:

R_{15}: $P_{1,1} \vee P_{2,2} \vee P_{3,1}$ with R_{13}: $\neg P_{2,2}$ ⇒ R_{16}: $\mathbf{P_{1,1} \vee P_{3,1}}$ (resolve away $P_{2,2}$)

R_{16}: $P_{1,1} \vee P_{3,1}$ with R_1: $\neg P_{1,1}$ ⇒ R_{17}: $\mathbf{P_{3,1}}$ the pit is in [3,1]!

Unit resolution

$$\frac{\ell_1 \vee \ldots \vee \ell_k, \quad m}{\ell_1 \vee \ldots \vee \ell_{i-1} \vee \ell_{i+1} \vee \ldots \vee \ell_k}$$

ℓ_i and m are **complementary literals** (one is the negation of the other). A clause + a single literal (unit clause).

Full resolution

$$\frac{\ell_1 \vee \ldots \vee \ell_k, \quad m_1 \vee \ldots \vee m_n}{\text{all literals of both except } \ell_i \text{ and } m_j}$$

where ℓ_i and m_j are complementary. E.g. $(P_{1,1} \vee P_{3,1})$, $(\neg P_{1,1} \vee \neg P_{2,2}) \Rightarrow (P_{3,1} \vee \neg P_{2,2})$.

Factoring: the resulting clause must contain only **one copy of each literal**; e.g. resolving $(A \vee B)$ with $(A \vee \neg B)$ gives $(A \vee A)$, which is reduced to just A. **Intuition:** if ℓ_i is true then m_j is false, so the rest of the second clause must be true; if ℓ_i is false then the rest of the first clause must be true. Either way the resolvent is true — resolution is sound.

COMMON MISTAKE

Resolve on **only one pair** of complementary literals at a time. From $(P \vee Q)$ and $(\neg P \vee \neg Q)$ you may **not** derive the empty clause; the correct resolvents are $(Q \vee \neg Q)$ or $(P \vee \neg P)$, which are tautologies (useless).

Conjunctive normal form (CNF)

Resolution works only on **clauses** (disjunctions of literals). A sentence is in **CNF** if it is a conjunction of clauses (an AND of ORs), e.g. $(A \vee \neg B) \wedge (B \vee C \vee \neg D)$. (k-CNF: exactly k literals per clause.) **Every propositional sentence is logically equivalent to a CNF sentence**, obtained in four steps:

WORKED EXAMPLE — CONVERT R_2: $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ TO CNF

Step	Result
1. **Eliminate ⇔**: replace $\alpha \Leftrightarrow \beta$ with $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$	$(B_{1,1} \Rightarrow (P_{1,2} \vee P_{2,1})) \wedge ((P_{1,2} \vee P_{2,1}) \Rightarrow B_{1,1})$
2. **Eliminate ⇒**: replace $\alpha \Rightarrow \beta$ with $\neg\alpha \vee \beta$	$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg(P_{1,2} \vee P_{2,1}) \vee B_{1,1})$
3. **Move ¬ inwards** (double negation, De Morgan) so ¬ appears only before literals	$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge ((\neg P_{1,2} \wedge \neg P_{2,1}) \vee B_{1,1})$
4. **Distribute ∨ over ∧**	$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$

The result is a conjunction of three clauses — ready for resolution.

MEMORY TRICK — CNF STEPS

"BIND" → **B**iconditional out, **I**mplication out, **N**egation in, **D**istribute ∨ over ∧.

The resolution algorithm (refutation)

```
function PL-RESOLUTION(KB, α) returns true or false
  inputs: KB, the knowledge base, a sentence in propositional logic
          α, the query, a sentence in propositional logic
  clauses ← the set of clauses in the CNF representation of KB ∧ ¬α
  new ← { }
  loop do
     for each pair of clauses C_i, C_j in clauses do
        resolvents ← PL-RESOLVE(C_i, C_j)
        if resolvents contains the empty clause then return true
        new ← new ∪ resolvents
     if new ⊆ clauses then return false
     clauses ← clauses ∪ new
```

1. To show KB ⊨ α, show (KB ∧ ¬α) is **unsatisfiable**: convert KB ∧ ¬α to CNF.
2. Repeatedly resolve pairs of clauses that contain complementary literals and add each new clause.
3. Stop when either (a) the **empty clause** { } is produced — a contradiction, so KB ⊨ α; or (b) no new clauses can be added — KB does not entail α.

The empty clause (a disjunction of no disjuncts) is equivalent to *False*; it arises only from resolving two complementary unit clauses such as *P* and ¬*P*.

WORKED EXAMPLE — PROVE $\neg P_{1,2}$ FROM KB = $R_2 \wedge R_4$ BY RESOLUTION

Clauses of KB ∧ ¬α: (1) $\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$ (2) $\neg P_{1,2} \vee B_{1,1}$ (3) $\neg P_{2,1} \vee B_{1,1}$ [from R_2] (4) $\neg B_{1,1}$ [R_4] (5) **$P_{1,2}$** [negation of the goal $\neg P_{1,2}$].

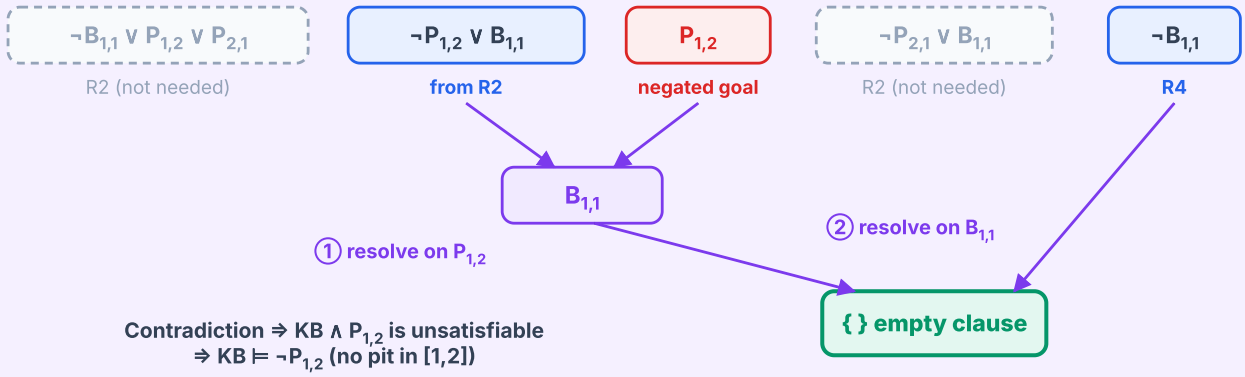


Fig 4.4 Resolution refutation tree: (2) + (5) give $B_{1,1}$; with (4) this gives the empty clause.

PRACTICE — KB: P ⇒ Q, Q ⇒ R, P. PROVE R BY RESOLUTION.

CNF clauses: (1) $\neg P \vee Q$ (2) $\neg Q \vee R$ (3) P (4) $\neg R$ (negated goal)
(1) + (3) on $P \Rightarrow$ (5) Q · (2) + (5) on $Q \Rightarrow$ (6) R · (6) + (4) on $R \Rightarrow$ **{ } empty clause** ⇒ KB ⊨ R

Completeness of resolution. The **resolution closure** RC(S) is the set of all clauses derivable by repeated resolution from the clauses S; it is finite. The **ground resolution theorem**: if a set of clauses is unsatisfiable, then its resolution closure contains the empty clause. Hence resolution is **refutation-complete**: it can always prove KB ⊨ α by refuting KB ∧ ¬α.

4.7 Horn clauses and definite clauses

Full resolution can be expensive. Many real KBs use restricted clause forms that allow very fast inference.

Clause type	Definition	Example
Definite clause	a disjunction of literals with **exactly one positive** literal	$\neg L_{1,1} \vee \neg Breeze \vee B_{1,1}$
Horn clause	a disjunction of literals with **at most one positive** literal (all definite clauses are Horn)	$\neg A \vee \neg B \vee C$; $\neg A \vee \neg B$
Goal clause	a clause with **no positive** literal	$\neg A \vee \neg B$
Not Horn	two or more positive literals	$\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}$

Why they are useful:

1. Every definite clause can be written as an **implication** whose premise is a conjunction of positive literals and whose conclusion is a single positive literal: $\neg L_{1,1} \vee \neg Breeze \vee B_{1,1} \equiv (L_{1,1} \wedge Breeze) \Rightarrow B_{1,1}$. The premise is the **body**, the conclusion the **head**. A single positive literal (e.g. $L_{1,1}$) is a **fact**. Such rules are easy for humans to read.
2. Inference can be done with **forward chaining** and **backward chaining**, which are natural and easy to follow.

3. Deciding entailment with Horn clauses takes time linear in the size of the KB. (Horn clauses are also closed under resolution.)

4.8 Forward and backward chaining ★ Very likely

Running example (all definite clauses) — prove the query ***Q***:

$$P \Rightarrow Q$$
$$L \wedge M \Rightarrow P$$
$$B \wedge L \Rightarrow M$$
$$A \wedge P \Rightarrow L$$
$$A \wedge B \Rightarrow L$$
$$A$$
$$B$$

In the **AND–OR graph** (right), arrows go from premises to conclusions. An ∧ node needs *all* its incoming arrows proved (AND); several rules into the same symbol are alternatives (OR).

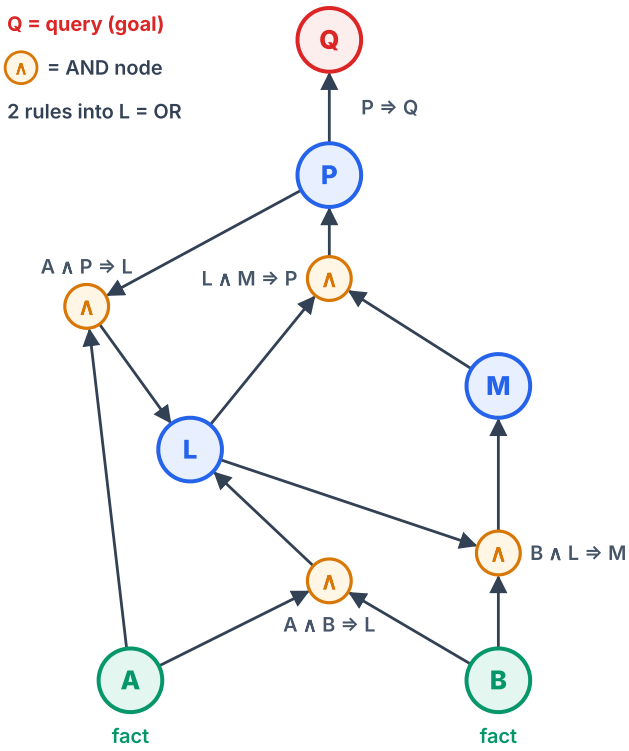


Fig 4.5 AND–OR graph of the Horn KB.

Forward chaining (data-driven)

Idea: start from the **known facts**; if all premises of an implication are known, **fire** the rule and add its conclusion to the known facts; repeat until the query is added or nothing new can be inferred. "What can I conclude from what I know?"

```
function PL-FC-ENTAILS?(KB, q) returns true or false
  inputs: KB, the knowledge base, a set of propositional definite clauses
          q, the query, a proposition symbol
  count    ← a table, where count[c] is the number of symbols in c's premise
  inferred ← a table, where inferred[s] is initially false for all symbols
  agenda   ← a queue of symbols, initially symbols known to be true in KB
  while agenda is not empty do
     p ← POP(agenda)
     if p = q then return true
     if inferred[p] = false then
        inferred[p] ← true
        for each clause c in KB where p is in c.PREMISE do
           decrement count[c]
           if count[c] = 0 then add c.CONCLUSION to agenda
  return false
```

Step	Pop	Effect on rule counts (number of premises still unknown)	Agenda after
0	—	Initial counts: P⇒Q: 1, L∧M⇒P: 2, B∧L⇒M: 2, A∧P⇒L: 2, A∧B⇒L: 2	[A, B]
1	A	A∧P⇒L: 2→1; A∧B⇒L: 2→1	[B]
2	B	B∧L⇒M: 2→1; A∧B⇒L: 1→**0 ⇒ add L**	[L]
3	L	L∧M⇒P: 2→1; B∧L⇒M: 1→**0 ⇒ add M**	[M]
4	M	L∧M⇒P: 1→**0 ⇒ add P**	[P]
5	P	P⇒Q: 1→**0 ⇒ add Q**; A∧P⇒L: 1→0 ⇒ add L (already inferred)	[Q, L]
6	Q	**Q = query ⇒ return true ✔** Facts inferred in order: A, B, L, M, P, Q.	

Forward chaining is **sound** and **complete** for definite-clause KBs and runs in **linear time**. Run without a query, it stops at a **fixed point** where no new inferences are possible.

Backward chaining (goal-driven)

Idea: work backward from the **query** *q*. If *q* is known to be true, done. Otherwise find the implications in the KB whose **conclusion is *q***; if all the premises of one of them can be proved true (by backward chaining, recursively), then *q* is true. "What do I need in order to prove my goal?"

```
Goal Q?   rule P ⇒ Q          → need P
  Goal P?   rule L ∧ M ⇒ P      → need L and M
    Goal L?   try A ∧ P ⇒ L    : A is a fact ✔; P? is already on the goal stack → LOOP,
abandon this rule
              try A ∧ B ⇒ L    : A is a fact ✔, B is a fact ✔   ⇒  L proved ✔
    Goal M?   rule B ∧ L ⇒ M    : B is a fact ✔, L already proved ✔  ⇒  M proved ✔
  ⇒ P proved ✔  (L ✔ and M ✔)
⇒ Q proved ✔
```

Two checks keep it efficient: avoid **loops** (is the new subgoal already on the goal stack?) and avoid **repeated work** (has this subgoal already been proved true or failed?). Backward chaining touches only relevant facts, so its cost is often **much less than linear** in the size of the KB.

	Forward chaining	Backward chaining
Direction	facts → conclusions (bottom-up)	query → supporting facts (top-down)
Type of reasoning	**data-driven**	**goal-driven** (goal-directed)
Starts from	known facts (agenda)	the query (goal stack)
Work done	may derive many facts irrelevant to the goal	only facts relevant to the goal
Complexity	linear in the size of the KB	often much less than linear
Good for	monitoring, perception, reacting to new data ("the light turned red → what follows?")	answering specific questions ("Where are my keys?", "What shall I do now?")
Soundness / completeness	both sound and complete for definite (Horn) clauses	

EXAM TIP — FC/BC QUESTIONS (10M)

- Write the KB, draw the **AND–OR graph**, then give the FC trace as a table (pop, counts, agenda) and the BC trace as an indented goal tree.
- Mention the words **data-driven** (FC) and **goal-driven** (BC), **linear time**, and **sound & complete for Horn/definite clauses**.
- If asked "Why Horn clauses?", give the three reasons (implication form, FC/BC, linear time).

4.9 Agents based on propositional logic

The current state of the world

The agent's KB holds (i) the **percept history** and (ii) general rules about the world (its "physics"): e.g. $\neg P_{1,1}$, $\neg W_{1,1}$, a breeze rule $B_{x,y} \Leftrightarrow (P_{x,y+1} \vee P_{x,y-1} \vee P_{x+1,y} \vee P_{x-1,y})$ for each square, a similar stench rule, "there is at least one wumpus" $W_{1,1} \vee W_{1,2} \vee \ldots \vee W_{4,4}$, and "at most one wumpus" $(\neg W_{x,y} \vee \neg W_{u,v}$ for every pair of squares).

Kind of variable	Meaning	Examples
Atemporal variables	aspects of the world that **never change** — no time superscript	$P_{3,1}$ (pit), $W_{1,3}$ (wumpus), $B_{2,1}$ (square is breezy)
Fluents (state variables)	aspects that **change over time** — indexed by time step t	$L^t_{x,y}$ (agent at $[x,y]$ at time t), $FacingEast^t$, $HaveArrow^t$, $WumpusAlive^t$
Percept variables	what is perceived at time t	$Stench^t$, $Breeze^t$, $Glitter^t$

Percepts are linked to atemporal properties of squares: $L^t_{x,y} \Rightarrow (Breeze^t \Leftrightarrow B_{x,y})$ and $L^t_{x,y} \Rightarrow (Stench^t \Leftrightarrow S_{x,y})$. A square is safe: $OK^t_{x,y} \Leftrightarrow \neg P_{x,y} \wedge \neg(W_{x,y} \wedge WumpusAlive^t)$.

Effect axioms and the frame problem

An **effect axiom** describes the outcome of an action at a time step, e.g.

$$L^0_{1,1} \wedge FacingEast^0 \wedge Forward^0 \Rightarrow (L^1_{2,1} \wedge \neg L^1_{1,1})$$

Effect axioms say what **changes**, but not what **stays the same**. After *Forward* at time 0, can the agent prove $HaveArrow^1$? **No!** Nothing says the arrow is still there. This is the **frame problem**. One fix is **frame axioms** stating what does not change, e.g. $Forward^t \Rightarrow (HaveArrow^t \Leftrightarrow HaveArrow^{t+1})$, but with m actions and n fluents we need $O(mn)$ of them (the *representational* frame problem).

SUCCESSOR-STATE AXIOM (THE SOLUTION)

Write one axiom per fluent F, describing its value at $t{+}1$ in terms of the actions that can change it:

$$F^{t+1} \Leftrightarrow ActionCausesF^t \vee (F^t \wedge \neg ActionCausesNotF^t)$$

In words: F is true next time iff some action just made it true, or it was already true and no action made it false. Example: $HaveArrow^{t+1} \Leftrightarrow (HaveArrow^t \wedge \neg Shoot^t)$. For location: $L^{t+1}_{1,1} \Leftrightarrow (L^t_{1,1} \wedge (\neg Forward^t \vee Bump^{t+1})) \vee (L^t_{1,2} \wedge FacingSouth^t \wedge Forward^t) \vee (L^t_{2,1} \wedge FacingWest^t \wedge Forward^t)$.

A hybrid agent

The **hybrid Wumpus agent** combines **logical inference** (to work out which squares are safe, where the wumpus may be, etc.) with **search** (A* to plan routes through safe squares) and condition–action style priorities.

```
function HYBRID-WUMPUS-AGENT(percept) returns an action
  inputs: percept, a list, [stench, breeze, glitter, bump, scream]
  persistent: KB, initially the atemporal "wumpus physics"
              t, a counter, initially 0;  plan, an action sequence, initially empty
  TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
  TELL the KB the temporal "physics" sentences for time t
  safe ← {[x,y] : ASK(KB, OK^t_{x,y}) = true}
  if ASK(KB, Glitter^t) = true then
     plan ← [Grab] + PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
  if plan is empty then
     unvisited ← {[x,y] : ASK(KB, L^{t'}_{x,y}) = false for all t' ≤ t}
     plan ← PLAN-ROUTE(current, unvisited ∩ safe, safe)
  if plan is empty and ASK(KB, HaveArrow^t) = true then
     possible_wumpus ← {[x,y] : ASK(KB, ¬W_{x,y}) = false}
     plan ← PLAN-SHOT(current, possible_wumpus, safe)
  if plan is empty then          // no choice but to take a risk
     not_unsafe ← {[x,y] : ASK(KB, ¬OK^t_{x,y}) = false}
     plan ← PLAN-ROUTE(current, unvisited ∩ not_unsafe, safe)
  if plan is empty then
     plan ← PLAN-ROUTE(current, {[1,1]}, safe) + [Climb]
  action ← POP(plan)
  TELL(KB, MAKE-ACTION-SENTENCE(action, t))
  t ← t + 1
  return action

function PLAN-ROUTE(current, goals, allowed) returns an action sequence
  problem ← ROUTE-PROBLEM(current, goals, allowed)
  return A*-GRAPH-SEARCH(problem)
```

Priorities in plain words: (1) if there is a glitter, grab the gold, go home and climb out; (2) otherwise visit the nearest **safe unvisited** square; (3) if none, try to **shoot** the wumpus if its possible location is known; (4) if still nothing, take a **risk** on a square that is not provably unsafe; (5) if the mission is impossible, go back to [1,1] and climb out.

Logical state estimation (brief)

Problem: ASK has to reason over the **whole percept history**, so inference gets slower as t grows. Solution: keep a **belief state** — a representation of the set of all possible current states of the world — as a logical formula over the current fluents, and update it after every action and percept (**state estimation**). Exact belief states can be exponentially large, so agents often use a **conservative approximation** (e.g. a conjunction of literals, 1-CNF: keep only the facts known for certain) and cache inference results.

Making plans by propositional inference (SATPlan, brief)

Instead of A*, plans can be found with a SAT solver. Build one sentence: $Init^0 \wedge Transition^1 \wedge \ldots \wedge Transition^T \wedge Goal^T$ (initial state, successor-state axioms for each step, goal true at time T). If a SAT solver finds a model, the action symbols that are *true* in it form the plan. Try T = 0, 1, 2, ... up to T_{max}. Extra axioms are needed: **precondition axioms** (e.g. $Shoot^t \Rightarrow HaveArrow^t$) and **action exclusion axioms** ($\neg A^t_i \vee \neg A^t_j$: only one action at a time).

```
function SATPLAN(init, transition, goal, T_max) returns solution or failure
  for t = 0 to T_max do
     cnf   ← TRANSLATE-TO-SAT(init, transition, goal, t)
     model ← SAT-SOLVER(cnf)
     if model is not null then return EXTRACT-SOLUTION(model)
  return failure
```

EXAM TIP — LOGIC QUESTIONS

- **Wumpus world (10M):** draw the 4×4 grid with START, pits, wumpus, gold, breezes and stenches; then PEAS; then environment properties; then the step-by-step reasoning with small grids.
- **Entailment/soundness/completeness (6M):** definitions with symbols $\models$ and $\vdash$, $M(KB) \subseteq M(\alpha)$, the 8-model table and the Venn figure.
- **Resolution (10M):** write the clauses of $KB \wedge \neg\alpha$ in CNF (show the 4 steps), then draw the resolution tree ending in the empty clause, and conclude "therefore $KB \models \alpha$".
- Write subscripts carefully ($P_{1,2} \neq P_{2,1}$) and name the rule at every step — that is where step marks are given.

COMMON MISTAKES

- $P \Rightarrow Q$ is **true** when P is false; it is false only for P = true, Q = false.
- Writing $\vdash$ (derivation) where $\models$ (entailment) is meant, or swapping the definitions of **sound** and **complete**.
- In resolution, forgetting to **add the negated goal**, or forgetting to convert to CNF first.
- Horn clause = **at most one** positive literal; definite = **exactly one**.
- FC goes from facts to the goal; BC goes from the goal to facts — do not reverse them.

LIKELY EXAM QUESTIONS — CHAPTER 4

1. What is a knowledge-based agent? Write the generic KB-AGENT program and explain TELL and ASK. Distinguish declarative and procedural approaches. **(6M)**
2. Describe the Wumpus world with its PEAS and environment properties. Explain how the agent reasons in the first few moves. **(10M)**
3. Define syntax, semantics, model, entailment, logical inference, soundness, completeness and grounding. Illustrate entailment with the Wumpus example. **(8M)**
4. Give the syntax (BNF) and semantics (truth table) of propositional logic. **(6M)**
5. List the standard logical equivalences. Define validity, satisfiability and the deduction theorem. **(6M)**
6. Using Modus Ponens, And-Elimination and equivalences, prove $\neg P_{1,2}$ from R_1–R_5. **(6M)**
7. Convert $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$ to CNF and prove $\neg P_{1,2}$ using resolution. **(10M)**
8. What are Horn and definite clauses? Explain forward and backward chaining with the AND–OR graph example. Compare them. **(10M)**
9. Explain fluents, effect axioms, the frame problem and successor-state axioms. **(6M)**
10. Explain the hybrid Wumpus agent. **(5M)**

60-SECOND RECAP

- KB = set of sentences; agent TELLs percepts, ASKs for actions; inference derives new sentences.
- Wumpus PEAS: +1000 gold out, −1000 death, −1 per action, −10 arrow; actuators Forward/TurnLeft/TurnRight/Grab/Shoot/Climb; percept [Stench, Breeze, Glitter, Bump, Scream].
- KB ⊨ α iff M(KB) ⊆ M(α); sound = derives only entailed sentences; complete = derives all entailed sentences.
- Connectives ¬ ∧ ∨ ⇒ ⇔ (that precedence); $P \Rightarrow Q$ false only for T ⇒ F. Model checking is $O(2^n)$.
- α ⊨ β iff α ∧ ¬β unsatisfiable. Rules: Modus Ponens, And-Elimination, equivalences. CNF steps: BIND. Resolution + refutation is complete.
- Horn: ≤ 1 positive literal; FC (data-driven) and BC (goal-driven) are linear-time, sound and complete for definite clauses.
- Fluents change with time; frame problem solved by successor-state axioms; hybrid agent = logic for safety + A* for routes.

UNIT I — LAST-MINUTE FACT SHEET

Rational agent	maximizes *expected* performance given percept sequence + built-in knowledge
PEAS	Performance, Environment, Actuators, Sensors
Environment dimensions	observable, agents, deterministic, episodic, static, discrete, known
Agent types	simple reflex → model-based → goal-based → utility-based → learning
Problem components	initial state, actions, transition model, goal test, path cost
BFS / UCS / DFS	FIFO, $O(b^d)$ / priority by g, $O(b^{1+\lfloor C^*/\varepsilon \rfloor})$ / LIFO, $O(b^m)$ time, $O(bm)$ space
DLS / IDS / Bidirectional	$O(b^l)$, $O(bl)$ / $O(b^d)$, $O(bd)$ / $O(b^{d/2})$
Greedy vs A*	$f = h$ (450 km) vs $f = g + h$ (418 km, optimal)
Admissible / consistent	$h(n) \le h^*(n)$ / $h(n) \le c(n,a,n') + h(n')$
8-puzzle example	$h_1 = 8$, $h_2 = 18$, true cost 26; 9!/2 = 181,440 states
Effective branching factor	$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$
Entailment	KB ⊨ α iff M(KB) ⊆ M(α) iff KB ∧ ¬α unsatisfiable
CNF of R_2	$(\neg B_{1,1} \vee P_{1,2} \vee P_{2,1}) \wedge (\neg P_{1,2} \vee B_{1,1}) \wedge (\neg P_{2,1} \vee B_{1,1})$
Successor-state axiom	$F^{t+1} \Leftrightarrow ActionCausesF^t \vee (F^t \wedge \neg ActionCausesNotF^t)$

UNIT II

Logic, Knowledge, Uncertainty & Intro to ML

- **Ch 5 · First-Order Logic**: syntax & semantics, quantifiers, using FOL, knowledge engineering
- **Ch 6 · Inference in FOL**: propositional vs FOL inference, unification & lifting, forward chaining, backward chaining, resolution
- **Ch 7 · Knowledge Graphs & Ontologies** self-study topic
- **Ch 8 · Reasoning under Uncertainty**: conditional probability, Bayes' theorem, MAP, Bayesian networks, Naive Bayes
- **Ch 9 · Introduction to Machine Learning**: definitions, terminology, key tasks, well-posed problems, types of ML, Find-S, Candidate-Elimination

Note: the revised 2026 syllabus lists First-Order Logic under Unit II; the older copy lists it at the end of Unit I. Either way it's in CIE-1.

5 First-Order Logic (FOL)

Propositional logic can only say "this whole sentence is true/false". FOL lets us talk about **objects**, their **properties** and the **relations** between them, and say things about **all** or **some** objects.

5.1 Why do we need FOL? (Representation revisited)

In propositional logic (PL) every fact is one indivisible symbol: `JohnIsKing`, `JohnIsGreedy`. PL has very limited expressive power. Try writing these in PL:

- "*Some humans are intelligent*" — PL has no way to say "some".
- "*All students who study pass*" — PL needs one sentence per student:
  $\text{Student(John)} \wedge \text{Studies(John)} \Rightarrow \text{Pass(John)}$, $\text{Student(Ravi)} \wedge \text{Studies(Ravi)} \Rightarrow \text{Pass(Ravi)}$, ... (thousands of rules!)

FOL writes it once: $\forall x\ \text{Student}(x) \wedge \text{Studies}(x) \Rightarrow \text{Pass}(x)$ — one rule for every object x.

DEFINITION

First-order logic (also called **predicate logic** / first-order predicate calculus) is an extension of propositional logic that represents knowledge in terms of objects, relations (predicates), and functions, and uses variables and quantifiers (∀, ∃) to make statements about objects.

OBJECTS

Things: people, houses, numbers, colours, wars, centuries, Richard, John...

RELATIONS

Unary (properties): red, round, prime, king.
n-ary: brother-of, bigger-than, part-of, comes-between.

FUNCTIONS

Map objects to one object: father-of, best-friend, left-leg-of, third-inning-of, end-of.

A FOL statement has two parts: the **subject** (main object) and the **predicate** (a relation that binds atoms together). E.g. "*x is an integer*" → `Integer(x)`; "*Pinky is a cat*" → `Cat(Pinky)`.

5.2 PL vs FOL ★ Very likely

Propositional Logic	First-Order Logic
Uses propositions (whole statements)	Uses predicates, objects, functions
No variables	Has variables (x, y, z...)
No quantifiers	Has quantifiers ∀ (for all), ∃ (there exists)
Limited representation	More expressive
Example: $\text{P} \Rightarrow \text{Q}$	Example: $\text{Student(x)} \Rightarrow \text{Studies(x)}$
Treats statements as indivisible	Represents objects & relationships
Easier (decidable) inference	More computationally complex inference
Ontological commitment: facts	Ontological commitment: facts, objects, relations

5.3 Syntax of FOL: the basic elements ★ Very likely

Element	Examples	Meaning
Constant symbols	John, Richard, 1, A, Mumbai	Name a specific **object**
Variables	x, y, z, a, b	Stand for any object (lowercase by convention)
Predicate symbols	Brother, King, Person, >	Name a **relation** (returns true/false)
Function symbols	LeftLeg, Father, sqrt	Name a **function** (returns an object)
Connectives	¬ ∧ ∨ ⇒ ⇔	Combine sentences (same as PL)
Equality	=	Two terms refer to the same object
Quantifiers	∀, ∃	"for all", "there exists"

Convention: constant, predicate and function symbols begin with **Uppercase**; variables are **lowercase**.

Terms

A **term** is a logical expression that refers to an object. Constants and variables are terms. A **complex term** = function symbol + parenthesised list of terms: $f(t_1, \ldots, t_n)$, e.g. $\text{LeftLeg}(\text{John})$ — "John's left leg" (an object, not true/false!).

Atomic sentences

The most basic sentence: predicate symbol + parenthesised list of terms: $\text{Predicate}(\text{term}_1, \ldots, \text{term}_n)$. "Richard and John are brothers" → $\text{Brother}(\text{Richard}, \text{John})$; also $\text{Married}(\text{Father}(\text{Richard}), \text{Mother}(\text{John}))$.

Complex sentences

Atomic sentences joined by connectives: $\text{King}(\text{Richard}) \vee \text{King}(\text{John})$, $\neg\text{Brother}(\text{LeftLeg}(\text{Richard}), \text{John})$.

> **EXAM TIP — TERM VS SENTENCE**
>
> $\text{LeftLeg}(\text{John})$ is a **term** (an object). $\text{Brother}(\text{Richard}, \text{John})$ is an **atomic sentence** (true or false). Examiners love asking this difference.

BNF grammar of FOL (Fig 8.3 of the textbook)

```
Sentence        → AtomicSentence | ComplexSentence
AtomicSentence  → Predicate | Predicate(Term, …) | Term = Term
ComplexSentence → ( Sentence ) | ¬ Sentence
                | Sentence ∧ Sentence | Sentence ∨ Sentence
                | Sentence ⇒ Sentence | Sentence ⇔ Sentence
                | Quantifier Variable, … Sentence
Term            → Function(Term, …) | Constant | Variable
Quantifier      → ∀ | ∃
Constant        → A | X1 | John | …     Variable → a | x | s | …
Predicate       → True | False | After | Loves | Raining | …
Function        → Mother | LeftLeg | …
Operator precedence:  ¬ , = , ∧ , ∨ , ⇒ , ⇔
```

5.4 Semantics: models and interpretations

A **model** in FOL contains a set of **objects** (the *domain*) plus relations and functions among them. An **interpretation** says which object/relation/function each symbol refers to.

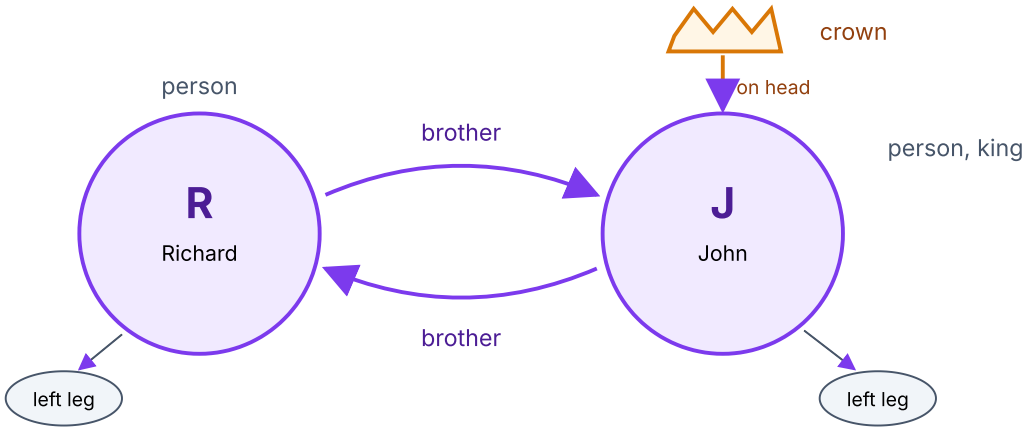


Fig 5.1 A model with 5 objects (Richard, John, their left legs, a crown), 2 binary relations (brother, on-head), 3 unary relations (person, king, crown) and 1 function (left-leg).

Interpretation used in the textbook: **Richard** → Richard the Lionheart (King of England 1189–1199); **John** → his younger brother, the evil King John (1199–1215); **Brother** → the brotherhood relation; **LeftLeg** → the "left leg" function. A relation is a set of **tuples** (objects in a fixed order, written with angle brackets), e.g. the brother relation = { ⟨Richard, John⟩, ⟨John, Richard⟩ }.

Symbols come in three kinds: **constant symbols** (objects) — Richard, John; **predicate symbols** (relations) — Brother, OnHead, Person, King, Crown; **function symbols** (functions) — LeftLeg.

5.5 Quantifiers ★ Very likely

A **quantifier** specifies the *quantity* of objects in the universe of discourse for which a statement holds — it fixes the **range and scope** of a variable.

UNIVERSAL QUANTIFIER ∀ ("FOR ALL")

True for **every** object. Read "for all x...", "everyone", "everything".
Main connective with ∀ is implication ⇒.
"All kings are persons": $\forall x\ \mathrm{King}(x) \Rightarrow \mathrm{Person}(x)$
"All men drink coffee": $\forall x\ \mathrm{Man}(x) \Rightarrow \mathrm{Drink}(x, \mathrm{Coffee})$

EXISTENTIAL QUANTIFIER ∃ ("THERE EXISTS")

True for **at least one** object. Read "there exists an x such that...", "for some x...".
Main connective with ∃ is conjunction ∧.
"King John has a crown on his head": $\exists x\ \mathrm{Crown}(x) \wedge \mathrm{OnHead}(x, \mathrm{John})$
"Some boys play cricket": $\exists x\ \mathrm{Boy}(x) \wedge \mathrm{Play}(x, \mathrm{Cricket})$

COMMON MISTAKE — #1 SOURCE OF LOST MARKS

Wrong: $\forall x\ \mathrm{King}(x) \wedge \mathrm{Person}(x)$ → says "*everything* is a king and a person" (false!).
Wrong: $\exists x\ \mathrm{Crown}(x) \Rightarrow \mathrm{OnHead}(x, \mathrm{John})$ → true if there's *anything* that's not a crown — too weak.
Rule: ∀ goes with ⇒, ∃ goes with ∧.

MEMORY TRICK

"**A**ll → **A**rrow (⇒)", "**E**xists → **E**and (∧)". The upside-down **A** = **A**ll, backwards **E** = **E**xists.

Nested quantifiers

- "Brothers are siblings": $\forall x\ \forall y\ \mathrm{Brother}(x, y) \Rightarrow \mathrm{Sibling}(x, y)$
- "Everybody loves somebody": $\forall x\ \exists y\ \mathrm{Loves}(x, y)$ vs "There is someone loved by everyone": $\exists y\ \forall x\ \mathrm{Loves}(x, y)$ — order matters!
- ∃! (or $\exists_1$) means "there exists exactly one".

Connection between ∀ and ∃ (De Morgan for quantifiers)

$$\neg\forall x\ P(x) \equiv \exists x\ \neg P(x) \quad \neg\exists x\ P(x) \equiv \forall x\ \neg P(x) \quad \forall x\ P(x) \equiv \neg\exists x\ \neg P(x) \quad \exists x\ P(x) \equiv \neg\forall x\ \neg P(x)$$

"Not everyone likes ice-cream" = "someone doesn't like ice-cream".

Equality

The equality symbol = says two terms refer to the **same object**: $\text{Father}(\text{John}) = \text{Henry}$. With negation it insists two terms are different: "Richard has at least two brothers": $\exists x\ \exists y\ \text{Brother}(x, \text{Richard}) \wedge \text{Brother}(y, \text{Richard}) \wedge \neg(x = y)$.

5.6 English → FOL: practice set from class ★ Very likely

English	FOL
Marcus was a man	$\text{Man}(\text{Marcus})$
x is an integer	$\text{Integer}(x)$
Caesar was a ruler	$\text{Ruler}(\text{Caesar})$
Ravi and Rama are brothers	$\text{Brothers}(\text{Ravi}, \text{Rama})$
Marcus tried to assassinate Caesar	$\text{TryToAssassinate}(\text{Marcus}, \text{Caesar})$
All men drink coffee	$\forall x\ \text{Man}(x) \Rightarrow \text{Drink}(x, \text{Coffee})$
Every man respects his parent	$\forall x\ \text{Man}(x) \Rightarrow \text{Respects}(x, \text{Parent})$
Some boys play cricket	$\exists x\ \text{Boy}(x) \wedge \text{Play}(x, \text{Cricket})$
Not all students like both Maths and Science	$\neg\forall x\ [\text{Student}(x) \Rightarrow \text{Like}(x, \text{Maths}) \wedge \text{Like}(x, \text{Science})]$
Everyone is loyal to someone	$\forall x\ \exists y\ \text{Loyal}(x, y)$
Everyone loves everyone	$\forall x\ \forall y\ \text{Loves}(x, y)$
All boys like cricket	$\forall x\ \text{Boy}(x) \Rightarrow \text{Like}(x, \text{Cricket})$
Some boys like cricket	$\exists x\ \text{Boy}(x) \wedge \text{Like}(x, \text{Cricket})$
Every person likes ice-cream	$\forall x\ \text{Person}(x) \Rightarrow \text{Likes}(x, \text{IceCream})$
All kings who are greedy are evil	$\forall x\ \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
No student failed	$\neg\exists x\ \text{Student}(x) \wedge \text{Failed}(x) \equiv \forall x\ \text{Student}(x) \Rightarrow \neg\text{Failed}(x)$

NOTE ON A SLIDE

The class slide writes "Everyone loves everyone" as $\forall x\ \exists y\ \text{loves}(x, y)$ — that formula actually means "everyone loves *someone*". The correct form is $\forall x\ \forall y\ \text{Loves}(x, y)$. Also "Every man respects his parent" is more precisely $\forall x\ \forall y\ \text{Man}(x) \wedge \text{Parent}(y, x) \Rightarrow \text{Respects}(x, y)$; write the precise form if you have time.

EXAM TIP — 4-STEP RECIPE FOR TRANSLATING

1. Find the **objects** (constants) and the **properties/relations** (predicates).
2. Look for quantity words: *all, every, each, any* → ∀; *some, a, there is, at least one* → ∃.
3. Use ⇒ under ∀, ∧ under ∃.
4. "Not all..." → put ¬ in front of ∀; "No one.../None..." → ¬∃ (or ∀ ... ⇒ ¬...).

5.7 Using FOL: assertions, queries and domains

Sentences are added to a KB with **TELL** (exactly as in PL) — these are **assertions**:

```
TELL(KB, King(John))        TELL(KB, Person(Richard))        TELL(KB, ∀x King(x) ⇒ Person(x))
```

Questions are asked with **ASK** — these are **queries** or **goals**: `ASK(KB, King(John))` → true. `ASK(KB, ∃x Person(x))` → true; **ASKVARS**(KB, Person(x)) returns a **substitution** (binding list) like {x/John} and {x/Richard}.

A **domain** is some part of the world about which we wish to express knowledge — e.g. the **kinship domain** and the **Wumpus world**.

The kinship (family) domain

- **Objects:** people
- **Unary predicates:** Male, Female
- **Binary predicates:** Parent, Sibling, Brother, Sister, Child, Daughter, Son, Spouse, Wife, Husband, Grandparent, Grandchild, Cousin, Aunt, Uncle
- **Functions:** Mother, Father
- Facts: "Elizabeth is the mother of Charles", "Charles is the father of William". Rule: "One's grandmother is the mother of one's parent".

Brothers are siblings: ∀x,y Brother(x,y) ⇒ Sibling(x,y)
Male and female are disjoint: ∀x Male(x) ⇔ ¬Female(x)
Parent/child are inverse: ∀p,c Parent(p,c) ⇔ Child(c,p)
Mother definition: ∀m,c Mother(c)=m ⇔ Female(m) ∧ Parent(m,c)
Husband: ∀w,h Husband(h,w) ⇔ Male(h) ∧ Spouse(h,w)
Grandparent: ∀g,c Grandparent(g,c) ⇔ ∃p Parent(g,p) ∧ Parent(p,c)
Sibling: ∀x,y Sibling(x,y) ⇔ x≠y ∧ ∃p Parent(p,x) ∧ Parent(p,y)

The Wumpus world in FOL

- **Percept** = predicate with the percept list and time: Percept([Stench, Breeze, Glitter, None, None], 5)
- **Actions** are logical terms: Turn(Right), Turn(Left), Forward, Shoot, Grab, Release, Climb. **Query:** ∃a BestAction(a, 5) → answer {a/Grab}.
- **Perception** (percept ⇒ fact): ∀t,s,b,m,c Percept([s, b, Glitter, m, c], t) ⇒ Glitter(t)
- **Reflex behaviour:** ∀t Glitter(t) ⇒ BestAction(Grab, t)
- **Environment:** objects = squares, pits, wumpus. Adjacency: ∀x,y,a,b Adjacent([x,y],[a,b]) ⇔ [a,b] ∈ {[x+1,y], [x−1,y],[x,y+1],[x,y−1]}; unary predicate Pit(x).
- Agent infers square properties from its percept: ∀s,t At(Agent, s, t) ∧ Breeze(t) ⇒ Breezy(s) (Breezy has no time argument). Having found which squares are breezy/smelly, the agent can deduce where the pits and the wumpus are: ∀s Breezy(s) ⇔ ∃r Adjacent(r, s) ∧ Pit(r).

5.8 Knowledge engineering in FOL ★ Very likely

DEFINITION

Knowledge engineering is the general process of constructing a knowledge base. A **knowledge engineer** investigates a particular domain, learns what concepts are important in it, and creates a formal representation of the objects and relations in the domain.

The 7 steps of the knowledge-engineering process

1. **Identify the task** — figure out the range of questions the KB must answer.
2. **Assemble the relevant knowledge** — knowledge acquisition / requirement analysis (work with domain experts).
3. **Decide on a vocabulary** of predicates, functions and constants — this vocabulary is the domain's **ontology**.
4. **Encode general knowledge about the domain** — write axioms (rules) that hold for all instances.
5. **Encode a description of the specific problem instance** — simple atomic sentences about this case.
6. **Pose queries to the inference procedure and get answers.**
7. **Debug the knowledge base** — fix missing or wrong axioms.

MEMORY TRICK

"**I** **A**m **D**oing **G**reat **S**o **Q**uit **D**oubting" → **I**dentify, **A**ssemble, **D**ecide vocabulary, **G**eneral knowledge, **S**pecific instance, **Q**uery, **D**ebug.

Worked example: the one-bit full adder (electronic circuits domain)

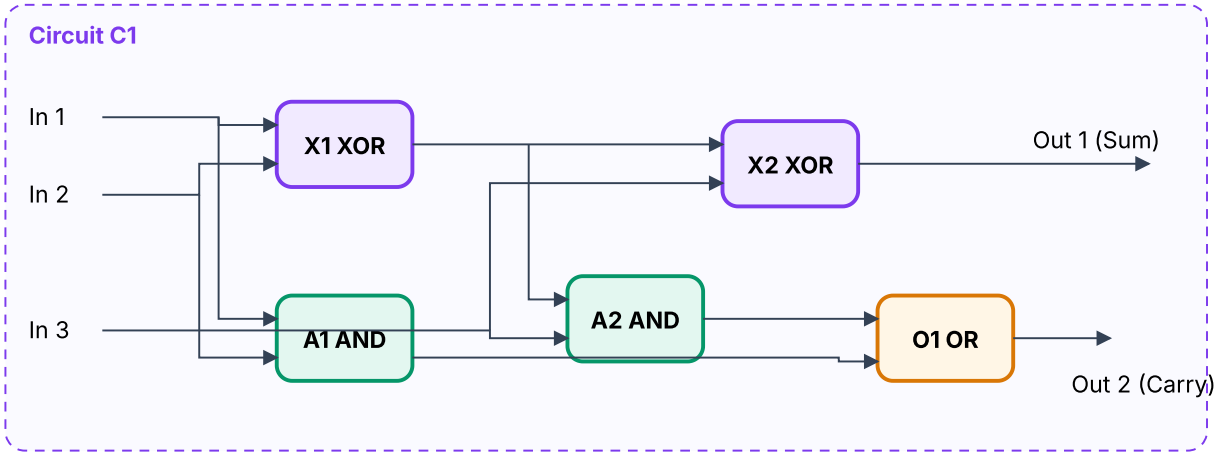


Fig 5.2 One-bit full adder C1: two XOR gates (X1, X2), two AND gates (A1, A2), one OR gate (O1). Out1 = sum bit, Out2 = carry bit.

1. **Identify the task.** Level 1 (functionality): *Does the circuit add properly? What is the output of gate A2 if all inputs are high?* Level 2 (structure): *Which gate is connected to the first input terminal? Does the circuit have feedback loops?*
2. **Assemble knowledge.** Circuits are made of **wires** and **gates**. Signals flow along wires to gate input terminals; each gate produces an output that flows on. Four gate types: AND, OR, XOR, NOT. All have two inputs and one output, except NOT (one input). Delays, power, cost are irrelevant here.
3. **Decide vocabulary.** Gates are objects named by constants (X1, X2, A1, A2, O1) — $\mathrm{Gate}(\mathrm{X1})$; gate type via function: $\mathrm{Type}(\mathrm{X1}) = \mathrm{XOR}$; circuit predicate $\mathrm{Circuit}(\mathrm{C1})$; terminals $\mathrm{Terminal}(\mathrm{x})$; input/output terminals by functions $\mathrm{In}(1, \mathrm{X1})$, $\mathrm{Out}(1, \mathrm{X1})$; connectivity $\mathrm{Connected}(\mathrm{Out}(1, \mathrm{X1}), \mathrm{In}(1, \mathrm{X2}))$; signal values 1 and 0 and function $\mathrm{Signal}(\mathrm{t})$.
4. **Encode general knowledge** (domain axioms):

If two terminals are connected, they have the same signal: $\forall t_1,t_2$ Terminal(t_1) ∧ Terminal(t_2) ∧ Connected(t_1,t_2) ⇒ Signal(t_1) = Signal(t_2)
Signal is either 1 or 0: $\forall t$ Terminal(t) ⇒ Signal(t) = 1 ∨ Signal(t) = 0 ; and 1 ≠ 0
Connected is commutative: $\forall t_1,t_2$ Connected(t_1,t_2) ⇔ Connected(t_2,t_1)
AND gate: ∀g Gate(g) ∧ Type(g)=AND ⇒ [Signal(Out(1,g)) = 0 ⇔ ∃n Signal(In(n,g)) = 0]
OR gate: ∀g Gate(g) ∧ Type(g)=OR ⇒ [Signal(Out(1,g)) = 1 ⇔ ∃n Signal(In(n,g)) = 1]
XOR gate: ∀g Gate(g) ∧ Type(g)=XOR ⇒ [Signal(Out(1,g)) = 1 ⇔ Signal(In(1,g)) ≠ Signal(In(2,g))]
NOT gate: ∀g Gate(g) ∧ Type(g)=NOT ⇒ Signal(Out(1,g)) ≠ Signal(In(1,g))

5. **Encode the specific instance:** Circuit(C1) ∧ Arity(C1,3,2); Gate(X1) ∧ Type(X1)=XOR, Gate(X2) ∧ Type(X2)=XOR, Gate(A1) ∧ Type(A1)=AND, Gate(A2) ∧ Type(A2)=AND, Gate(O1) ∧ Type(O1)=OR; connections such as Connected(Out(1,X1), In(1,X2)), Connected(In(1,C1), In(1,X1)), Connected(Out(1,X1), In(2,A2)), Connected(Out(1,A2), In(1,O1)), Connected(Out(1,A1), In(2,O1)), Connected(Out(1,X2), Out(1,C1)), Connected(Out(1,O1), Out(2,C1)), ...
6. **Pose queries.** "What combination of inputs makes the first output of C1 = 0 and the second output = 1?"
$\exists i_1,i_2,i_3$ Signal(In(1,C1))=i_1 ∧ Signal(In(2,C1))=i_2 ∧ Signal(In(3,C1))=i_3 ∧ Signal(Out(1,C1))=0 ∧ Signal(Out(2,C1))=1
Answers: (i_1,i_2,i_3) = (1,1,0), (1,0,1), (0,1,1) — i.e. exactly two inputs are 1 (sum 0, carry 1). Another query: find all possible sets of values of all terminals → the full adder truth table.
7. **Debug the KB.** E.g. if we forgot to assert 1 ≠ 0, some queries return nothing; check outputs of every gate against expectations.

LIKELY EXAM QUESTIONS — CH 5

1. Differentiate propositional logic and first-order logic. (5M)
2. Explain the syntax of FOL with its basic elements; define term, atomic sentence and complex sentence. (6M)
3. Explain universal and existential quantifiers with examples. Why is ⇒ used with ∀ and ∧ with ∃? (5M)
4. Translate the given English sentences into FOL. (5–6M — almost always asked)
5. What is knowledge engineering? Explain the steps with the electronic-circuit (full adder) example. (10M)
6. Write FOL axioms for the kinship domain / Wumpus world. (5M)

60-SECOND RECAP

FOL = objects + relations + functions + variables + quantifiers. Term → object; atomic sentence → true/false. ∀ with ⇒, ∃ with ∧; ¬∀ = ∃¬. TELL adds assertions, ASK poses queries. Knowledge engineering: Identify task → Assemble knowledge → Decide vocabulary → General axioms → Specific instance → Query → Debug.

6 Inference in First-Order Logic

Inference = deriving a *new* sentence from sentences we already know. Example: from $\text{Human}(\text{Socrates})$ and $\forall x\ \text{Human}(x) \Rightarrow \text{Mortal}(x)$ we derive $\text{Mortal}(\text{Socrates})$.

Method	Basic idea (one line to remember)	Starts with	Direction
Propositional inference	Convert FOL sentences into specific (ground) propositions, then use PL inference	Ground propositions	Depends on method
Unification	Find substitutions that make two expressions identical	Two FOL expressions	Matching
Lifting	Perform inference directly with variables (Generalized Modus Ponens)	FOL rules + facts	Generalized
Forward chaining	Start from facts and derive new facts	Known facts	Forward (facts → goal)
Backward chaining	Start from goal and find supporting facts	Query / goal	Backward (goal → facts)
Resolution	Prove a statement by contradiction	KB + negated query	Refutation

6.1 Propositional vs first-order inference

Propositional logic works with simple statements that are either true or false (`JohnIsKing`, `JohnIsGreedy`, `JohnIsEvil`). FOL works with objects, properties, relationships and variables ($\text{King}(\text{John})$, $\forall x\ \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$). One way to do FOL inference is to remove the quantifiers (instantiate the variables) and then use propositional inference.

Substitution

DEFINITION

A **substitution** θ is a finite set $\{v_1/t_1, \ldots, v_n/t_n\}$ where each v_i is a **variable**, each t_i is a **term** different from v_i, and no variable appears twice. $\text{SUBST}(\theta, \alpha)$ = the sentence obtained by applying θ to α.

$$\alpha = \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x), \quad \theta = \{x/\text{John}\}$$
$$\text{SUBST}(\theta, \alpha) = \text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$$

Notation: the textbook writes {x/John} ("x is replaced by John"). Some class slides write it the other way round, [John/x] or {t/v}. Both mean the same — just be consistent in your answer.

Inference rules for quantifiers ★ Very likely

Rule	Form	Meaning & example
Universal Instantiation (UI) a.k.a. universal elimination	$\frac{\forall v\ \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$ g = any ground term	From "for all" infer it for any specific object. Can be applied many times. "Every person likes ice-cream" ⊢ "John likes ice-cream". From $\forall x\ \text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ infer $\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$, $\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$, $\text{King}(\text{Father}(\text{John})) \wedge \ldots$
Existential Instantiation (EI) a.k.a. existential elimination	$\frac{\exists v\ \alpha}{\text{SUBST}(\{v/k\}, \alpha)}$ k = NEW constant	Give the "something" a brand-new name. Applied only once; the constant must not appear anywhere else in the KB. $\exists x\ \text{Crown}(x) \wedge \text{OnHead}(x, \text{John}) \vdash \text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$. The new constant is a **Skolem constant**; EI is a special case of **Skolemization**. New KB is not logically equivalent to the old one, but is **satisfiable iff** the old one was (inferentially equivalent).
Universal Generalization (UG)	$\frac{P(c) \text{ for arbitrary } c}{\forall x\ P(x)}$	If P(c) holds for an **arbitrary** element c, conclude ∀x P(x). "A byte contains 8 bits" (any byte) ⊢ "All bytes contain 8 bits".
Existential Introduction / Generalization (EG)	$\frac{P(c)}{\exists x\ P(x)}$	If some specific c has property P, then something has P. "Pinky got good marks in English" ⊢ "Someone got good marks in English".

MEMORY TRICK

UI = **U**se any constant, as many times as you like. **E**I = **E**xactly once, with a fr**e**sh name.

Propositionalization (reduction to propositional inference)

Apply UI with **every** ground term and EI once per ∃ → the KB becomes propositional (treat $\text{King}(\text{John})$ as one symbol like `JohnIsKing`) → use any PL algorithm. **Problems:**

- With function symbols there are infinitely many ground terms: Father(John), Father(Father(John)), … (Herbrand's theorem: if α is entailed, a proof exists using a finite subset — so the method is complete but only **semidecidable**: it may never halt if α is not entailed).
- It generates lots of irrelevant sentences (e.g. $\text{Greedy}(\text{Richard})$ …) — very inefficient. This motivates **lifting**.

6.2 Unification ★ Very likely

DEFINITION

Unification is the process of finding a substitution θ that makes two different logical expressions look identical:

$$\text{UNIFY}(p, q) = \theta \quad \text{where} \quad \text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$$

θ is called the **unifier**. Unification is the key component of all first-order inference algorithms (forward chaining, backward chaining, resolution).

Conditions for unification

1. **Predicate symbols must be the same** — atoms with different predicate symbols can never be unified (Knows(...) vs Loves(...) → FAIL).
2. **Number of arguments must be identical** (Knows(A, B) vs Knows(A) → FAIL).
3. A variable can't take two different values at once, and a variable can't be unified with a term containing itself (**occurs check**: x vs F(x) → FAIL).

Worked examples (textbook + class)

p	q	θ = UNIFY(p, q)
Knows(John, x)	Knows(John, Jane)	{x/Jane}
Knows(John, x)	Knows(y, Bill)	{x/Bill, y/John}
Knows(John, x)	Knows(y, Mother(y))	{y/John, x/Mother(John)}
Knows(John, x)	Knows(x, Elizabeth)	**FAIL** (x can't be John and Elizabeth)
p(x, F(y))	p(a, F(g(z)))	{x/a, y/g(z)} (slide: [a/x, g(z)/y])
Q(a, g(x, a), f(y))	Q(a, g(f(b), a), x)	{x/f(b), y/b} → both become Q(a, g(f(b), a), f(b))
Student(x)	Student(Anu)	{x/Anu}

WORKED EXAMPLE — STEP BY STEP

Unify Q(a, g(x, a), f(y)) and Q(a, g(f(b), a), x):

1. Same predicate Q, 3 arguments each ✔.
2. Arg 1: a = a ✔ (no substitution).
3. Arg 2: g(x, a) vs g(f(b), a) → x = f(b). Substitution so far {x/f(b)}. Apply it to the rest: arg 3 of q becomes f(b).
4. Arg 3: f(y) vs f(b) → y = b. Final θ = **{x/f(b), y/b}**.

Standardizing apart

The failure of Knows(John, x) vs Knows(x, Elizabeth) is only because both sentences happen to use the name x. Variables in different sentences are independent, so we rename the variables of one sentence before unifying: Knows(x_{17}, Elizabeth) → θ = {x/Elizabeth, x_{17}/John}. Essential for backward chaining and resolution.

Most General Unifier (MGU)

There can be many unifiers. UNIFY(Knows(John, x), Knows(y, z)) could return {y/John, x/z} or {y/John, x/John, z/John}. The first is **more general** (places fewer restrictions). For every unifiable pair there is a single MGU, unique up to renaming — algorithms always return it.

The unification algorithm (class version)

```
UNIFY(L1, L2)
Step 1: If L1 or L2 is a variable or constant:
    a) If L1 and L2 are identical → return NIL (empty substitution)
    b) Else if L1 is a variable: if L1 occurs in L2 → FAIL, else return {L1/L2}
    c) Else if L2 is a variable: if L2 occurs in L1 → FAIL, else return {L2/L1}
    d) Else → FAIL
Step 2: If the initial predicate symbols of L1 and L2 differ → FAIL
Step 3: If L1 and L2 have a different number of arguments → FAIL
Step 4: SUBST ← NIL
Step 5: For i ← 1 to number of arguments in L1:
    a) S ← UNIFY(i-th argument of L1, i-th argument of L2)
    b) If S = FAIL → return FAIL
    c) If S ≠ NIL → apply S to the remainders of L1 and L2;  SUBST ← APPEND(S, SUBST)
Step 6: Return SUBST
```

Textbook version: `UNIFY(x, y, θ)` — if θ = failure return failure; if x = y return θ; if x is a variable return UNIFY-VAR(x, y, θ); if y is a variable return UNIFY-VAR(y, x, θ); if both compound, unify their operators then their argument lists; if both lists, unify FIRST then REST; else failure. `UNIFY-VAR` does the occurs check.

6.3 Lifting & Generalized Modus Ponens ★ Very likely

DEFINITION

Lifting is applying inference rules directly to FOL sentences containing variables (using unification), instead of first converting everything into propositional logic. Lifting was introduced by **Alan Robinson** — it was the key innovation of his resolution procedure. The lifted version of Modus Ponens is **Generalized Modus Ponens (GMP)**.

$$\frac{p_1', p_2', \ldots, p_n', \quad (p_1 \wedge p_2 \wedge \ldots \wedge p_n \Rightarrow q)}{SUBST(\theta, q)} \quad \text{where } SUBST(\theta, p_i') = SUBST(\theta, p_i) \text{ for all } i$$

n atomic sentences p_i' + one implication → if one θ makes every p_i' match p_i, conclude q with θ applied.

EXAMPLE 1 (TEXTBOOK)

p_1' = King(John), p_2' = Greedy(y)
rule: King(x) ∧ Greedy(x) ⇒ Evil(x)
θ = {x/John, y/John}
SUBST(θ, q) = **Evil(John)**

EXAMPLE 2 (CLASS)

facts: Student(John), Studies(John)
rule: Student(x) ∧ Studies(x) ⇒ Pass(x)
θ = {x/John}
Pass(x) becomes **Pass(John)**

WHY LIFTING MATTERS

Without lifting we'd propositionalize: Student(John) ⇒ Pass(John), Student(Ravi) ⇒ Pass(Ravi), Student(Sita) ⇒ Pass(Sita)... With lifting we keep $\text{Student}(x) \Rightarrow \text{Pass}(x)$ and use {x/John} **only when needed**. So lifting avoids unnecessary grounding of all possible objects. GMP is sound.

6.4 Forward chaining ★ Very likely

The **inference engine** is the part of a system that applies logical rules to the KB to deduce new information. It works in two modes: **forward chaining** (forward reasoning) and **backward chaining**

(backward reasoning).

DEFINITION

Forward chaining starts from the known facts, triggers all rules whose premises are satisfied, and adds their conclusions to the known facts. The process repeats until the query is answered or no new facts are added (a *fixed point*). It is **data-driven**: Facts → Rules → New facts → Rules → ...

It works on **first-order definite clauses**: disjunctions of literals with **exactly one positive literal**, i.e. either an atomic fact (King(John)) or an implication whose premise is a conjunction of positive literals (King(x) ∧ Greedy(x) ⇒ Evil(x)).

WARM-UP EXAMPLE

Facts: Student(John), Studies(John). Rule 1: Student(x) ∧ Studies(x) ⇒ Pass(x). Rule 2: Pass(x) ⇒ Graduate(x). Query: Graduate(John)?
Step 1: Rule 1 matches with {x/John} → add **Pass(John)**. **Step 2:** Rule 2 matches → add **Graduate(John)** ✔.

The classic crime example ("Colonel West") ★ Very likely

PROBLEM

"The law says that it is a crime for an American to sell weapons to hostile nations. The country Nono, an enemy of America, has some missiles, and all of its missiles were sold to it by Colonel West, who is American." Prove that West is a criminal.

Step 1 — convert every sentence into a first-order definite clause (x, y, z are variables):

#	English	Definite clause
1	It is a crime for an American to sell weapons to hostile nations	$\text{American}(x) \land \text{Weapon}(y) \land \text{Sells}(x, y, z) \land \text{Hostile}(z) \Rightarrow \text{Criminal}(x)$
2	Nono has some missiles: ∃x Owns(Nono, x) ∧ Missile(x) — use EI with new constant M_1	$\text{Owns}(\text{Nono}, M_1)$
3	(same sentence, second half)	$\text{Missile}(M_1)$
4	All of its missiles were sold to it by Colonel West	$\text{Missile}(x) \land \text{Owns}(\text{Nono}, x) \Rightarrow \text{Sells}(\text{West}, x, \text{Nono})$
5	Missiles are weapons (background knowledge)	$\text{Missile}(x) \Rightarrow \text{Weapon}(x)$
6	An enemy of America counts as hostile (background)	$\text{Enemy}(x, \text{America}) \Rightarrow \text{Hostile}(x)$
7	West, who is American	$\text{American}(\text{West})$
8	Nono, an enemy of America	$\text{Enemy}(\text{Nono}, \text{America})$

Step 2 — iterate. Iteration 0 (facts with no implications): American(West), Missile(M_1), Owns(Nono, M_1), Enemy(Nono, America).
Iteration 1: rule 4 {x/M_1} → **Sells(West, M_1, Nono)**; rule 5 {x/M_1} → **Weapon(M_1)**; rule 6 {x/Nono} → **Hostile(Nono)**.
Iteration 2: rule 1 {x/West, y/M_1, z/Nono} → **Criminal(West)** ✔ — goal reached; no new facts possible after this (fixed point).

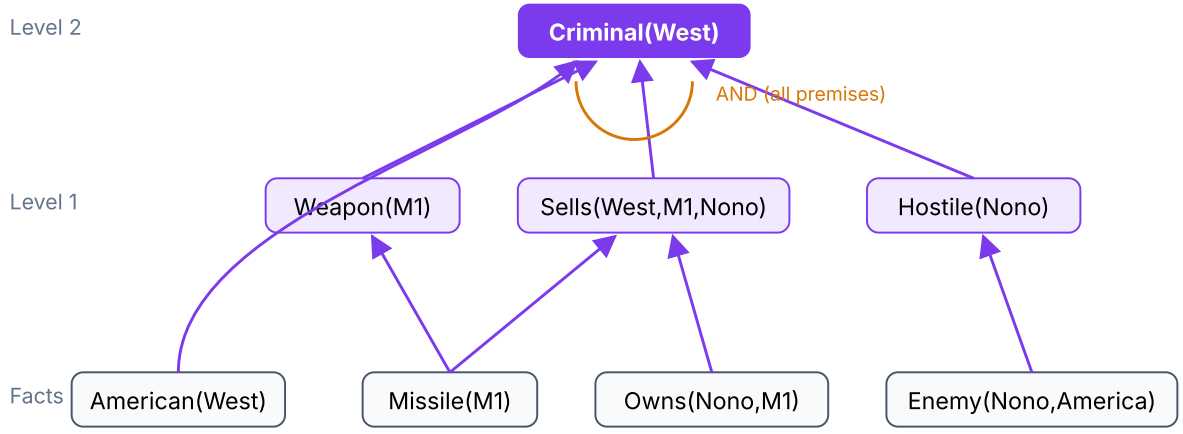


Fig 6.1 Forward-chaining proof tree: facts at the bottom, facts inferred in iteration 1 in the middle, Criminal(West) inferred in iteration 2 at the top.

EXAM TIP — HOW TO WRITE THIS 10-MARK ANSWER

(1) Write the 8 definite clauses in a table. (2) Show iteration-by-iteration which rule fires with which substitution. (3) Draw the proof tree (bottom-up). (4) Conclude "Hence Criminal(West) is proved". Mention **EI** for "Nono has some missiles" — examiners look for that.

Second class example: "John likes peanuts" (forward chaining)

Sentences: (1) John likes all kinds of food. (2) Apples are food. (3) Chicken is food. (4) Anything anyone eats and isn't killed by is food. (5) Bill eats peanuts and is still alive. (6) Sue eats everything Bill eats.

(i) Translation into predicate logic

1. ∀x Food(x) ⇒ Likes(John, x)
2. Food(Apple)
3. Food(Chicken)
4. ∀x ∀y Eats(x, y) ∧ ¬Killed(x) ⇒ Food(y)
5. Eats(Bill, Peanut) ∧ ¬Killed(Bill)
6. ∀x Eats(Bill, x) ⇒ Eats(Sue, x)

(ii) Forward chaining — working memory (WM): Food(Apple), Food(Chicken), Eats(Bill, Peanut), ¬Killed(Bill)

- Rule 6, {x/Peanut}: add **Eats(Sue, Peanut)**
- Rule 4, {x/Bill, y/Peanut}: add **Food(Peanut)**
- Rule 1, {x/Peanut}: add **Likes(John, Peanut)** ✓

Hence proved by forward chaining.

Properties, advantages and disadvantages of forward chaining

- **Sound and complete** for first-order definite clauses. **Datalog** = first-order definite clauses with **no function symbols** (e.g. the crime KB) — FC always terminates on Datalog in polynomial time.
- With function symbols FC may run forever if the query is not entailed (semidecidable).
- **Advantages:** data-driven (starts from available facts); can derive many conclusions until a fixed point; widely used in **deductive databases** and **production systems** (expert systems, e.g. rule engines).
- **Disadvantage:** with 1000 rules and the question "Is John a graduate?", FC may derive many facts irrelevant to the goal — the main source of inefficiency → motivates backward chaining.

FOL-FC-ASK algorithm (idea): repeat { new ← {}; for each rule, standardize it apart; for each θ such that SUBST(θ, $p_1 \wedge \ldots \wedge p_n$) = SUBST(θ, $p_1' \wedge \ldots \wedge p_n'$) for facts p′ in KB: q′ ← SUBST(θ, q); if q′ is not a renaming of a known sentence, add to new; if q′ unifies with the query, return the unifier } until new is empty; add new to KB.

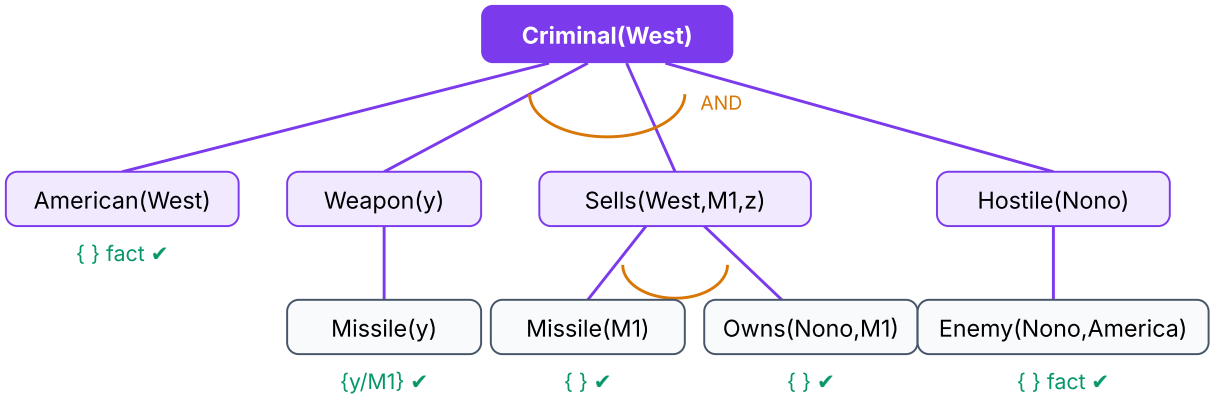
6.5 Backward chaining ★ Very likely

DEFINITION

Backward chaining starts from the **goal (query)** and works backwards: find rules whose conclusion unifies with the goal, and turn their premises into **new sub-goals**, recursively, until all sub-goals are known facts. It is **goal-driven / goal-directed reasoning**: Goal → Rules → Sub-goals → Facts. Backward chaining is a depth-first search algorithm.

WARM-UP EXAMPLE

Rule: Student(x) ∧ Studies(x) ⇒ Pass(x). Facts: Student(John), Studies(John). Query: Pass(John)? **Step 1** Start with goal Pass(John); ask "which rule can conclude Pass(x)?" → the rule above. **Step 2** Unify Pass(x) with Pass(John): x = John. **Step 3** New sub-goals: Student(John) AND Studies(John). **Step 4** Both are facts → Pass(John) is proved ✔.



Read top-down, left-to-right (depth-first). Substitutions found: {x/West, y/M1, z/Nono}

Fig 6.2 Backward-chaining proof tree for Criminal(West). Each sub-goal is proved left to right, depth first; the curved arc marks an AND node.

"John likes peanuts" by backward chaining

```
Goal: Likes(John, Peanut)
  └─ use rule 1 {x/Peanut}:  Food(Peanut)
       └─ use rule 4 {y/Peanut}:  Eats(x, Peanut) ∧ ¬Killed(x)
            ├─ Eats(x, Peanut)  {x/Bill}  → Eats(Bill, Peanut)   fact 5 ✔
            └─ ¬Killed(x)       {x/Bill}  → ¬Killed(Bill)        fact 5 ✔
  ⇒ Food(Peanut) ✔  ⇒  Likes(John, Peanut) ✔
```

AND/OR interpretation

Backward chaining is an **AND/OR search**. **OR**: several rules may prove the same goal — e.g. Pass(John) via $\text{Student}(x) \wedge \text{Studies}(x) \Rightarrow \text{Pass}(x)$ or $\text{TopRank}(x) \Rightarrow \text{Pass}(x)$; any one succeeding is enough. **AND**: once a rule is chosen, all its premises (Student(John) AND Studies(John)) must be proved.

ADVANTAGES

- Goal-directed; avoids many irrelevant facts
- Good for answering specific queries
- Foundation of **logic programming**; **Prolog** uses depth-first backward chaining

DISADVANTAGES

- Repeated states & redundant computations
- Can fall into **infinite loops** (with unsuitable rule ordering)
- Incomplete as plain DFS

6.6 Forward vs backward chaining ★ Very likely

Feature	Forward chaining	Backward chaining
Also called	Data-driven, bottom-up	Goal-driven, top-down
Starting point	Known facts	Goal / query
Direction	Facts → conclusion	Goal → premises (sub-goals)
Main question	"What can I derive?"	"What do I need to prove?"
Irrelevant inference	More likely	Less likely
Search style	Breadth-first-like, data-driven	Depth-first, AND/OR search
Useful for	Databases, production/expert systems, monitoring	Query answering, diagnosis, logic programming
Used by Prolog?	No	Yes
Infinite loops	Generally not (finite Datalog closure)	Yes, with unsuitable rule ordering
Example	Crime KB iterations up to Criminal(West)	Criminal(West) tree down to facts

6.7 Resolution ★ Very likely

DEFINITION

Resolution is a theorem-proving technique that builds **refutation proofs** (proofs by contradiction). It was invented by mathematician **John Alan Robinson in 1965**. To prove $KB \models \alpha$, we show that $KB \wedge \neg\alpha$ is **unsatisfiable** by deriving the **empty clause** $\square$ (a contradiction).

$$KB \models \alpha \iff KB \wedge \neg\alpha \text{ is unsatisfiable} \iff KB \wedge \neg\alpha \vdash \square$$

The resolution rule

Propositional:

$$\frac{P \vee Q, \quad \neg Q}{P} \qquad \frac{\ell_1 \vee \ldots \vee \ell_k, \quad m_1 \vee \ldots \vee m_n}{\text{remaining literals}}$$

ℓ_i and m_j are complementary (Q and ¬Q): remove both, OR the rest.

First-order: complementary literals need not look identical — they must **unify**: if UNIFY(ℓ_i, $\neg m_j$) = θ, the resolvent is SUBST(θ, rest of both clauses)

Terminology on the class slides: the pair of clauses being resolved are the **resolvents**' parents (slide: "resolvents"); the resulting clause is the **resolvent** (slide: "resolute"). Choosing the right pair to resolve is a matter of search.

SIMPLE FOL RESOLUTION STEP (TEXTBOOK)

C1: $Animal(F(x)) \vee Loves(G(x), x)$ C2: $\neg Loves(u, v) \vee \neg Kills(u, v)$

Step 1 unify Loves(G(x), x) with Loves(u, v): θ = {u/G(x), v/x}. Step 2 remove the complementary pair. Step 3 apply θ → resolvent **Animal(F(x)) ∨ ¬Kills(G(x), x)**.

Converting FOL to CNF (clause form) ★ Very likely

Resolution needs every sentence in **Conjunctive Normal Form**: a conjunction (∧) of clauses, where each clause is a disjunction (∨) of literals, e.g. $(A \vee B) \wedge (\neg C \vee D)$.

#	Step	What to do	Example
1	**Eliminate ⇔ and ⇒**	P ⇔ Q → (P ⇒ Q) ∧ (Q ⇒ P); P ⇒ Q → ¬P ∨ Q	P ⇒ Q ≡ ¬P ∨ Q
2	**Move ¬ inwards**	¬¬P → P; De Morgan; ¬∀x P → ∃x ¬P; ¬∃x P → ∀x ¬P	¬(P ∧ Q) ≡ ¬P ∨ ¬Q
3	**Standardize variables**	Each quantifier gets its own unique variable name	∀x Man(x), ∀x Happy(x) → ∀x Man(x), ∀y Happy(y)
4	Move quantifiers left (prenex form)	Bring all quantifiers to the front	P(x) ∧ ∀y Q(y) → ∀y [P(x) ∧ Q(y)]
5	**Skolemize** (remove ∃)	Replace ∃ variable by a **Skolem constant**, or a **Skolem function** of the enclosing ∀ variables	∃x Smile(x) → Smile(A); ∀y ∃x Loves(x, y) → ∀y Loves(F(y), y)
6	**Drop ∀**	All remaining variables are implicitly universal	∀x Smile(x) → Smile(x)
7	**Distribute ∨ over ∧**	(P ∧ Q) ∨ R → (P ∨ R) ∧ (Q ∨ R); (P ∨ Q) ∨ R → P ∨ Q ∨ R	(P ∧ Q) ∨ R → (P ∨ R) ∧ (Q ∨ R)
8	**Split conjuncts** into separate clauses	Each conjunct becomes one clause	(P ∨ R) ∧ (Q ∨ R) → C1: P ∨ R, C2: Q ∨ R
9	**Standardize apart** again	No two clauses share a variable name	P(x) ∨ Q(x), ¬P(x) ∨ R(x) → … ¬P(y) ∨ R(y)

SKOLEM CONSTANT VS SKOLEM FUNCTION

If an ∃ sits **inside** a ∀, the "something" may depend on the ∀ variable, so use a **function**: "Everyone has a heart" ∀x ∃y Heart(y, x) → Heart(H(x), x), not Heart(H, x) (which would mean everyone shares one heart!).

FULL CNF CONVERSION (TEXTBOOK): "EVERYONE WHO LOVES ALL ANIMALS IS LOVED BY SOMEONE"

∀x [∀y Animal(y) ⇒ Loves(x, y)] ⇒ [∃y Loves(y, x)]

1. ⇒ elim: ∀x ¬[∀y ¬Animal(y) ∨ Loves(x, y)] ∨ [∃y Loves(y, x)]
2. ¬ in: ∀x [∃y ¬(¬Animal(y) ∨ Loves(x, y))] ∨ [∃y Loves(y, x)] → ∀x [∃y Animal(y) ∧ ¬Loves(x, y)] ∨ [∃y Loves(y, x)]
3. standardize: ∀x [∃y Animal(y) ∧ ¬Loves(x, y)] ∨ [∃z Loves(z, x)]
5. Skolemize (y and z depend on x): ∀x [Animal(F(x)) ∧ ¬Loves(x, F(x))] ∨ Loves(G(x), x)
6. drop ∀: [Animal(F(x)) ∧ ¬Loves(x, F(x))] ∨ Loves(G(x), x)
7. distribute: **[Animal(F(x)) ∨ Loves(G(x), x)] ∧ [¬Loves(x, F(x)) ∨ Loves(G(x), x)]** — two clauses.

Steps of a resolution-refutation proof (write these in every answer)

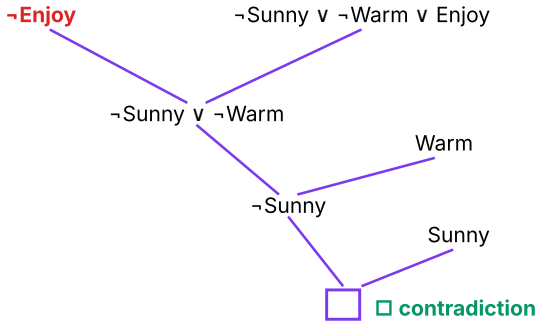
1. Convert all facts/axioms into **FOL**.
2. Convert the FOL sentences into **clause form (CNF)**.
3. **Negate the statement to be proved**, convert to clause form, and add it to the KB.
4. **Resolve** pairs of clauses (using unification), producing new clauses that logically follow; draw the **resolution graph**.

5 If the **empty clause (NIL, □)** is produced → contradiction → the original statement is **true**. The substitutions used tell *under which bindings* the goal is true (answer extraction).

Example 1 (PL): "You will enjoy"

Facts: (1) If it is sunny and warm, you will enjoy. (2) If it is raining you will get wet. (3) It is warm. (4) It is raining. (5) It is sunny. Prove: enjoy.

FOL → CNF
Sunny ∧ Warm ⇒ Enjoy → ¬(Sunny ∧ Warm) ∨ Enjoy → **¬Sunny ∨ ¬Warm ∨ Enjoy**
Raining ⇒ Wet → **¬Raining ∨ Wet**
Warm, **Raining**, **Sunny**
Negated goal: **¬Enjoy**



The negation of the conclusion leads to a complete contradiction with the given facts. **Hence proved: you will enjoy.** (Note "Raining ∨ Wet" clauses were not needed — that's normal.)

Example 2 (PL): "It will rain"

(1) The humidity is high or the sky is cloudy. (2) If the sky is cloudy, then it will rain. (3) If the humidity is high, then it is hot. (4) It is not hot. Goal: it will rain. Symbols: P = humidity is high, Q = sky is cloudy, R = it will rain, S = it is hot.

#	Statement	FOL	Clause
C1	humidity high or cloudy	P ∨ Q	P ∨ Q
C2	cloudy ⇒ rain	Q ⇒ R	¬Q ∨ R
C3	humid ⇒ hot	P ⇒ S	¬P ∨ S
C4	not hot	¬S	¬S
C5	negated goal		**¬R**

① C5 ¬R + C2 ¬Q ∨ R → **C6: ¬Q**
② C6 ¬Q + C1 P ∨ Q → **C7: P**
③ C7 P + C3 ¬P ∨ S → **C8: S**
④ C8 S + C4 ¬S → □ (empty clause)
Contradiction ⇒ ¬R is false ⇒ **It will rain**
(Slide order: P∨Q + ¬Q∨R → P∨R; + ¬P∨S → R∨S; + ¬S → R; + ¬R → □ — same result.)

Example 3: Socrates / "Dead dog" problem (FOL)

Socrates: C1 Human(Socrates); C2 ¬Human(x) ∨ Mortal(x); C3 ¬Mortal(Socrates) (negated goal). C1 + C2 with {x/Socrates} → Mortal(Socrates); + C3 → □. Hence Mortal(Socrates).

Dead dog: All dogs are animals; Fido is a dog; all animals will die. Prove: Fido will die.
C1 ¬dog(X) ∨ animal(X); C2 dog(fido); C3 ¬animal(Y) ∨ die(Y); C4 ¬die(fido).
C4 + C3 {Y/fido} → ¬animal(fido); + C1 {X/fido} → ¬dog(fido); + C2 → □. Hence die(fido).

Example 4: "Is John happy?" ★ Very likely

"Anyone passing his history exams and winning the lottery is happy. But anyone who studies or is lucky can pass all his exams. John did not study, but John is lucky. Anyone who is lucky wins the lottery."

FOL

∀X pass(X, history) ∧ win(X, lottery) ⇒ happy(X)

∀X ∀Y study(X) ∨ lucky(X) ⇒ pass(X, Y)

¬study(john) ∧ lucky(john)

∀X lucky(X) ⇒ win(X, lottery)

Second rule: ¬(study ∨ lucky) ∨ pass → (¬study ∨ pass) ∧ (¬lucky ∨ pass) → two clauses, standardized apart.

Clauses

1. ¬pass(X, history) ∨ ¬win(X, lottery) ∨ happy(X)
2. ¬study(Y) ∨ pass(Y, Z)
3. ¬lucky(W) ∨ pass(W, V)
4. ¬study(john) 5. lucky(john)
6. ¬lucky(U) ∨ win(U, lottery)
7. **¬happy(john)** (negated goal)

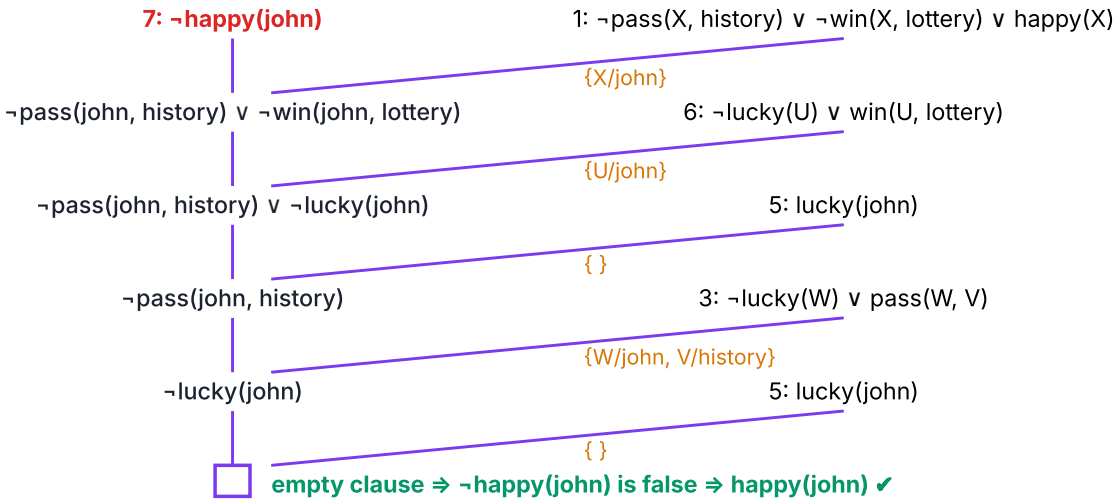


Fig 6.3 Resolution refutation for "John is happy". Left column: resolvents; right column: clause used; orange: substitution.

Example 5: "Exciting life" (answer extraction with ∃)

All people who are not poor and are smart are happy. Those people who read are not stupid. John can read and is wealthy. Happy people have exciting lives. Can anyone be found with an exciting life? Assume smart ≡ ¬stupid and wealthy ≡ ¬poor.

∀X (¬poor(X) ∧ smart(X) ⇒ happy(X)) → **poor(X) ∨ ¬smart(X) ∨ happy(X)**

∀Y (read(Y) ⇒ smart(Y)) → **¬read(Y) ∨ smart(Y)**

read(john) ∧ ¬poor(john) → **read(john)**, **¬poor(john)**

∀Z (happy(Z) ⇒ exciting(Z)) → **¬happy(Z) ∨ exciting(Z)**

Negated goal ¬∃W exciting(W) → **¬exciting(W)**

```
¬exciting(W) + ¬happy(Z)vexciting(Z)   {Z/W}
 → ¬happy(Z) + poor(X)v¬smart(X)vhappy(X)   {X/Z}
 → poor(X)v¬smart(X) + ¬read(Y)vsmart(Y)   {Y/X}
 → poor(Y)v¬read(Y) + ¬poor(john)          {john/Y}
 → ¬read(john) + read(john)  → □
Answer extraction: W = john → exciting(john)
```

Example 6: "Where is Fido?" (answer extraction)

Dog Fido goes wherever John goes: ∀X at(john, X) ⇒ at(fido, X) → **¬at(john, X) ∨ at(fido, X)**. John is at the library: **at(john, library)**. Negated query ¬∃Z at(fido, Z) → **¬at(fido, Z)**. Resolve ¬at(fido, Z) with clause 1 {X/Z} → ¬at(john, X); with at(john, library) {library/X} → □. Keep a copy of the goal at(fido, Z) and apply the same substitutions → **at(fido, library)**.

Example 7: "What does Jade eat?"

"Jade eats whatever Tom eats, and Tom eats banana. What does Jade eat?" (The slide words it as "Tom eats whatever Jade eats", but its FOL — used below — says whatever Tom eats, Jade eats.) 1. $\forall X\ \text{eat}(\text{tom}, X) \Rightarrow \text{eat}(\text{jade}, X)$ → ¬eat(tom, X) ∨ eat(jade, X); 2. eat(tom, banana); negated conclusion ¬∃X eat(jade, X) → 3. ¬eat(jade, Y). Resolve 3 with 1 {Y/X} → ¬eat(tom, Y); with 2 {banana/Y} → □. Answer: **eat(jade, banana)**.

Example 8: "Mohan will deliver an excellent lecture"

All teachers are good: T(X) ⇒ G(X) → **¬T(X) ∨ G(X)**. Anyone good and intelligent delivers excellent lectures: ∀Y G(Y) ∧ I(Y) ⇒ L(Y) → **¬G(Y) ∨ ¬I(Y) ∨ L(Y)**. Mohan is an intelligent teacher: **I(Mohan)**, **T(Mohan)**. Negated goal: **¬L(Mohan)**. ¬L(Mohan) + clause 2 {Mohan/Y} → ¬G(Mohan) ∨ ¬I(Mohan); + I(Mohan) → ¬G(Mohan); + clause 1 {Mohan/X} → ¬T(Mohan); + T(Mohan) → □. **Hence L(Mohan).**

Example 9: "John likes peanuts" by resolution

Clauses: 1. ¬Food(x) ∨ Likes(John, x) 2. Food(Apple) 3. Food(Chicken) 4. ¬Eats(y, z) ∨ Killed(y) ∨ Food(z) 5. Eats(Bill, Peanut) 6. ¬Killed(Bill) 7. ¬Eats(Bill, w) ∨ Eats(Sue, w) 8. **¬Likes(John, Peanut)**
8 + 1 {x/Peanut} → ¬Food(Peanut); + 4 {z/Peanut} → ¬Eats(y, Peanut) ∨ Killed(y); + 5 {y/Bill} → Killed(Bill); + 6 → □. Hence Likes(John, Peanut).

PROPERTIES OF RESOLUTION

Resolution is **sound** and **refutation-complete**: if a set of sentences is unsatisfiable, resolution will always be able to derive a contradiction (□). It can't be used to *generate* all consequences, but it can confirm or refute any given sentence.

EXAM TIP — RESOLUTION ANSWERS

Always show: FOL → CNF (with the step names: "eliminate implication", "move negation inside", "Skolemize", "drop ∀") → numbered clause list → negated goal clearly marked → resolution graph with the substitution written on each edge → "Empty clause obtained, hence proved". Drawing the graph neatly earns marks even if one substitution is off.

6.8 One knowledge base, three methods (summary)

KB	Facts: Student(John), Studies(John). Rule: Student(x) ∧ Studies(x) ⇒ Pass(x). Query: Pass(John)?
Forward	Start from facts → apply rule with {x/John} → Pass(John). Direction: Facts → Goal.
Backward	Start with Pass(John) → rule concluding Pass → unify x = John → sub-goals Student(John), Studies(John) → both facts → proved. Direction: Goal → Facts.
Resolution	Rule → ¬Student(x) ∨ ¬Studies(x) ∨ Pass(x); add ¬Pass(John); resolve with Student(John) → ¬Studies(John) ∨ Pass(John); with Studies(John) → Pass(John); with ¬Pass(John) → □. Direction: Refutation.

Why unification is so important: it is not a separate inference algorithm — it is the matching mechanism used by every FOL method: FC (match Student(x) with fact Student(John)), BC (match goal Pass(John) with rule head Pass(x)), resolution (match P(x) with ¬P(John) to get Q(John) from P(x) ∨ Q(x)). Unification is the bridge that lets FOL inference work with variables.

LIKELY EXAM QUESTIONS — CH 6

1. Explain Universal Instantiation and Existential Instantiation with examples. (5M)
2. What is unification? State its conditions / write the algorithm; find the MGU of the given pairs. (6–8M)
3. Explain lifting and Generalized Modus Ponens with an example. (5M)
4. Prove "West is a criminal" using forward chaining (and/or backward chaining) — draw the proof tree. (10M) ★
5. Prove "John likes peanuts" using forward / backward chaining / resolution. (10M)
6. Differentiate forward and backward chaining. (5M)
7. List the steps to convert FOL to CNF; convert the given sentence. (6M)
8. Using resolution refutation, check whether John is happy / it will rain / you will enjoy. (10M) ★

60-SECOND RECAP

UI: ∀ → any ground term (many times). EI: ∃ → new Skolem constant (once). Unification = substitution making two expressions identical (same predicate, same arity, occurs check); MGU = most general. Lifting/GMP = Modus Ponens with variables. FC = data-driven to fixed point; BC = goal-driven DFS (Prolog). Resolution = add ¬goal, CNF, resolve to □; sound & refutation-complete. CNF steps: ⇒ out, ¬ in, standardize, Skolemize, drop ∀, distribute, split.

7 Knowledge Graphs & Ontologies Self-study topic

From connected facts → meaning → reasoning → intelligent answers. Big idea: **store knowledge in a form that both humans and machines can follow**.

7.1 Why do we need a knowledge graph?

PROBLEM

Information is scattered across tables, documents and apps. Keyword search finds *words* but doesn't understand how facts are *connected*.

IDEA

Represent important things as **entities** and connect them with meaningful **relationships**. The graph becomes a network of facts.

BENEFIT

We can follow connections, combine facts, **infer new knowledge** and answer **multi-step questions**.

A table stores rows; a graph makes relationships easy to follow. Example question: "*Who teaches the course studied by Asha?*" needs following more than one relationship.

7.2 Four words to understand first ★ Very likely

Term	Meaning	Campus examples
Entity	A thing we want to describe	Asha, AIML, CSE, Dr Rao
Relationship	A named connection between two entities	studies, teaches, offeredBy
Class	A category or type	Student, Person, Course, Department
Instance	A specific member of a class	Asha is an instance of Student

MEMORY TIP

Entity = **thing** • Relationship = **connection** • Class = **category** • Instance = **member**

7.3 Knowledge graph = entities + labelled edges

DEFINITION

A **knowledge graph (KG)** is a graph-structured knowledge base in which nodes represent entities (and classes) and labelled directed edges represent relationships between them. Every arrow tells us what the connection means; read an arrow as a sentence: **Asha —studies→ AIML**.

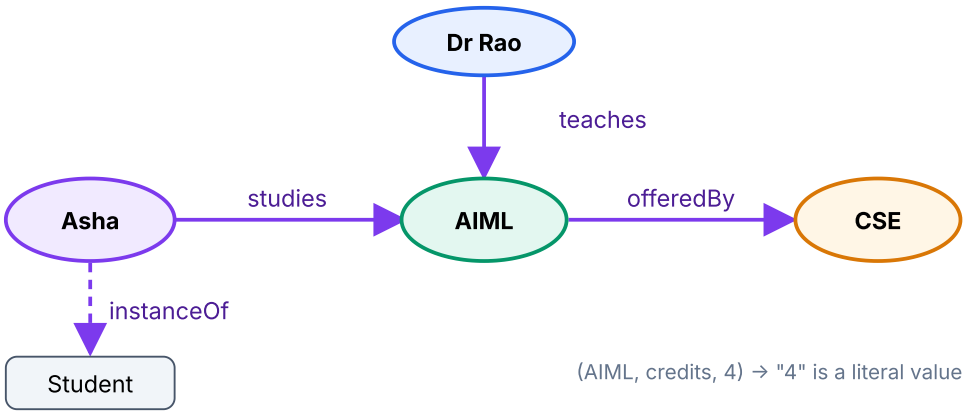


Fig 7.1 A small campus knowledge graph. Entities are nodes; each labelled arrow is one fact.

7.4 RDF triple: one fact in three parts ★ Very likely

Subject — Predicate — Object → (Asha, studies, AIML)

RDF = Resource Description Framework, the W3C standard data model for knowledge graphs.

- **Subject**: the entity the fact is about (Asha). **Predicate**: the relationship/property (studies). **Object**: the value or the other entity (AIML).
- **Literal example**: (AIML, credits, 4) — here "4" is a **value**, not another entity.
- **How triples become a graph:** repeated identifiers connect separate facts. (Asha, studies, AIML), (DrRao, teaches, AIML), (AIML, offeredBy, CSE), (Asha, instanceOf, Student) — the same AIML node links them. The graph is not drawn separately; it *emerges* because identifiers are reused.

RDF written in Turtle syntax

```
@prefix ex: <https://example.org/campus/> .
ex:Asha   ex:studies    ex:AIML .
ex:AIML   ex:offeredBy  ex:CSE .
ex:DrRao  ex:teaches    ex:AIML .
```

Turtle is a compact, human-readable syntax for RDF. `@prefix` creates a short name (ex:) for a long **IRI** (internationalized resource identifier); `ex:Asha` identifies the resource Asha in this namespace; the full stop **.** ends one statement.

7.5 Knowledge graph vs ontology ★ Very likely

KNOWLEDGE GRAPH — STORES PARTICULAR FACTS

- Asha studies AIML
- Dr Rao teaches AIML
- AIML is offered by CSE

ONTOLOGY — DEFINES MEANING, CATEGORIES & RULES

- Student is a subclass of Person
- "studies" has domain Student
- "studies" has range Course

DEFINITION — ONTOLOGY

An **ontology** is a formal, explicit specification of the vocabulary of a domain: its classes, properties (relations), the hierarchy among them, and the logical constraints/rules that give them meaning. Standards: RDFS and OWL.

Graph = facts | Ontology = vocabulary + semantics

Classes, instances and properties

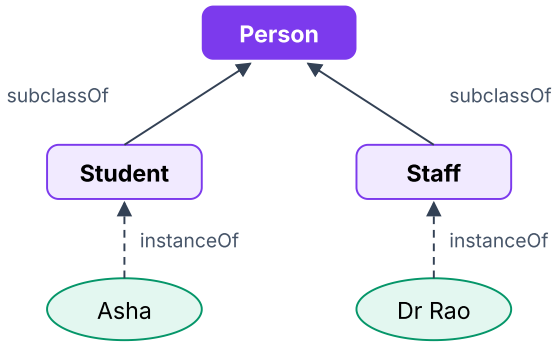


Fig 7.2 Ontology hierarchy (classes on top, instances at bottom)

Class hierarchy: Student ⊑ Person, Staff ⊑ Person (⊑ = "is a subclass of").

Instances: Asha ∈ Student, Dr Rao ∈ Staff.

Property examples: studies(Asha, AIML) — an object property linking two entities; credits(AIML, 4) — a data property linking an entity to a literal.

Semantic relationships an ontology can express: subclass (is-a), instance-of, part-of, domain, range, inverse, symmetric/transitive properties, disjoint classes.

7.6 Ontology-based reasoning (rule-based inference) ★ Very likely

Reasoning = known facts + semantic rules → additional knowledge. The ontology lets a **reasoner** derive facts that were never explicitly stored (they are *entailed*).

EXAMPLE 1 — SUBCLASS INFERENCE

Given 1: $\text{Student} \sqsubseteq \text{Person}$ (Student is a subclass of Person). Given 2: $\text{Student}(\text{Asha})$. ⇒ **Inferred fact: $\mathbf{Person(Asha)}$**. In FOL this is just $\forall x\ \text{Student}(x) \Rightarrow \text{Person}(x)$ + Modus Ponens — rule-based inference.

EXAMPLE 2 — DOMAIN, RANGE AND INVERSE: ONE TRIPLE, THREE INFERENCES

Stored fact: $\text{studies}(\text{Asha}, \text{AIML})$

Ontology rule	Inference
domain(studies) = Student	Student(Asha)
range(studies) = Course	Course(AIML)
inverse(studies) = hasStudent	hasStudent(AIML, Asha)

Important: in RDFS/OWL, domain and range are inference rules — not merely form-validation checks.

7.7 Open-world assumption: missing ≠ false

WHAT IS STORED?

"Asha studies AIML." Nothing is stored about whether Ravi studies AIML.

WHAT CAN WE CONCLUDE?

~~"Ravi does not study AIML."~~ ✘

Correct: **"We do not know from the current graph."** ✔

Open world (OWA): the graph may be incomplete, so the absence of a fact does not prove its opposite (used by RDF/OWL). Contrast with the **closed-world assumption** (databases, Prolog): anything not stored is false.

7.8 Question answering by following graph paths

Multi-step questions become **graph pattern matching**. "Who teaches a course studied by Asha?"

1. Match **Asha —studies→ ?course** → result: AIML.
2. Match **?teacher —teaches→ AIML** → result: Dr Rao.
3. Answer: **Dr Rao**, supported by the two matching relationships (the shared course connects the patterns).

SPARQL: asking the graph a question

```
PREFIX ex: <https://example.org/campus/>
SELECT ?teacher WHERE {
   ex:Asha   ex:studies  ?course .
   ?teacher  ex:teaches  ?course .
}
```

SPARQL is the query language for RDF. Variables begin with **?**. `SELECT ?teacher` returns the value bound to ?teacher. ?course appears in both patterns, so it **joins** the two facts (like a shared variable in FOL unification). Result here: Dr Rao.

7.9 Where are knowledge graphs used?

Area	Use
Search & QA	Connect people, places, topics and facts to improve answers (e.g. search-engine info panels)
Recommendation	Link users, items, categories and preferences
Healthcare	Connect diseases, symptoms, drugs and biomedical concepts
Enterprise	Connect employees, products, documents and business processes
Education	Connect students, courses, concepts, outcomes and resources
AI agents	Retrieve structured evidence before generating an answer (grounding, fewer hallucinations)

Wikidata — a real public knowledge graph. Entities use **Q-identifiers**; properties use **P-identifiers** (e.g. **P31 = "instance of"**, **P279 = "subclass of"**). SPARQL can follow relationships and class paths. Display names are for people; stable identifiers are for reliable machine linking.

7.10 How an intelligent agent uses a knowledge graph

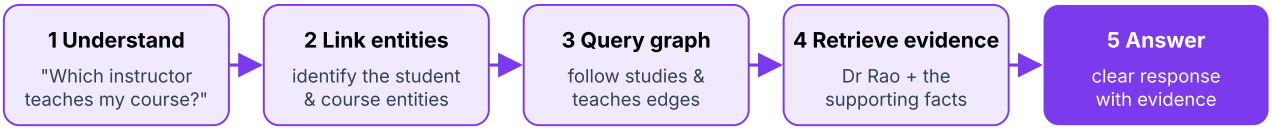


Fig 7.3 KG-powered question answering pipeline. If an entity name is ambiguous, the agent should ask for clarification rather than guess.

CLASS Q&A (EXIT TICKET) — ANSWER THESE

1. In (Asha, studies, AIML), identify subject, predicate and object. → Subject Asha, predicate studies, object AIML.
2. Difference between a knowledge graph and an ontology? → KG stores particular facts; ontology defines classes, properties, meaning and rules.
3. If Student ⊑ Person and Student(Asha), what can we infer? → Person(Asha).
4. Why does a missing triple not automatically mean "false"? → Open-world assumption: the graph may be incomplete.
5. In the SPARQL example, why is ?course used in both triple patterns? → The shared variable joins the two facts.
6. Explain "knowledge graph" in one sentence using the words entity and relationship. → A knowledge graph is a network of entities connected by labelled relationships.

ONE-SLIDE RECAP (5 LINES)

1 **Knowledge graph** connects entities using labelled relationships. 2 **RDF triple** represents one fact as subject–predicate–object. 3 **Ontology** defines classes, properties, meaning and logical constraints. 4 **Reasoning** derives implicit facts from explicit facts + ontology rules. 5 **SPARQL / agent** retrieves connected facts to answer questions. *Connected facts become more useful when their meaning is explicit.*

8 Reasoning under Uncertainty

Logic says TRUE or FALSE. The real world says "probably". This chapter teaches how an agent reasons with **degrees of belief** using probability.

8.1 What is uncertainty and why does AI need to handle it? ★ Very likely

With certain knowledge we can write a rule A → B. But if we are **not sure whether A is true**, we cannot use such a rule — this situation is called **uncertainty**. In classical logic "Ravi is a student" is simply $\text{Student}(\text{Ravi}) = \text{True}$; but for "Will it rain tomorrow?" we can't say True or False — instead $P(\text{RainTomorrow}) = 0.8$: the agent believes there is an 80% chance of rain.

DEFINITION

Uncertainty is an agent's inability to reach a definite decision about the truth of a proposition. **Probabilistic reasoning** is a way of knowledge representation where we apply probability to indicate the uncertainty in knowledge — combining probability theory with logic. Probabilities summarize the agent's beliefs relative to the available evidence.

WHY UNCERTAINTY OCCURS

- **Ignorance / laziness** — too many rules or facts to list; incomplete theory
- **Partial observability** — the agent can't see everything
- **Noisy / unreliable** sensors, experimental errors, unreliable sources
- **Nondeterminism** — outcomes of actions are unpredictable
- **Complexity**; conditions (temperature, environment) vary

NEED FOR UNCERTAINTY HANDLING

- Reason when outcomes are unpredictable
- Combine prior knowledge with newly observed evidence
- Make useful decisions even with missing information
- Avoid forcing every situation into TRUE/FALSE
- Examples: medical diagnosis, weather prediction, robot localization, spam filtering, matches between teams

EXAMPLE — AUTONOMOUS CAR

Camera: possibly a pedestrian · Radar: object detected · GPS: approximate position · Weather & road: uncertain. The car must still decide: Camera detects object → is it a pedestrian? → maybe 90% → should it brake? → decision. A probabilistic system reasons with $P(\text{Pedestrian} \mid \text{Evidence}) = 0.90$ and uses that to choose an action. Two ways to solve such problems: **Bayes' rule** and **Bayesian statistics**.

8.2 Basic probability concepts

$$P(A) = \frac{\text{number of desired outcomes}}{\text{total number of outcomes}}$$

$0 \le P(A) \le 1 \cdot P(\neg A) + P(A) = 1$

- **Event**: each possible outcome of a variable.
- **Sample space**: the collection of all possible events.
- **Random variables**: represent events and objects in the real world.
- **Prior probability**: probability computed *before* observing new information.
- **Posterior probability**: probability computed *after* all evidence is taken into account (prior + new information).
- **Sum rule**: $P(A \vee B) = P(A) + P(B) - P(A \wedge B)$.

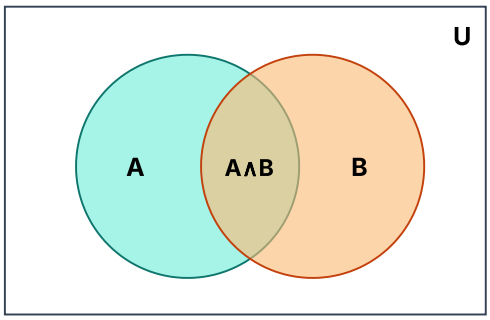


Fig 8.1 P(A|B): we zoom into circle B and ask what fraction of it is also A.

8.3 Conditional probability ★ Very likely

DEFINITION

Conditional probability P(A|B) is the probability of event A given that event B has already occurred ("probability of A under the condition B").

$$P(A \mid B) = \frac{P(A \wedge B)}{P(B)} \quad (P(B) > 0)$$

$P(A \wedge B)$ = joint probability of A and B; $P(B)$ = marginal probability of B

EXAMPLE 1 — SUSAN'S TESTS

P(passing both tests) = 0.6; P(passing the first) = 0.8. P(second | first)?

$$P(\text{second} \mid \text{first}) = \frac{P(\text{first} \wedge \text{second})}{P(\text{first})} = \frac{0.6}{0.8} = \boxed{0.75}$$

EXAMPLE 2 — ENGLISH & MATHS

70% of students like English, 40% like English *and* Maths. What % of English-likers also like Maths? A = likes Maths, B = likes English.

$$P(A|B) = \frac{P(A \wedge B)}{P(B)} = \frac{0.4}{0.7} = \boxed{57\%}$$

Why it matters in AI: an agent observes evidence (Fever = Yes, Cough = Yes) and wants $P(\text{Disease} \mid \text{Fever}, \text{Cough})$ — evidence changes the probability of the hypothesis. Fundamental to medical diagnosis, spam detection, fault diagnosis, speech recognition, recommendation, fraud detection and intrusion detection.

Product rule

$$P(A \wedge B) = P(A \mid B)\,P(B) = P(B \mid A)\,P(A)$$

Both describe the same joint event $A \wedge B$ — this leads directly to Bayes' theorem.

8.4 Bayes' theorem ★ Very likely

Also called **Bayes' rule, Bayes' law** or **Bayesian reasoning**; named after British mathematician **Thomas Bayes**. It relates the conditional and marginal probabilities of two events, i.e. it lets us compute P(B|A) when we know P(A|B) — it "reverses" a conditional probability. It is the basis of Bayesian inference and of most modern AI systems for probabilistic inference.

WHY DO WE NEED IT?

Doctors know $P(\text{Symptom} \mid \text{Disease})$ (how often the disease causes the symptom) — but diagnosis needs $P(\text{Disease} \mid \text{Symptom})$. These are **not** the same. Bayes' theorem converts one into the other.

Derivation (write this for 3 marks)

Product rule: $P(A \wedge B) = P(A \mid B)\,P(B)$ … (1) and $P(A \wedge B) = P(B \mid A)\,P(A)$ … (2)
Equating the right-hand sides of (1) and (2): $P(A \mid B)\,P(B) = P(B \mid A)\,P(A)$
Dividing by $P(B)$: **$P(A \mid B) = P(B \mid A)\,P(A) / P(B)$** … Bayes' theorem

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

Posterior = **Likelihood** × **Prior** / **Evidence**

Term	Name	Meaning (email example: A = Spam, B = contains "offer")
$P(A\|B)$	**Posterior**	Probability of hypothesis A after observing evidence B — what we want
$P(B\|A)$	**Likelihood**	Probability of the evidence assuming the hypothesis is true (spam emails containing "offer")
$P(A)$	**Prior**	Probability of the hypothesis before seeing evidence (overall fraction of spam)
$P(B)$	**Evidence / marginal**	Pure probability of the evidence (any email containing "offer")

General form with the theorem of total probability

$$P(B) = \Sigma_i\, P(B \mid A_i)\, P(A_i) \quad \Rightarrow \quad P(A_i \mid B) = \frac{P(B \mid A_i)\, P(A_i)}{\Sigma_k\, P(B \mid A_k)\, P(A_k)}$$

$A_1, \ldots, A_n$ mutually exclusive and exhaustive ($\Sigma P(A_i) = 1$)

Mitchell's (ML textbook) notation

$$P(h \mid D) = \frac{P(D \mid h)\, P(h)}{P(D)}$$

h = hypothesis, D = observed training data. P(h): prior · P(D): marginal (prior probability of seeing D) · P(D|h): likelihood · P(h|D): posterior.

P(h|D) increases with P(h) and with P(D|h); P(h|D) **decreases as P(D) increases** — the more probable D is independent of h, the less evidence D gives in support of h.

Solved Bayes problems ★ Very likely

1 · MENINGITIS & STIFF NECK

M = meningitis, S = stiff neck. P(S|M) = 0.7, P(M) = 1/50000, P(S) = 0.01.

$$P(M|S) = \frac{P(S|M)\,P(M)}{P(S)} = \frac{0.7 \times (1/50000)}{0.01} = \frac{0.000014}{0.01} = \boxed{0.0014}\ (1 \text{ in } \sim 714)$$

Lesson: even though meningitis causes a stiff neck 70% of the time, a stiff-neck patient has a tiny chance of meningitis because the disease is extremely rare. A strong symptom does not mean a high probability of disease — the prior matters.

2 · SPAM AND THE WORD "OFFER"

P(S) = 0.20, P(W|S) = 0.80, P(W) = 0.40.

$$P(S|W) = \frac{0.80 \times 0.20}{0.40} = \boxed{0.40} \rightarrow$$

$$P(\text{Spam}|\text{Offer}) = 40\%$$

3 · FACE CARD → KING

One card from 52. Given it's a face card (J, Q, K → 12 cards), P(King)?

$$P(K|F) = \frac{P(F|K)\,P(K)}{P(F)} = \frac{1 \times 4/52}{12/52} =$$

$$\boxed{1/3 \approx 33.33\%}$$

(Every King is a face card so P(F|K) = 1; out of every 3 face cards 1 is a King.)

4 · EXERCISE FROM CLASS — LIVER DISEASE & ALCOHOLISM

A = patient has liver disease, B = patient is alcoholic. 10% of patients have liver disease, 5% are alcoholics, and among liver-disease patients 7% are alcoholics. P(A|B)?

$$P(A) = 0.10,\ P(B) = 0.05,\ P(B|A) = 0.07 \rightarrow P(A|B) = \frac{0.07 \times 0.10}{0.05} = \boxed{0.14}\ (14\%)$$

Applications of Bayes' theorem in AI: medical diagnosis (update disease probability from symptoms/tests), robot localization (update belief about position from sensors), weather prediction, fault diagnosis (most likely cause of a failure), classification (P(class | features)).

8.5 MAP — Maximum A Posteriori hypothesis ★ Very likely

A **hypothesis** h is a possible explanation/assumption about something we want to predict. Given a set of candidate hypotheses H = {h_1, h_2, ...} and observed data D, we want the hypothesis that is **most likely to be true given D**.

$$h_{MAP} \equiv \text{argmax}_{h \in H}\ P(h|D) = \text{argmax}_{h \in H}\ \frac{P(D|h)\,P(h)}{P(D)} = \text{argmax}_{h \in H}\ P(D|h)\,P(h)$$

P(D) is the same for every hypothesis, so it can be dropped. (If all priors are equal, this reduces to the maximum-likelihood hypothesis h_{ML} = argmax P(D|h).)

BOX OF BALLS (CLASS EXAMPLE)

Hypotheses: h_1 = box with mostly red balls, h_2 = mostly blue, h_3 = equal red and blue. Observed D = {red, red, red}. Posteriors computed with Bayes: $P(h_1|D) = 0.70$ (most likely), $P(h_2|D) = 0.10$, $P(h_3|D) = 0.20$ → $\mathbf{h_{MAP} = h_1}$, the most probable explanation of the data.

THE CANCER TEST PROBLEM (MITCHELL) — MOST ASKED

Two hypotheses: patient has cancer (*cancer*) or not (¬*cancer*). The lab test returns a correct positive in only **98%** of cases where the disease is present, and a correct negative in only **97%** of cases where it is not. **0.008** of the population has this cancer. A new patient tests **positive**. Diagnose using MAP.

Given	Complement
P(cancer) = 0.008	P(¬cancer) = 1 − 0.008 = 0.992
P(+ | cancer) = 0.98	P(− | cancer) = 0.02
P(− | ¬cancer) = 0.97	P(+ | ¬cancer) = 0.03

$P(+ \mid \text{cancer})\, P(\text{cancer}) = 0.98 \times 0.008 = \mathbf{0.0078}$

$P(+ \mid \neg\text{cancer})\, P(\neg\text{cancer}) = 0.03 \times 0.992 = \mathbf{0.0298}$

$0.0298 > 0.0078 \Rightarrow$ **$h_{MAP} = \neg$cancer** — the patient with a positive test is diagnosed as **not** having cancer.

Exact posterior (normalize): $P(\text{cancer} \mid +) = 0.0078 / (0.0078 + 0.0298) = \mathbf{0.21}$; $P(\neg\text{cancer} \mid +) = 0.79$.

Negative test result (second slide problem):

$P(- \mid \text{cancer})\, P(\text{cancer}) = 0.02 \times 0.008 = 0.00016 \quad P(- \mid \neg\text{cancer})\, P(\neg\text{cancer}) = 0.97 \times 0.992 = 0.96224 \Rightarrow h_{MAP} = \neg\text{cancer}$

$P(-) = 0.00016 + 0.96224 = 0.9624 \quad \rightarrow P(\text{cancer} \mid -) = 0.00016 / 0.9624 = 0.000166 \approx \mathbf{0.0166\%}$, $\quad P(\neg\text{cancer} \mid -) = 0.96224 / 0.9624 \approx \mathbf{99.98\%}$ → not having cancer.

Why is a positive test still "no cancer"? The disease is so rare (prior 0.008) that false positives from the huge healthy population (3% of 99.2%) outnumber true positives.

EXAM TIP — BAYES/MAP NUMERICALS

(1) Write all given probabilities **and their complements** in a table. (2) Write the formula. (3) Compute $P(D|h)P(h)$ for each hypothesis. (4) Pick the largest = MAP. (5) Normalize if asked for the actual probability (divide each by their sum; they must add to 1).

8.6 Bayesian networks ★ Very likely

DEFINITION

A **Bayesian network** (Bayes network, belief network, decision network, Bayesian model) is a probabilistic graphical model that represents a set of random variables and their conditional dependencies using a Directed Acyclic Graph (DAG), with a conditional probability distribution attached to each node.

- Each **node** = a random variable (continuous or discrete).

- Each **arc (directed arrow)** = a direct causal relationship / probabilistic dependence. If X → Y, X is a **parent** of Y. No arrow between two nodes means they are (conditionally) independent.
- The graph has **no directed cycles**.
- Each node X_i has a **CPT (conditional probability table)** giving $P(X_i \mid Parents(X_i))$; nodes without parents just have a prior.
- Two components: **causal component** (structure: the DAG) + **actual numbers** (the CPTs). Bayesian network = structure + probability values. The generalized form for decision problems is an **influence diagram**.
- Uses: prediction, anomaly detection, diagnostics, automated insight, reasoning, time-series prediction, decision making under uncertainty.

Why not just use one big probability table?

The full joint distribution of n Boolean variables needs 2^n entries: 2^{10} = 1024, 2^{20} = 1,048,576, 2^{100} — enormous. A Bayesian network exploits **independence and conditional independence** to store far fewer numbers.

Joint probability: chain rule vs Bayesian network

Chain rule (always true): $P(x_1, \ldots, x_n) = P(x_1 \mid x_2, \ldots, x_n)\, P(x_2 \mid x_3, \ldots, x_n) \ldots P(x_n)$
e.g. $P(R, S, W) = P(R \mid S, W)\, P(S \mid W)\, P(W)$ — hard to use: each variable depends on all others.

Bayesian network: $P(x_1, \ldots, x_n) = \prod_{i=1..n} P(x_i \mid Parents(X_i))$
Each variable depends only on its parents. Rain → Sprinkler, both → WetGrass: $P(R, S, W) = P(R) \cdot P(S|R) \cdot P(W|S,R)$.

Conditional independence

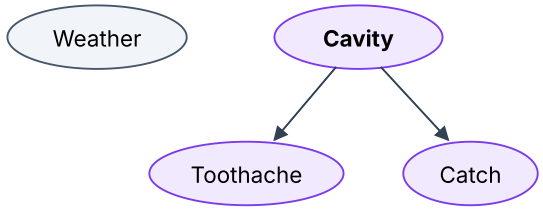


Fig 8.2 Weather is independent of the rest; Toothache and Catch are conditionally independent given Cavity.

Once we know whether there is a cavity, toothache tells us nothing more about whether the probe catches. So

$$P(Toothache, Catch, Cavity) = P(Toothache \mid Cavity) \cdot P(Catch \mid Cavity) \cdot P(Cavity)$$

instead of one large table. This is the key idea behind Bayesian networks (and Naive Bayes).

CPT example and the simplest joint computation

Cloudy → Rain: P(Rain | Cloudy) = 0.8, P(Rain | ¬Cloudy) = 0.2.

Cloudy	P(Rain)
True	0.8
False	0.2

This is a **Conditional Probability Table**. Each row must sum to 1 with its complement column (P(¬Rain|Cloudy) = 0.2, etc.).

CLOUDY → RAIN → WETGRASS

P(Cloudy) = 0.5, P(Rain|Cloudy) = 0.8, P(WetGrass|Rain) = 0.9.

$$P(C, R, W) = P(C)\, P(R|C)\, P(W|R) = 0.5 \times 0.8 \times 0.9 = \boxed{0.36}$$

Example — A, B, C, D network (class)

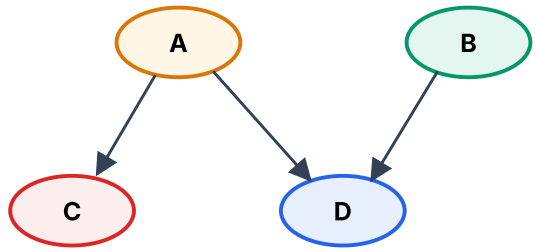
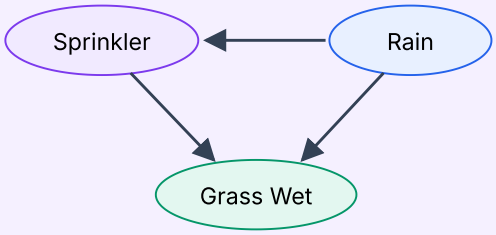


Fig 8.3 A, B: no parents (unconditional, independent, evidence/parent nodes). C, D: conditional, dependent, hypothesis/child nodes.

Given	Value	Given	Value
P(A)	0.3	P(D | A, B)	0.7
P(B)	0.6	P(D | A, ¬B)	0.4
P(C | A)	0.4	P(D | ¬A, B)	0.2
P(C | ¬A)	0.2	P(D | ¬A, ¬B)	0.01

$P(A, B, C, D) = P(D \mid A, B) \times P(C \mid A) \times P(B) \times P(A)$
e.g. all true: $0.7 \times 0.4 \times 0.6 \times 0.3 = \mathbf{0.0504}$
$P(\neg A, B, \neg C, D) = 0.2 \times 0.8 \times 0.6 \times 0.7 = \mathbf{0.0672}$

PROBLEM — RAIN, SPRINKLER, GRASS WET: FIND P(W) ★



R	P(R)	R	P(S|R)
T	0.2	F	0.4
F	0.8	T	0.01

S	R	P(W | S, R)
¬S	¬R	0.0
¬S	R	0.8
S	¬R	0.9
S	R	0.99

Step 1 complements: $P(\neg R) = 0.8$; $P(\neg S|R) = 0.99$; $P(\neg S|\neg R) = 0.60$.

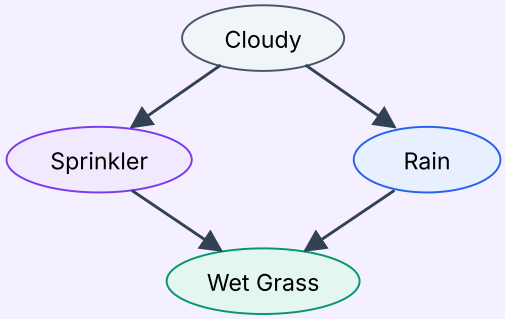
Step 2 law of total probability: $P(W) = \Sigma_S \Sigma_R P(W \mid S, R)\, P(S \mid R)\, P(R)$

Step 3 $P(W|S,R)P(S|R)P(R) = 0.99 \times 0.01 \times 0.2 = 0.00198$ | $P(W|S,\neg R)P(S|\neg R)P(\neg R) = 0.90 \times 0.4 \times 0.8 = 0.28800$

$P(W|\neg S,R)P(\neg S|R)P(R) = 0.80 \times 0.99 \times 0.2 = 0.15840$ | $P(W|\neg S,\neg R)P(\neg S|\neg R)P(\neg R) = 0.00 \times 0.60 \times 0.8 = 0$

Step 4 $P(W) = 0.00198 + 0.28800 + 0.15840 + 0 =$ **0.44838 ≈ 0.448**

PROBLEM 3 (25 PTS) — CLOUDY, SPRINKLER, RAIN, WET GRASS: P(W) AND P(S|W)



P(C) = 0.5. P(S|C): t 0.10, f 0.50. P(R|C): t 0.80, f 0.20. P(W|S,R): tt 0.99, tf 0.90, ft 0.90, ff 0.00.
(Hint: requires summing out over a few variables.)

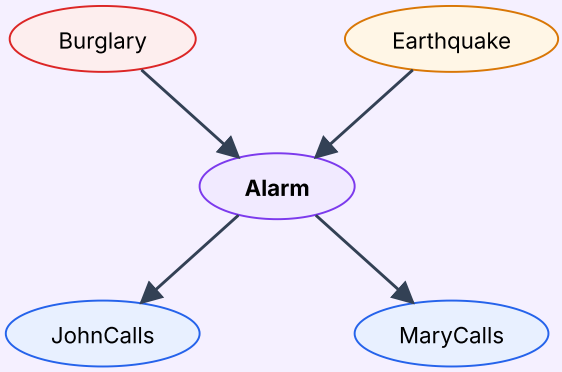
P(W) = $\Sigma_{c,s,r}$ P(W|s,r) P(s|c) P(r|c) P(c)
C = T: 0.99·0.10·0.80 + 0.90·0.10·0.20 + 0.90·0.90·0.80 + 0·0.90·0.20 = 0.0792 + 0.0180 + 0.6480 + 0 = 0.7452 → × 0.5 = 0.3726
C = F: 0.99·0.50·0.20 + 0.90·0.50·0.80 + 0.90·0.50·0.20 + 0 = 0.0990 + 0.3600 + 0.0900 = 0.5490 → × 0.5 = 0.2745
P(W) = 0.3726 + 0.2745 = **0.6471**

P(S|W) = P(S, W) / P(W). First P(S) = 0.10×0.5 + 0.50×0.5 = 0.30.
P(S, W) = Σ_c P(c) P(S|c) Σ_r P(r|c) P(W|S,r) = 0.5·0.10·(0.80·0.99 + 0.20·0.90) + 0.5·0.50·(0.20·0.99 + 0.80·0.90)
= 0.05 × 0.972 + 0.25 × 0.918 = 0.0486 + 0.2295 = 0.2781 ⇒ P(S|W) = 0.2781 / 0.6471 = **0.4298 ≈ 0.43**

NOTE ABOUT THE CLASS SLIDE

The slide computes P(W|S) = Σ P(W|S,r) P(r|c) P(c) = 0.945 and gets P(S|W) = 0.945 × 0.30 / 0.6471 ≈ **0.438**. That shortcut uses P(c) where it should use P(c|S) (knowing the sprinkler is on makes "cloudy" less likely: P(C|S) = 0.05/0.30 = 1/6). The exact answer is **0.430**. In the exam, show the method; if your teacher expects 0.438, mention both.

THE BURGLARY NETWORK (TEXTBOOK FIG 14.2)



B	E	P(A|B,E)
t	t	0.95
t	f	0.94
f	t	0.29
f	f	0.001

P(B) = 0.001, P(E) = 0.002.
P(J|A) = 0.90, P(J|¬A) = 0.05.
P(M|A) = 0.70, P(M|¬A) = 0.01.

Meaning: a burglary or an earthquake can set off the alarm; if the alarm sounds, John and/or Mary may call. JohnCalls and MaryCalls are conditionally independent given Alarm.

"Alarm sounded, both called, but no burglary and no earthquake":
P(j, m, a, ¬b, ¬e) = P(j|a) P(m|a) P(a|¬b,¬e) P(¬b) P(¬e) = 0.90 × 0.70 × 0.001 × 0.999 × 0.998 = **0.000628**

EXAM TIP — BAYESIAN NETWORK NUMERICALS

(1) Draw the DAG. (2) Write the factorization P = ∏ P(node | parents) for exactly the variables asked. (3) Fill each factor from the CPT, using 1 − p for negations. (4) For a marginal like P(W), sum the joint over every combination of the hidden variables — make a small table so you don't miss a row.

8.7 Naive Bayes classifier (Mitchell's formulation) ★ Very likely

One highly practical Bayesian learning method. Each instance x is described by a conjunction of attribute values ⟨a_1, a_2, …, a_n⟩, and the target function f(x) takes values from a finite set V. Given training

examples and a new instance, predict its target value (classification).

$$v_{MAP} = \text{argmax}_{v_j \in V} P(v_j \mid a_1, \ldots, a_n) = \text{argmax}_{v_j} \frac{P(a_1, \ldots, a_n \mid v_j)\, P(v_j)}{P(a_1, \ldots, a_n)} = \text{argmax}_{v_j} P(a_1, \ldots, a_n \mid v_j)\, P(v_j)$$

Naive Bayes assumption (attributes conditionally independent given the class): $P(a_1, \ldots, a_n \mid v_j) = \prod_i P(a_i \mid v_j)$

$$v_{NB} = \text{argmax}_{v_j \in V} P(v_j) \prod_i P(a_i \mid v_j)$$

PLAYTENNIS — CLASSIFY ⟨OUTLOOK = SUNNY, TEMPERATURE = COOL, HUMIDITY = HIGH, WIND = STRONG⟩ ★ VERY LIKELY

Day	Outlook	Temp	Humidity	Wind	Play
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Outlook	Yes	No	Temp	Yes	No
sunny	2/9	3/5	hot	2/9	2/5
overcast	4/9	0	mild	4/9	2/5
rain	3/9	2/5	cool	3/9	1/5
Humidity	**Yes**	**No**	**Wind**	**Yes**	**No**
high	3/9	4/5	strong	3/9	3/5
normal	6/9	1/5	weak	6/9	2/5

Priors: P(yes) = 9/14 = 0.64, P(no) = 5/14 = 0.36.

v_{NB}(yes) = P(yes) P(sunny|yes) P(cool|yes) P(high|yes) P(strong|yes) = (9/14)(2/9)(3/9)(3/9)(3/9) = **0.0053**

v_{NB}(no) = P(no) P(sunny|no) P(cool|no) P(high|no) P(strong|no) = (5/14)(3/5)(1/5)(4/5)(3/5) = **0.0206**

0.0206 > 0.0053 ⇒ **PlayTennis = No**

Normalize: P(no | x) = 0.0206 / (0.0206 + 0.0053) = **0.795**; P(yes | x) = **0.205**; check 0.205 + 0.795 = 1 ✔

STOLEN VEHICLES — CLASSIFY (RED, SUV, DOMESTIC)

#	Color	Type	Origin	Stolen?
1	Red	Sports	Domestic	Yes
2	Red	Sports	Domestic	No
3	Red	Sports	Domestic	Yes
4	Yellow	Sports	Domestic	No
5	Yellow	Sports	Imported	Yes
6	Yellow	SUV	Imported	No
7	Yellow	SUV	Imported	Yes
8	Yellow	SUV	Domestic	No
9	Red	SUV	Imported	No
10	Red	Sports	Imported	Yes

P(Yes) = 5/10 = 0.5, P(No) = 0.5.

	Yes	No
Red	3/5	2/5
SUV	1/5	3/5
Domestic	2/5	3/5

$P(\text{Yes} \mid x) \propto (5/10)(3/5)(1/5)(2/5) = 3/125 = \mathbf{0.024}$ $P(\text{No} \mid x) \propto (5/10)(2/5)(3/5)(3/5) = 9/125 = \mathbf{0.072}$

$0.072 > 0.024 \Rightarrow$ **Not stolen**

More Naive Bayes (email example with Laplace smoothing, types of NB, zero-frequency, Unit III view) is in **Chapter 12**. A related 10-mark question from the slides: estimate conditional probabilities of {colour, legs, height, smelly} for species {M, H} and classify (Colour = Green, Legs = 2, Height = Tall, Smelly = No) — solved in Chapter 12.

LIKELY EXAM QUESTIONS — CH 8

1. What is uncertainty? Why is uncertainty handling needed in AI? Give reasons and examples. (5M)
2. Define conditional probability; solve a small problem (Susan / English–Maths). (4M)
3. State and derive Bayes' theorem; explain prior, likelihood, posterior, evidence. (6M) ★
4. Solve: meningitis / cancer test using MAP. (6–10M) ★
5. What is a Bayesian network? Explain its components, CPT and joint probability with an example. (8–10M) ★
6. Given a Bayesian network, compute P(W) / a joint probability. (10M)
7. Explain the Naive Bayes classifier and classify the PlayTennis / stolen-car instance. (10M) ★

60-SECOND RECAP

$P(A|B) = P(A \wedge B)/P(B)$. Bayes: posterior = likelihood × prior / evidence. $h_{MAP} = \text{argmax}\ P(D|h)P(h)$. Cancer: 0.0078 vs 0.0298 → ¬cancer. Bayesian network = DAG + CPTs; $P(x_{1..n}) = \prod P(x_i|\text{parents})$. Full joint needs 2^n numbers; BN uses conditional independence. Naive Bayes: $v_{NB} = \text{argmax}\ P(v)\prod P(a_i|v)$; PlayTennis → No (0.795).

9 Introduction to Machine Learning

Text books: Tom M. Mitchell, *Machine Learning*; Peter Harrington, *Machine Learning in Action* (Ch 1.1–1.3). Topics: what is ML, key terminology, key tasks, well-posed learning problems, types of ML — plus concept learning (Find-S, Candidate-Elimination) taught in class.

9.1 What is machine learning? ★ Very likely

THREE DEFINITIONS — LEARN ALL THREE, WRITE TWO

1. **Arthur Samuel (1959):** "Machine Learning is the field of study that gives computers the capability to learn without being explicitly programmed."
2. **Tom M. Mitchell (1997):** "A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E."
3. **Svensson & Söderberg (2008):** "ML is concerned with the design and development of algorithms and techniques that allow computers to 'learn'. The major focus of ML research is to extract information from data automatically, by computational and statistical methods. It is thus closely related to data mining and statistics."

In short: **learning = improving with experience at some task** — improve over task T, with respect to performance measure P, based on experience E. The machine receives data as input and uses an algorithm to formulate answers; ML combines data with statistical tools to predict an output, which is then used for actionable insights. ML is an application (sub-field) of Artificial Intelligence; its core objective is **learning and inference**.

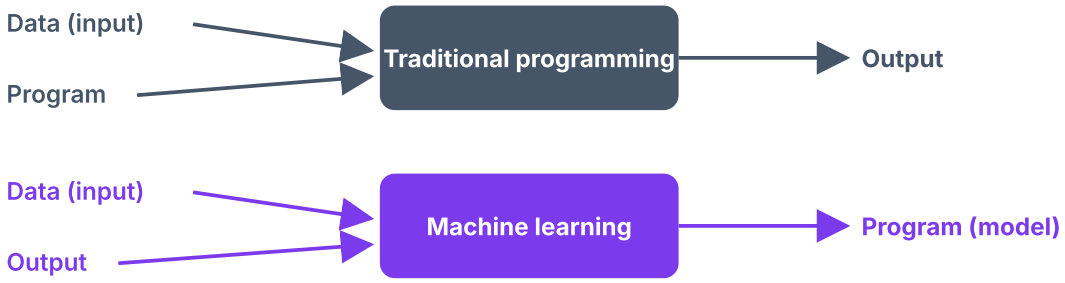


Fig 9.1 Traditional programming: data + program → output. Machine learning: data + output (examples) → program (the learned model).

Why machine learning? / Why is it important?

- Recent progress in algorithms and theory; growing flood of online data; huge memory & compute power; industry demand; less or no human intervention; better decision making and predictions.
- Computers capture terabytes of data daily; its size and complexity mean humans can't extract useful information. We can reduce dimensions, or let computers (which don't get bored and don't find high dimensions hard) do the work. A 2-D projection of a 3-D object hides information; glyphs (size/colour) help only up to a few dimensions, not 100.
- Some tasks can be defined only by examples (recognizing people).
- Relationships and correlations hidden in large data can be found (data mining).
- Human-designed machines often don't work as well as desired in their environment.
- Knowledge for some tasks is too large to encode by hand (medical diagnosis).
- Environments change over time; new knowledge keeps being discovered — redesigning by hand is hard.

When to learn?

When **human expertise does not exist** (navigating on Mars); when **humans can't explain their expertise** (speech recognition); when **solutions change over time** (routing in networks); when solutions must be adapted to particular

cases (user biometrics).

Learning — key ideas from the slides

- Machines learn from **data** ("experience"). Early AI = **symbolic processing** (computer manipulates symbols reflecting the environment); ML methods are sometimes called **subsymbolic** (no symbolic manipulation).
- ML is about making computers **modify or adapt their actions** (predictions or controlling a robot) so that they get more accurate, accuracy = how well the actions reflect the correct ones.
- **Scrabble example:** you beat the computer at first, but after many games it beats you every time and can use the same strategies against other players → generalisation.
- ML merges ideas from neuroscience and biology, statistics, mathematics and physics; data mining shifted emphasis back to efficient algorithms.
- **Computational complexity** has two parts: complexity of **training** (rare, not time-critical) and of **applying** the trained model (must be fast — many test points).
- **What to learn? Direct learning** (discriminative / model-free): learn a function mapping input → output. **Indirect learning** (generative / model learning): learn a model of the domain, then use it to answer questions.

Areas of influence (ML is interdisciplinary)

Field	Contribution / question
Statistics	How best to use samples drawn from unknown probability distributions to decide from which distribution a new sample is drawn?
Brain models	Non-linear elements with weighted inputs (artificial neural networks) as simple models of biological neurons
Adaptive control theory	Controlling a process with unknown parameters that must be estimated during operation
Psychology	Modelling human performance on learning tasks
Artificial intelligence	Algorithms that acquire knowledge humans acquire, at least as well as humans
Evolutionary models	Modelling biological evolution to improve program performance
Information theory, philosophy	Measures of information; Occam's razor, induction

9.2 Applications

Domain	Examples
Everyday software	Spam filters, voice recognition, computer games, automatic number-plate recognition (petrol stations, toll roads), vehicle stability systems, bank loan decisions
Healthcare	Diagnostics; breast cancer / diabetes / Covid-19 prediction; tumours on mammograms; radiology & radiotherapy; assistive medical tech (brain tumours, fetal imaging, cardiac analysis), personalized treatment, drug discovery, clinical research
Agriculture	Crop management, field-conditions management
Education	Personalized learning
Transportation	Traffic regulation, automated driving, vehicle quality assurance; traffic prediction (Google/Bing Maps: average times + real-time vehicle locations)
Manufacturing	Smart maintenance, better product development, quality improvement, market adaptation
Others	Virtual personal assistants (Google Assistant, Alexa, Siri, Cortana); social-media personalization; online fraud detection (identity theft, fake accounts, man-in-the-middle; feed-forward NN for genuine vs fraud transactions); stock trading (LSTM to predict trends); automatic translation (sequence-to-sequence learning, CNN image recognition, OCR); credit-card fraud, fake news, sentiment analysis, duplicate-question detection, Android malware classification, food & calorie detection, telecom customer churn
Classic successes (Mitchell)	**SPHINX** — recognizing spoken words (phonemes; neural nets & HMMs adapt to speakers, vocabularies, microphones, noise); **ALVINN** — drove unassisted at 70 mph for 90 miles on public highways; **NASA sky survey** — decision trees classify celestial objects from 3 TB of Palomar images; **TD-Gammon** — learned backgammon by playing >1 million games against itself, competitive with the human world champion

9.3 Key terminology ★ Very likely

Term	Meaning	Fruit example
Model / hypothesis	A specific representation learned from data by applying an ML algorithm	The learned rule that maps features → fruit name
Feature	An individual measurable property of the data; a set of numeric features = **feature vector**, fed as input to the model. Choose informative, discriminating, independent features (feature extractor)	Colour, smell, taste, weight
Target (label)	The value to be predicted by the model	Apple, orange, banana
Training	Giving inputs (features) with expected outputs (labels) so that we get a model that maps new data to the trained categories	Show 1000 labelled fruits
Prediction	Feeding new inputs to the ready model to get a predicted output	New fruit → "apple"
Training set / test set	Data used to learn / held-out data used to evaluate on unseen examples	80% / 20% split
Instance, attribute	One example (row) / one feature (column)	One fruit / "colour"

More data > better model > higher accuracy. The right algorithm depends on the problem statement (e.g. predict future stock prices), size/quality/nature of data, and algorithm complexity. Note: ML algorithms are not types of ML.

9.4 Key tasks of machine learning ★ Very likely

Task	Type	What it does	Algorithms / examples
Classification	Supervised	Predict a discrete class (Yes/No, True/False, spam/not)	Decision trees, Naive Bayes, logistic regression, KNN, SVM, AdaBoost
Regression	Supervised	Predict a numeric value (stock price, product price from demand)	Linear, tree-based, multivariate regression
Clustering	Unsupervised	Group similar items without labels	k-means, hierarchical; search engines, optimal placement of base stations
Association	Unsupervised	Find patterns of items that occur together	Apriori, FP-Growth; market-basket analysis (Amazon/Flipkart)
Density estimation	Unsupervised	Find statistical values that describe the data	—
Dimensionality reduction	Unsupervised	Reduce number of features (visualization, compression)	PCA

SCENARIO → TECHNIQUE (WITH JUSTIFICATION) — ASKED AS SHORT QUESTIONS

- Filtering spam emails → **classification** (output is a category).
- Predicting stock market price; how much will it rain → **regression** (numeric output).
- Facebook recognizing friends in your album; optimal placement of base stations → **clustering** (group similar items).
- Amazon/Flipkart suggesting products; Netflix recommending movies from past choices → **association rule mining**.

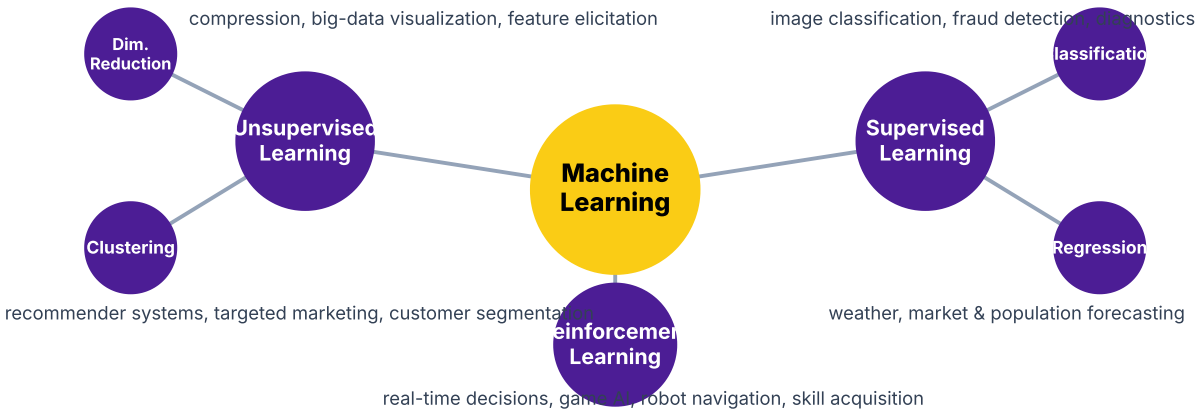


Fig 9.2 ML algorithms and where they are used.

9.5 Well-posed learning problems ★ Very likely

DEFINITION

A learning problem is **well-posed** (well-defined) when we identify three features: (1) the class of tasks T, (2) the measure of performance P to be improved, (3) the source of experience E.

Problem	Task T	Performance P	Experience E
Checkers	Playing checkers	% of games won against opponents	Playing practice games against itself
Handwriting recognition	Recognizing and classifying handwritten words within images	% of words correctly classified	A database of handwritten words with given classifications
Robot driving	Driving on public four-lane highways using vision sensors	Average distance travelled before an error (judged by human overseer)	Sequence of images and steering commands recorded while observing a human driver
Face recognition	Recognize faces	% of correct recognitions	Opportunity to make guesses and being told the truth
Finding clusters	Finding clusters in data	Compactness of groups detected	Analyses of a growing set of data
Traffic prediction	Predict traffic patterns at a busy intersection	Accuracy of future predictions	Data about past traffic patterns
Spam filtering	Classify emails as spam / not spam	% of emails correctly classified	Emails labelled by the user

CHECKERS — GAME BASICS (FROM THE SLIDES, IN CASE THEY ASK YOU TO DESCRIBE IT)

Two players, 12 discs each (black and red) on the 12 dark squares closest to them; board has 64 squares (32 dark, 32 light); black moves first, then alternate. Moves only on dark squares, so pieces move diagonally; single pieces move only forward; a non-capturing move goes one square; a capture (jump) leaps over one opponent piece diagonally; multiple jumps allowed in one turn; captures are compulsory (free choice if several). A piece reaching the farthest row is crowned king (a captured piece is placed on top); kings move diagonally forward and backward and can combine jumps in several directions. A player wins when the opponent cannot move (all pieces captured or blocked).

EXAM TIP

For "Explain well-posed learning problems", write Mitchell's definition, then the T/P/E table for at least **three** examples (checkers, handwriting, robot driving). If they give a new scenario, invent sensible T, P, E for it.

9.6 Types of machine learning ★ Very likely

How does the computer know whether it is getting better, and how does it know how to improve? The answer defines the type of learning:

Type	Idea	Examples / used for
Supervised	A training set of examples **with correct responses (targets)** is provided; the algorithm generalises to respond correctly to all possible inputs. Also called **learning from exemplars**.	Regression & classification (spam, price prediction)
Unsupervised	Correct responses are **not** provided; the algorithm identifies similarities between inputs so that inputs with something in common are categorized together. The statistical approach is called **density estimation**.	Clustering, PCA (customer segmentation)
Reinforcement	Between supervised and unsupervised: the algorithm is told **when the answer is wrong, but not how to correct it**; it must explore and try possibilities until it gets it right. Called **learning with a critic** (the monitor scores the answer but doesn't suggest improvements). Making decisions sequentially.	Game AI, robot navigation, real-time decisions
Evolutionary	Biological evolution as learning: organisms adapt to improve survival and offspring. Modelled using **fitness** — a score of how good the current solution is.	Genetic algorithms, optimization

MEMORY TRICK

Supervised = **S**tudent with a teacher (answers given). **U**nsupervised = **U**nguided explorer (finds groups). **R**einforcement = **R**eward/penalty from a critic. **E**volutionary = **E**volve the fittest.

9.7 Supervised learning in more depth

- **Classification:** inputs are divided into two or more classes; the learner produces a model that assigns unseen inputs to one or more classes (multi-label). Spam filtering: inputs = emails, classes = spam/not spam. The problem is **discrete** — each example belongs to precisely one class, and the set of classes covers the whole output space. These constraints aren't always realistic: examples may belong partially to two classes → **fuzzy classifiers**; some inputs may not belong to any trained class.
- **Regression:** outputs are **continuous** (e.g. stock prices from historical data). In statistics: fit a mathematical function (curve) passing as close as possible to all data points — **function approximation / interpolation** (working out values between known values); easy in 2-D, harder in high dimensions.

THE COIN CLASSIFIER (VENDING MACHINE)

- When a coin is pushed in, the machine measures features: **diameter, weight, shape** → a 3-element input vector (shape encoded as a number: 1 = circle, 2 = hexagon...). Could also use density, composition, or a photo.
- Trained on all New Zealand coins — what if a British coin is inserted? It will be labelled as the nearest NZ coin; really it should say "not one of my coins" → **novelty detection**.
- Which features to choose isn't easy: too many inputs make training slower, and as the number of input dimensions grows, the number of data points required increases even faster → curse of dimensionality.
- We need features that reliably separate classes, and **decision boundaries** that separate the classes.

9.8 The machine learning process ★ Likely

1. **Data collection and preparation** — collect from scratch or assemble and prepare; for a new problem merge with feature selection (collect a small dataset with all candidate features, experiment, then collect the full dataset). Supervised learning also needs **target data** (often requires

domain experts and time). Consider quantity: ML needs lots of data with little noise, but larger data = more computation; whether there is enough data is hard to predict.

2. **Feature selection** — identify the features most useful for the problem (e.g. coin recognition); requires prior knowledge; features should be cheap to collect and robust to noise.
3. **Algorithm choice** — needs knowledge of each algorithm's principles and examples of use.
4. **Parameter and model selection** — many algorithms have parameters set manually or by experimentation.
5. **Training** — use computational resources to build a model of the data that predicts outputs on new data.
6. **Evaluation** — test for accuracy on data it was **not trained on** before deployment; may compare with human experts using appropriate metrics.

CHALLENGES OF ML

- Data representation
- Needs massive stores of training data
- Labelling training data is tedious
- Machines cannot explain themselves
- Bias in the data

ADVANTAGES / LIMITATIONS

Advantages: no knowledge engineers needed (everything from data); scalable in constructing a knowledge base; adaptive to changing conditions; easy migration to new domains.
Limitations (classical ML): struggles with high-dimensional data; image/object recognition hard; features must be extracted manually (tedious with huge datasets); can't generate features on its own → **why deep learning**: works on high-dimensional data, learns the right features itself.

Frameworks: TensorFlow, Theano, Scikit-learn, Caffe, Amazon ML, Google Cloud ML Engine, Azure ML Studio, Spark MLlib.

9.9 Concept learning: Find-S & Candidate-Elimination ★ Very likely (numericals!)

CONCEPT LEARNING

Inferring a boolean-valued function (the **target concept** c: X → {0,1}) from training examples of its input and output. It can be viewed as searching through a large space of hypotheses H for the hypothesis that best fits the training examples.

The EnjoySport concept-learning task

- **Instances X:** all possible days, described by Sky (Sunny, Cloudy, Rainy), AirTemp (Warm, Cold), Humidity (Normal, High), Wind (Strong, Weak), Water (Warm, Cool), Forecast (Same, Change).
- **Target concept c:** EnjoySport: X → {0, 1}.
- **Hypotheses H:** each hypothesis is a **conjunction of constraints** on the attributes. Each constraint is a specific value (e.g. Warm), **"?"** (any value acceptable) or **"Ø"** (no value acceptable).
- **Training examples D:** positive and negative examples with c(x). **Determine** h ∈ H such that h(x) = c(x) for all x in D.
- Most general hypothesis: ⟨?, ?, ?, ?, ?, ?⟩ (every day is positive). Most specific: ⟨Ø, Ø, Ø, Ø, Ø, Ø⟩ (no day is positive).
- **Satisfies:** x satisfies h when h(x) = 1, regardless of whether x is positive or negative. **Consistent:** h is consistent with D when h(x) = c(x) for every example.
- **More-general-than-or-equal (≥g):** $h_j \geq_g h_k$ iff every instance satisfying h_k also satisfies h_j. E.g. h_2 = ⟨Sunny, ?, ?, ?, ?, ?⟩ is more general than h_1 = ⟨Sunny, ?, ?, Strong, ?, ?⟩. This partial order organizes the search.

Ex	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
1	Sunny	Warm	Normal	Strong	Warm	Same	Yes
2	Sunny	Warm	High	Strong	Warm	Same	Yes
3	Rainy	Cold	High	Strong	Warm	Change	No
4	Sunny	Warm	High	Strong	Cool	Change	Yes

Find-S algorithm — find the maximally specific hypothesis

```
1. Initialize h to the most specific hypothesis in H:  h = <Ø, Ø, Ø, Ø, Ø, Ø>
2. For each POSITIVE training instance x:
      For each attribute constraint a_i in h:
          If x's value for a_i satisfies the constraint (equal) → do nothing
          Else → replace a_i in h by the next more general constraint ('?')
3. Output hypothesis h            (negative examples are IGNORED)
```

FIND-S ON ENJOYSPORT

$h_0 = \langle Ø, Ø, Ø, Ø, Ø, Ø \rangle$

x_1 (+): $h_1 = \langle$Sunny, Warm, Normal, Strong, Warm, Same$\rangle$ (copy the first positive example)

x_2 (+): Humidity differs (Normal vs High) → ? $h_2 = \langle$Sunny, Warm, ?, Strong, Warm, Same$\rangle$

x_3 (−): ignored → $h_3 = h_2$

x_4 (+): Water (Warm vs Cool) and Forecast (Same vs Change) differ → h_4 =

⟨Sunny, Warm, ?, Strong, ?, ?⟩

FIND-S — POISONOUS SEEDS (CLASS)

#	Color	Toughness	Fungus	Appearance	Poiso
1	Green	Hard	No	Wrinkled	Ye
2	Green	Hard	Yes	Smooth	No
3	Brown	Soft	No	Wrinkled	No
4	Orange	Hard	No	Wrinkled	Ye
5	Green	Soft	Yes	Smooth	Ye
6	Green	Hard	Yes	Wrinkled	Ye
7	Orange	Hard	No	Wrinkled	Yes

$h_0 = \langle Ø, Ø, Ø, Ø \rangle$

x_1+: $h_1 = \langle$Green, Hard, No, Wrinkled$\rangle$

x_2−, x_3−: ignore → $h_3 = h_1$

x_4+ (Orange): $h_4 = \langle$?, Hard, No, Wrinkled$\rangle$

x_5+ (Green, Soft, Yes, Smooth): $h_5 = \langle$?, ?, ?, ?$\rangle$

x_6+, x_7+: no change

Final: **⟨?, ?, ?, ?⟩**

FIND-S — GOING FOR A WALK

Time	Weather	Temp	Company	Humidity	Wind	Goes
Morning	Sunny	Warm	Yes	Mild	Strong	Yes
Evening	Rainy	Cold	No	Mild	Normal	No
Morning	Sunny	Moderate	Yes	Normal	Normal	Yes
Evening	Sunny	Cold	Yes	High	Strong	Yes

$h_1 = \langle$Morning, Sunny, Warm, Yes, Mild, Strong$\rangle$ → $h_3 = \langle$Morning, Sunny, ?, Yes, ?, ?$\rangle$ → h_4 =

⟨?, Sunny, ?, Yes, ?, ?⟩

FIND-S — CAR DATASET (JAPANESE ECONOMY CAR)

Org	Manf	Color	Year	Type	Class
JP	HO	Blue	1980	ECO	+
JP	TO	Green	1970	SPO	–
JP	TO	Blue	1990	ECO	+
USA	CH	Red	1980	ECO	–
JP	HO	White	1980	ECO	+
JP	TO	Green	1980	ECO	+
JP	HO	Red	1990	ECO	–

h_1 = ⟨JP, HO, Blue, 1980, ECO⟩ → h_3 = ⟨JP, ?, Blue, ?, ECO⟩ → h_5 = h_6 = **⟨JP, ?, ?, ?, ECO⟩**. (Note: this h wrongly covers negative example 7 — Find-S can't detect that because it ignores negatives.)

PROPERTIES & SHORTCOMINGS OF FIND-S

- Guaranteed to output the **most specific** hypothesis in H consistent with the positive examples (if the correct target is in H and data has no errors).
- Can't tell whether it has learned the correct concept (no way to know if it's the only consistent one). Is preferring the most specific hypothesis a good strategy?
- Can't detect inconsistent (noisy) training data. If there are several maximally specific consistent hypotheses, it can't backtrack.
- **Negative examples are not considered.**

Version space

DEFINITION

The **version space** $VS_{H,D}$ is the subset of hypotheses from H consistent with all training examples in D: $VS_{H,D}$ = { h ∈ H | Consistent(h, D) }. It can be represented by its **general boundary G** (maximally general consistent hypotheses) and **specific boundary S** (maximally specific consistent hypotheses).

LIST-THEN-ELIMINATE algorithm: (1) VersionSpace ← a list of every hypothesis in H; (2) for each training example ⟨x, c(x)⟩ remove any h with h(x) ≠ c(x); (3) output the list. It works whenever H is finite and is guaranteed to output all consistent hypotheses, but requires exhaustively enumerating H — unrealistic for all but trivial spaces.

Candidate-Elimination algorithm ★ Very likely

Finds **all** describable hypotheses consistent with the observed examples, using **both positive and negative** examples, by maintaining S and G.

```
Initialize G ← maximally general hypotheses in H   (<?,?,…,?>)
Initialize S ← maximally specific hypotheses in H  (<Ø,Ø,…,Ø>)
For each training example d:
 • If d is POSITIVE:
     – Remove from G any hypothesis inconsistent with d
     – For each s in S not consistent with d: remove s, and add to S all minimal
       generalizations h of s such that h is consistent with d and some member of G
       is more general than h
     – Remove from S any hypothesis more general than another hypothesis in S
 • If d is NEGATIVE:
     – Remove from S any hypothesis inconsistent with d
     – For each g in G not consistent with d: remove g, and add to G all minimal
       specializations h of g such that h is consistent with d and some member of S
       is more specific than h
     – Remove from G any hypothesis less general than another hypothesis in G
```

MEMORY TRICK

Positive → S grows (generalize), G gets pruned. Negative → G shrinks (specialize), S gets pruned.
"S moves up on Yes, G moves down on No."

CANDIDATE-ELIMINATION ON ENJOYSPORT (STEP BY STEP)

Init: $G_0 = \{\langle ?, ?, ?, ?, ?, ?\rangle\}$, $S_0 = \{\langle Ø, Ø, Ø, Ø, Ø, Ø\rangle\}$

x_1 = ⟨Sunny, Warm, Normal, Strong, Warm, Same⟩ (+): $S_1 = \{\langle$Sunny, Warm, Normal, Strong, Warm, Same$\rangle\}$, $G_1 = G_0$

x_2 = ⟨Sunny, Warm, High, Strong, Warm, Same⟩ (+): $S_2 = \{\langle$Sunny, Warm, ?, Strong, Warm, Same$\rangle\}$, $G_2 = G_0$

x_3 = ⟨Rainy, Cold, High, Strong, Warm, Change⟩ (−): $S_3 = S_2$. Specialize G minimally so x_3 is excluded but each member stays more general than S_2:

$G_3 = \{\langle$Sunny, ?, ?, ?, ?, ?$\rangle, \langle$?, Warm, ?, ?, ?, ?$\rangle, \langle$?, ?, ?, ?, ?, Same$\rangle\}$ (⟨?,?,Normal,…⟩ and ⟨?,?,?,?,Cool,?⟩ rejected — not more general than S_2)

x_4 = ⟨Sunny, Warm, High, Strong, Cool, Change⟩ (+): $S_4 = \{\langle$Sunny, Warm, ?, Strong, ?, ?$\rangle\}$; remove ⟨?, ?, ?, ?, ?, Same⟩ from G (inconsistent with x_4)

$G_4 = \{\langle$Sunny, ?, ?, ?, ?, ?$\rangle, \langle$?, Warm, ?, ?, ?, ?$\rangle\}$

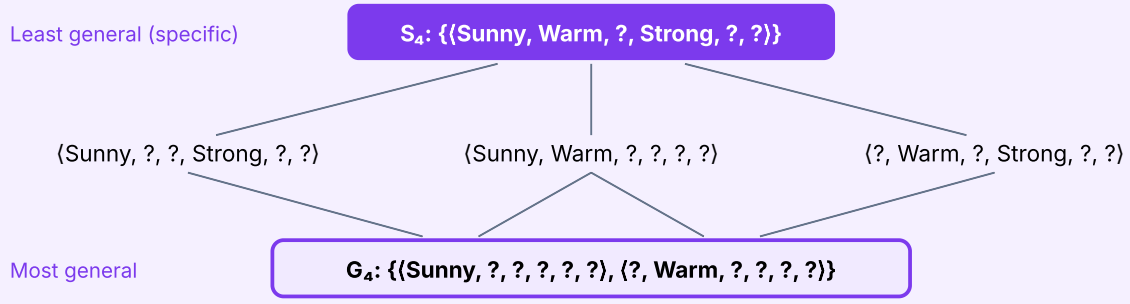


Fig 9.3 Final version space for EnjoySport: every hypothesis between S_4 and G_4 is consistent with all 4 examples (6 hypotheses in total).

CANDIDATE-ELIMINATION — "SMALL CIRCLES" (CLASS)

Size	Color	Shape	Class
big	red	circle	No
small	red	triangle	No
small	red	circle	Yes
big	blue	circle	No
small	blue	circle	Yes

$G_0 = \{(?, ?, ?)\}$, $S_0 = \{(\emptyset, \emptyset, \emptyset)\}$

1 (big, red, circle) −: minimal specializations: $G_1 = \{\langle small, ?, ?\rangle, \langle ?, blue, ?\rangle, \langle ?, ?, triangle\rangle\}$; $S_1 = S_0$

2 (small, red, triangle) −: ⟨small,?,?⟩ covers it → ⟨small, blue, ?⟩, ⟨small, ?, circle⟩; ⟨?,blue,?⟩ is fine; ⟨?,?,triangle⟩ covers it → ⟨big, ?, triangle⟩, ⟨?, blue, triangle⟩

$G_2 = \{\langle small, blue, ?\rangle, \langle small, ?, circle\rangle, \langle ?, blue, ?\rangle, \langle big, ?, triangle\rangle, \langle ?, blue, triangle\rangle\}$; $S_2 = S_0$ (as on the slide)

Strictly, the last step of the algorithm removes ⟨small, blue, ?⟩ and ⟨?, blue, triangle⟩ because they are less general than ⟨?, blue, ?⟩ → $G_2 = \{\langle small, ?, circle\rangle, \langle ?, blue, ?\rangle, \langle big, ?, triangle\rangle\}$. The final answer is the same either way.

3 (small, red, circle) +: $S_3 = \{\langle small, red, circle\rangle\}$; drop every g not covering it → $G_3 = \{\langle small, ?, circle\rangle\}$

4 (big, blue, circle) −: already excluded by G_3 and S_3 → no change

5 (small, blue, circle) +: $S_5 = \{\langle small, ?, circle\rangle\}$; $G_5 = \{\langle small, ?, circle\rangle\}$

S = G ⇒ converged: the learned concept is **"small circles" ⟨small, ?, circle⟩**

CANDIDATE-ELIMINATION — JAPANESE ECONOMY CAR (VERSION SPACE COLLAPSES)

$S_0 = \langle \Phi, \Phi, \Phi, \Phi, \Phi\rangle$, $G_0 = \langle ?, ?, ?, ?, ?\rangle$

x_1 ⟨Japan, Honda, Blue, 1980, Economy⟩ +: $S_1 = \langle Japan, Honda, Blue, 1980, Economy\rangle$; $G_1 = G_0$

x_2 ⟨Japan, Toyota, Green, 1970, Sports⟩ −: $S_2 = S_1$; $G_2 = \{\langle ?, Honda, ?, ?, ?\rangle, \langle ?, ?, Blue, ?, ?\rangle, \langle ?, ?, ?, 1980, ?\rangle, \langle ?, ?, ?, ?, Economy\rangle\}$

x_3 ⟨Japan, Toyota, Blue, 1990, Economy⟩ +: $S_3 = \langle Japan, ?, Blue, ?, Economy\rangle$; $G_3 = \{\langle ?, ?, Blue, ?, ?\rangle, \langle ?, ?, ?, ?, Economy\rangle\}$

x_4 ⟨USA, Chrysler, Red, 1980, Economy⟩ −: $S_4 = S_3$; $G_4 = \{\langle ?, ?, Blue, ?, ?\rangle, \langle Japan, ?, ?, ?, Economy\rangle\}$

x_5 ⟨Japan, Honda, White, 1980, Economy⟩ +: $S_5 = \langle Japan, ?, ?, ?, Economy\rangle$; $G_5 = \{\langle Japan, ?, ?, ?, Economy\rangle\}$ (Blue removed)

x_6 ⟨Japan, Toyota, Green, 1980, Economy⟩ +: $S_6 = G_6 = \langle Japan, ?, ?, ?, Economy\rangle$

x_7 ⟨Japan, Honda, Red, 1990, Economy⟩ −: S covers this negative → S must be removed; G can't be specialized while staying above S → **the version space collapses** (S and G become empty).

Conclusion: **no conjunctive hypothesis is consistent with the data set** (example 7 is inconsistent with the version space).

REMARKS ON VERSION SPACES

The version space learned by Candidate-Elimination converges toward the hypothesis that correctly describes the target concept, provided (1) there are no errors in the training examples and (2) some hypothesis in H correctly describes the target concept. When S and G become identical, the concept is learned exactly; if they become empty, the data is noisy or the target isn't in H.

Algorithm	Order / direction	Strategy	Examples used
Find-S	Specific-to-general	Top-down	Positive only
List-Then-Eliminate	General-to-specific	Bottom-up	Negative (and positive) — eliminates
Candidate-Elimination	Bi-directional	Bi-directional	Both

(Table reproduced as given on the class slide.)

LIKELY EXAM QUESTIONS — CH 9

1. Define machine learning (Samuel & Mitchell). Differentiate traditional programming and ML with a diagram. (5M)
2. Explain key terminology: model, feature, target, training, prediction. (5M)
3. What are the key tasks of ML? For each given scenario, which type of learning is used — justify. (5M)
4. What is a well-posed learning problem? Give T, P, E for checkers / handwriting / robot driving. (6M) ★
5. Explain the types of ML with examples. (6M) ★
6. Explain the steps in the ML process. (5M)
7. Apply Find-S to the given dataset and find the maximally specific hypothesis. (6–8M) ★
8. Apply Candidate-Elimination; show S and G after each example. Define version space. (10M) ★

60-SECOND RECAP

Mitchell: learn from E w.r.t. T and P. Terms: model, feature, label, training, prediction. Tasks: classification, regression (supervised); clustering, association, density estimation, PCA (unsupervised). Types: supervised (exemplars), unsupervised (similarity), reinforcement (critic), evolutionary (fitness). Find-S: start Ø, generalize on positives only. Candidate-Elimination: S up on +, G down on −; version space = all consistent h between S and G.

III

UNIT III · CIE-1 PORTION

Supervised Learning: Classification & Regression

- **Ch 10 · Classification vs Regression** — CR's question 1
- **Ch 11 · Linear Regression (with worked examples)** — CR's question 3, plus polynomial regression
- **Ch 12 · Naive Bayes Classifier** — CR's question 2
- **Ch 13 · Safety net**: Logistic Regression, SVM, KNN — one page each, in case the paper strays from the 3 questions

For CIE-1 your CR shared these 3 topics from Unit III: (1) difference between classification and regression, (2) Naive Bayes classifier, (3) linear regression with example. Master Ch 10–12 completely; skim Ch 13.

10 Classification vs Regression ★ CIE-1 Q

Both are **supervised learning**: we learn from examples that already have the correct answer (label). The only difference is **what kind of answer** we predict.

CLASSIFICATION

Supervised learning that predicts a categorical class label from input features. Answers "**Which category?**". Example: classify an email as spam or not spam; pass / fail.

REGRESSION

Supervised learning that predicts a numerical (continuous) target from input features. Answers "**How much? How many?**". Example: predict house price from area and location; exam marks.

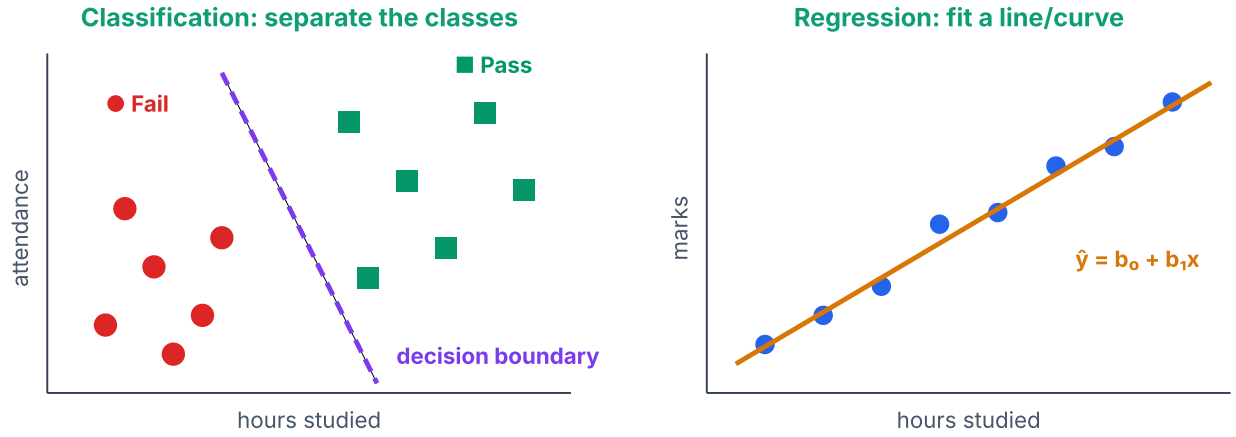


Fig 10.1 Classification outputs a class (which side of the boundary?). Regression outputs a number (where on the line?).

10.1 The complete comparison table ★ Very likely

Aspect	Classification	Regression
Objective	Predict a class or category	Predict a numerical magnitude
Target / output variable	Discrete / categorical (binary, multiclass, multilabel, ordinal)	Numeric, usually continuous
Output	A label, often with class probabilities	An estimated real-valued response
Question answered	"Which category?"	"How much / how many?"
Example: students	Predict pass / fail	Predict examination marks
Example: healthcare	Disease present / absent	Blood-glucose measurement
Example: real estate	Budget / mid-range / premium category	Selling price
Common algorithms	Logistic regression, Naive Bayes, SVM classifier, decision tree, random forest, k-NN	Linear, polynomial, ridge & lasso regression, SVR, regression tree, random-forest regressor
Learning loss	Cross-entropy (log loss), hinge loss	Squared error, absolute error
Evaluation metrics	Accuracy, precision, recall, F1, ROC-AUC, confusion matrix	MAE, MSE, RMSE, R^2
Plots	Decision boundary, confusion matrix, ROC/PR curve	Fitted line/curve, residual plot, actual vs predicted
What an error means	Wrong class or inaccurate class probability	Difference between observed and predicted values
Decision threshold	Often converts a probability to a label (e.g. 0.5)	Usually not needed
Data concerns	Class imbalance, mislabelled samples	Outliers, unequal coverage of the numeric range
Interpretation	Class membership and probability	Magnitude, slope and change in the target
Applications	Spam, fraud, sentiment, image recognition, diagnosis	Price, sales, rainfall, demand forecasting, weather

10.2 The mathematical models side by side

Binary classification (logistic model):

$$P(Y = 1 \mid x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

output squashed into 0...1, then thresholded

Linear regression:

$\hat{y} = w^T x + b \quad (= b_0 + b_1 x$ for one feature$)$

output is any real number

TRAP QUESTION

Logistic regression is a classification method despite its name — its output is a **class probability** which is then turned into a class by a threshold. (Why "regression"? It first models a continuous value — the probability $0 \le P(y=1) \le 1$ — and then a threshold converts it: P = 0.82 → class 1; P = 0.23 → class 0.)

10.3 Types of classification & quick ways to tell them apart

- **Binary**: 2 classes (Yes/No, Spam/Not spam, Fraud/Not fraud). **Multiclass**: > 2 classes, exactly one per example (Iris: Setosa / Versicolor / Virginica). **Multilabel**: an example can have several labels (a movie is both comedy and romance). **Ordinal**: ordered categories (low / medium / high).
- Regression can be **simple** (one input), **multiple** (many inputs) or **polynomial** (curved relationship).

MEMORY TRICK — THE "UNIT TEST"

Ask: **Can I average two answers meaningfully?** Average of ₹40 L and ₹60 L = ₹50 L ✔ → **regression**. Average of "cat" and "dog" = ? ✘ → **classification**. Also: if the answer has **units** (₹, kg, °C, marks) it's regression.

HOW TO WRITE THIS ANSWER FOR FULL MARKS (5–6M)

1. Define both in one line each (mention "supervised").
2. Draw Fig 10.1 (two small sketches: boundary vs line).
3. Write a comparison table with at least 8 rows: objective, output type, question answered, example, algorithms, loss, metrics, threshold.
4. End with the trap: logistic regression = classification.

LIKELY EXAM QUESTIONS — CH 10

1. Differentiate between classification and regression with examples. (5–6M) ★★★
2. Identify whether classification or regression: predict tomorrow's temperature; detect fraud; predict a student's CGPA; classify a tumour as benign/malignant; estimate house price. (2M each) → regression, classification, regression, classification, regression.
3. Why is logistic regression called a classification algorithm? (2M)

11 Linear Regression ★ CIE-1 Q

DEFINITION

Linear regression is a supervised learning and statistical method that models the expected numerical response as a linear combination of one or more predictors (input variables). Its coefficients are commonly estimated by **Ordinary Least Squares (OLS)**. In simple words: *draw the straight line that best fits the data points, then use the line to predict.*

- A statistical model showing the relation between two variables as a linear equation: **Y = mx + c** — "add up the inputs multiplied by some constants to get the output".
- **Regression analysis**: a form of predictive modelling that investigates the relationship between a **dependent** variable (target, y-axis) and **independent** variable(s) (predictors, x-axis); drawing a line through the data points that best fits the overall shape.
- **Origin of the word:** invented by **Francis Galton** (cousin of Charles Darwin). In 1877 he estimated the size of pea seeds from their parents' seeds; with human heights he saw that children of tall parents were tall but less extreme — heights **"regressed" toward the mean**. English meaning of regression: "a return to a former or less developed state" — unrelated to prediction. Statistical meaning: a measure of the relation between the mean value of one variable (output) and the corresponding values of other variables (e.g. time and cost).

11.1 Explanation with an example

Let x = hours studied and y = examination marks. A straight-line model estimates the **expected change in marks for a one-hour increase** in study time. The **intercept** (b_0) is the predicted value when x = 0; the **slope** (b_1) is the predicted change in y per unit of x.

POSITIVE RELATIONSHIP — Y = MX + C

Speed vs distance (time constant): y = distance travelled in a fixed time (dependent), x = speed (independent), m = positive slope, c = y-intercept. As x increases, y increases.

NEGATIVE RELATIONSHIP — Y = −MX + C

Speed vs time (distance constant): y = time taken to travel a fixed distance, x = speed, m = negative slope, c = y-intercept. As x increases, y decreases.

Understanding the regression line: plot the actual data points; draw a line and find the **error** between each estimated point and the actual point; the best-fit line is the one with the least error (least total squared distance between estimated and actual points).

11.2 Mathematical formulation ★ Very likely

Simple linear regression

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$$

β_0 = intercept, β_1 = slope, ε_i = error (noise) term

Fitted line & residual

$$\hat{y}_i = b_0 + b_1 x_i \qquad e_i = y_i - \hat{y}_i$$

b_0, b_1 are the estimates of β_0, β_1; e_i is the residual

Multiple linear regression: $y_i = \beta_0 + \Sigma_{j=1..p}\, \beta_j x_{ij} + \varepsilon_i$ or in matrix form $\mathbf{y = X\beta + \varepsilon}$

Least-squares cost and derivation of the normal equations

Sum of squared errors: $SSE = \Sigma_{i=1..n} (y_i - b_0 - b_1 x_i)^2$ (cost used in ML: $J(b_0, b_1) = \frac{1}{2n} \Sigma (\hat{y}_i - y_i)^2$)

Minimize: set partial derivatives to zero:

$\partial SSE/\partial b_0 = -2 \Sigma (y_i - b_0 - b_1 x_i) = 0$ $\partial SSE/\partial b_1 = -2 \Sigma x_i (y_i - b_0 - b_1 x_i) = 0$

Solving these two **normal equations** gives:

$$b_1 = \frac{\Sigma(x_i - \bar{x})(y_i - \bar{y})}{\Sigma(x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}} \qquad b_0 = \bar{y} - b_1 \bar{x}$$

Shortcut: $S_{xy} = \Sigma xy - (\Sigma x)(\Sigma y)/n$, $S_{xx} = \Sigma x^2 - (\Sigma x)^2/n$. Equivalent: $m = [n\Sigma xy - \Sigma x \Sigma y] / [n\Sigma x^2 - (\Sigma x)^2]$, $c = \bar{y} - m\bar{x}$.

Multiple regression: $\mathbf{b} = (X^T X)^{-1} X^T y$ (when $X^T X$ is invertible).

11.3 Worked example 1: study hours → marks ★ Very likely

Student	Hours (x)	Marks (y)	x^2	xy
1	1	40	1	40
2	2	50	4	100
3	3	65	9	195
4	4	70	16	280
5	5	85	25	425
Total	**15**	**310**	**55**	**1040**

Step 1 — means: $\bar{x} = 15/5 = 3$, $\bar{y} = 310/5 = 62$

Step 2 — centred sums:

$S_{xy} = 1040 - (15 \times 310)/5 = 1040 - 930 = 110$

$S_{xx} = 55 - 15^2/5 = 55 - 45 = 10$

Step 3 — slope & intercept:

$b_1 = 110/10 = \mathbf{11}$, $b_0 = 62 - 11(3) = \mathbf{29}$

Fitted line: $\hat{y} = 29 + 11x$

Interpretation: **+11 marks per extra study hour**.

Step 4 — predict 6 hours: $\hat{y}(6) = 29 + 66 = \mathbf{95}$ (extrapolation beyond 1–5 h — use with care)

Step 5 — fitted values and residuals

x	Observed y	Predicted ŷ	Residual e	e^2
1	40	40	0	0
2	50	51	−1	1
3	65	62	3	9
4	70	73	−3	9
5	85	84	1	1
Total (SSE)				**20**

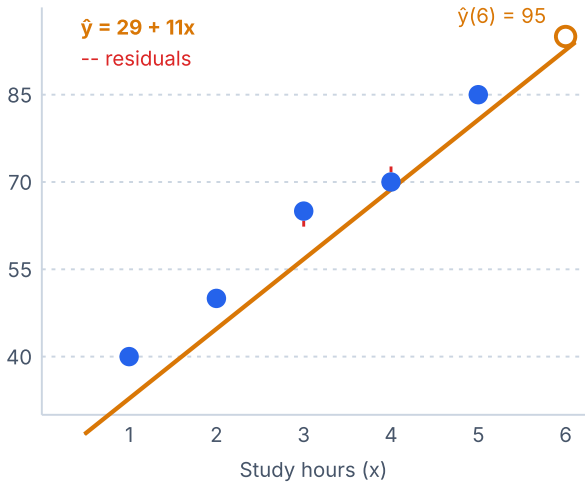


Fig 11.1 Observations (blue), fitted line (orange), residuals (red).

SSE = 20; MSE = 20/5 = **4**; RMSE = √4 = **2**
SST = Σ(y − 62)² = 484 + 144 + 9 + 64 + 529 = **1230**
$R^2 = 1 - SSE/SST = 1 - 20/1230 =$ **0.9837** (98% of the variation explained — excellent fit)

11.4 Worked example 2: the slide's "$R^2 = 0.3$" example (poor fit)

x	y	x − x̄	y − ȳ	(x−x̄)(y−ȳ)	(x−x̄)²
1	3	−2	−0.6	1.2	4
2	4	−1	0.4	−0.4	1
3	2	0	−1.6	0	0
4	4	1	0.4	0.4	1
5	5	2	1.4	2.8	4
x̄ = 3, ȳ = 3.6 · Totals				**4**	**10**

m = 4/10 = **0.4**, c = 3.6 − 0.4 × 3 = **2.4** → **y = 0.4x + 2.4**
Predicted y_p: 2.8, 3.2, 3.6, 4.0, 4.4
$\Sigma(y_p - \bar{y})^2 = 0.64 + 0.16 + 0 + 0.16 + 0.64 = 1.6$
$\Sigma(y - \bar{y})^2 = 0.36 + 0.16 + 2.56 + 0.16 + 1.96 = 5.2$
$R^2 = \Sigma(y_p - \bar{y})^2 / \Sigma(y - \bar{y})^2 = 1.6/5.2 =$ **0.31 ≈ 0.3**
→ **not a good fit**: actual and estimated points are far apart.

GOODNESS OF FIT — R^2 (COEFFICIENT OF DETERMINATION)

R^2 is a statistical measure of how close the data are to the fitted regression line (also called the coefficient of multiple determination). Two equivalent formulas for a least-squares line:

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{\Sigma(y - \hat{y})^2}{\Sigma(y - \bar{y})^2} = \frac{\Sigma(\hat{y} - \bar{y})^2}{\Sigma(y - \bar{y})^2}$$ (slide: "distance predicted − mean vs actual − mean")

R^2 ranges 0 to 1: close to 1 = good fit; close to 0 = poor fit. (Check: example 1 → 1210/1230 = 0.9837 by the second formula too ✔.)

11.5 Worked example 3: salary vs experience (Unit 3 notes)

Experience (years)	Salary (₹ lakhs)
1	3
2	4
3	5
4	6
5	7

$\bar{x} = 3$, $\bar{y} = 5$; $S_{xy} = (-2)(-2) + (-1)(-1) + 0 + 1 + 4 = 10$; $S_{xx} = 10$
$b_1 = 1$, $b_0 = 5 - 3 = 2$ → **y = 2 + x**
Perfectly linear ($R^2 = 1$): salary rises at a constant ₹1 lakh per year of experience. Prediction for 6 years: ₹8 lakhs.

11.6 Evaluation measures

$$MAE = \frac{1}{n}\Sigma|y_i - \hat{y}_i| \quad MSE = \frac{1}{n}\Sigma(y_i - \hat{y}_i)^2 \quad RMSE = \sqrt{MSE} \quad R^2 = 1 - SSE/SST$$

Accuracy / goodness of fit are measured by loss, R^2, adjusted R^2, RMSE, etc.

11.7 Gradient descent (iterative alternative to the normal equations)

$$b_0 \leftarrow b_0 - \alpha \frac{1}{n} \Sigma (\hat{y}_i - y_i) \qquad b_1 \leftarrow b_1 - \alpha \frac{1}{n} \Sigma (\hat{y}_i - y_i)\, x_i$$

α = learning rate. Start with any b_0, b_1; repeat the updates; each step moves "downhill" on the cost J until it converges to the least-squares solution.

11.8 Assumptions, limitations, selection criteria, uses

ASSUMPTIONS

- Linear conditional mean (relationship is linear)
- Zero conditional mean of errors
- No perfect multicollinearity among predictors
- Independent errors (for standard inference)
- Constant error variance — homoscedasticity (for conventional standard errors)
- Normal errors mainly needed for exact small-sample inference

LIMITATIONS

- Sensitive to influential **outliers**
- May fail to capture **nonlinear** relationships
- Unreliable **extrapolation**
- Cannot establish **causality** from association alone
- Missing values can remove useful data points

Selection criteria: use when predicting a continuous variable (sales per day, temperature of a city); **not good for classification**; not computationally expensive compared with decision trees etc.; comprehensible and transparent (simple notation).
Uses of regression analysis: (1) determining the strength of predictors (sales vs marketing, age vs income); (2) forecasting an effect — how much y changes per unit change in x ("how much extra sales income for each $100 spent on marketing?"); (3) trend forecasting (price of bitcoin in 6 months).
Where linear regression is used: evaluating trends and sales forecasts (monthly sales on y-axis vs time on x-axis, use the slope to forecast); analysing the impact of price changes (quantity sold vs price); risk assessment in financial services and insurance (number of claims per customer vs age).

11.9 Linear vs polynomial regression

$$\text{Linear: } y = b_0 + b_1 x$$

"Fit a straight line."

$$\text{Polynomial (degree 2): } y = b_0 + b_1 x + b_2 x^2$$

$$\text{(degree 3: } \ldots + b_3 x^3\text{)}$$

"Create powers of the input and fit a curve."

Both predict a continuous value; the difference is the **shape** of relationship they can model. Polynomial regression is still *linear in the coefficients*, so it is solved with the same least-squares method after creating x^2, x^3 features. **Example from notes:** X = 1…8, y = 3, 5, 9, 15, 23, 33, 45, 59 (curved). A straight line gives $\hat{y} = -12 + 8x$ (predicts −4 at x = 1 — bad), while degree 2 fits exactly: **$\hat{y} = 3 - x + x^2$**.

```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
X = np.array([1,2,3,4,5,6,7,8]).reshape(-1,1); y = np.array([3,5,9,15,23,33,45,59])
lin = LinearRegression().fit(X, y)                        # straight line
X_poly = PolynomialFeatures(degree=2).fit_transform(X)    # adds 1, x, x^2 columns
poly = LinearRegression().fit(X_poly, y)                  # curve
```

HOW TO WRITE "EXPLAIN LINEAR REGRESSION WITH AN EXAMPLE" (10M)

1. Definition + purpose (predict continuous y from x) + dependent/independent variables.
2. Equation $y = b_0 + b_1x + \varepsilon$ with each symbol explained; residual.
3. Least-squares idea + the b_1, b_0 formulas (derivation if 10M).
4. **Numerical example**: table with x, y, x^2, xy → means → S_{xy}, S_{xx} → b_1, b_0 → line → one prediction.
5. Draw the scatter plot with the line.
6. Goodness of fit (R^2), assumptions, applications, limitations.

LIKELY EXAM QUESTIONS — CH 11

1. Explain linear regression with an example. (10M) ★★★
2. Derive the least-squares estimates of slope and intercept. (6M)
3. Given data, fit a regression line and predict y for a new x; compute R^2. (6–8M) ★
4. What is R^2? Interpret a value of 0.98 and 0.3. (3M)
5. Differentiate linear and polynomial regression. (4M)

12 Naive Bayes Classifier ★ CIE-1 Q

DEFINITION

A **Naive Bayes classifier** is a supervised machine-learning algorithm that predicts the class of an instance using probability. It applies **Bayes' theorem** to estimate how likely each class is given the observed features, and selects the class with the **highest posterior probability**. It is called **naive** because it assumes the features are conditionally independent given the class — once the class is known, one feature tells us nothing about another. This simplifies calculation, although real features may be related.

Popular for: spam email detection, text/document classification, sentiment analysis, news classification, medical diagnosis, recommendation and filtering systems. Example: "Congratulations! You won a free prize." — NB looks at evidence (contains "free"? "won"? "prize"?), computes the probability of each class and picks the highest. **Basic question:** *given the observed features, what is the probability that this observation belongs to a particular class?*

12.1 Bayes' theorem (the engine)

$$P(C \mid X) = \frac{P(X \mid C) \cdot P(C)}{P(X)} \qquad \text{Posterior} \propto \text{Likelihood} \times \text{Prior}$$

Term	Meaning	Email example
C	Class being considered	Spam
X	Observed features / evidence	Email contains "offer"
$P(C)$	**Prior** — probability of the class before observing evidence	Overall proportion of spam emails
$P(X\|C)$	**Likelihood** — probability of the evidence within that class	Probability that a spam email contains "offer"
$P(X)$	**Evidence** — overall probability of the evidence	Probability that any email contains "offer"
$P(C\|X)$	**Posterior** — probability of the class after observing the evidence	Probability the email is spam given "offer"

12.2 How Naive Bayes classifies an instance ★ Very likely

Instance $X = (x_1, x_2, \ldots, x_n)$. Naive (conditional independence) assumption: $P(X \mid C) = \prod_{i=1..n} P(x_i \mid C)$

So $P(C \mid X) = P(C) \prod P(x_i \mid C) / P(X)$. $P(X)$ is the same for all classes → compare only the **unnormalized scores**:

$$\text{Score}(C) = P(C) \prod_{i=1..n} P(x_i \mid C) \qquad \hat{C} = \text{argmax}_C \left[P(C) \prod P(x_i \mid C) \right]$$

This is the **Maximum A Posteriori (MAP)** decision.

Why "naive"? An email with X_1 = contains "free", X_2 = "prize", X_3 = "winner" — these words are clearly related, yet NB approximates $P(X_1, X_2, X_3 \mid C) \approx P(X_1|C)\, P(X_2|C)\, P(X_3|C)$. The assumption is often unrealistic — hence "naive" — but surprisingly it works very well, especially for text classification.

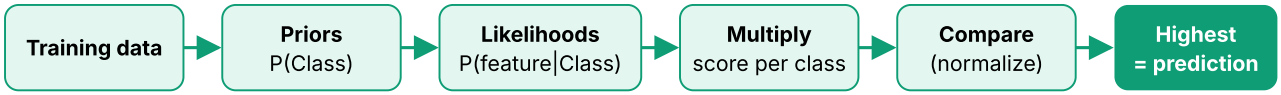


Fig 12.1 The complete Naive Bayes process — use these 6 steps in every numerical.

12.3 How Naive Bayes handles uncertainty

Uncertainty = the evidence doesn't identify the class with complete certainty (some spam contains "offer", some doesn't; some legitimate mail also contains "offer"; an email may have both promotional and ordinary words). NB assigns **probabilities** instead of hard rules: **before** features → priors from training data; **after** features → likelihoods update the class probabilities; **conflicting evidence** → combined probabilistically (a feature supporting Spam is balanced against one supporting Not Spam); **decision** → usually the class with the highest posterior. E.g. P(Spam|X) = 0.80, P(Not Spam|X) = 0.20 → predicts Spam but still assigns 20% to Not Spam; 0.51 vs 0.49 would be far less decisive. Limitation: probabilities depend on the training data and the independence assumption — NB can be overconfident, so its probabilities are not perfectly calibrated measures of certainty.

12.4 Worked example A — email classification (Bernoulli NB) ★ Very likely

Features: x_1 = does the email contain "offer"? x_2 = does it contain "meeting"? (Bernoulli NB: each feature present/absent.)

Email	"offer"	"meeting"	Class
E1	Yes	No	Spam
E2	Yes	No	Spam
E3	Yes	No	Spam
E4	Yes	Yes	Spam
E5	No	No	Spam
E6	No	Yes	Not Spam
E7	No	Yes	Not Spam
E8	No	Yes	Not Spam
E9	Yes	Yes	Not Spam
E10	No	No	Not Spam

New email: contains "offer", does not contain "meeting" → X = (offer = Yes, meeting = No).

Probability	Spam	Not Spam
Prior	5/10 = 0.5	5/10 = 0.5
P(offer = Yes | class)	4/5 = 0.8	1/5 = 0.2
P(meeting = No | class)	4/5 = 0.8	1/5 = 0.2

Step 3 — scores: Score(Spam) = 0.5 × 0.8 × 0.8 = **0.32**; Score(Not Spam) = 0.5 × 0.2 × 0.2 = **0.02** → 0.32 > 0.02 ⇒ **Spam**

Step 4 — normalize: sum = 0.34 → P(Spam|X) = 0.32/0.34 = **0.9412**, P(Not Spam|X) = 0.02/0.34 = 0.0588 (check: sum = 1 ✔)

Step 5 — how belief updates: no word evidence → 50%; "offer" present → (0.5 × 0.8)/(0.5 × 0.8 + 0.5 × 0.2) = 80%; "offer" present and "meeting" absent → 94.12%.

WHAT IF EVIDENCE CONFLICTS? (OFFER = YES, MEETING = YES)

Score(Spam) = 0.5 × 0.8 × 0.2 = 0.08; Score(Not Spam) = 0.5 × 0.2 × 0.8 = 0.08 → P = 0.5 / 0.5. The model has **no preference** — a system may apply a tie-breaking rule or flag the email for human review. NB can represent uncertainty even when evidence doesn't support a clear decision.

12.5 The zero-frequency problem & Laplace smoothing ★ Very likely

If a feature value never appears with a class in training, its likelihood is **0** — e.g. P(offer = Yes | Not Spam) = 0 — and because NB **multiplies** likelihoods, the whole score becomes 0 regardless of all other evidence. Fix: **Laplace (add-one) smoothing** — add 1 to each feature-value count:

$$P(x_i = v \mid C) = \frac{\text{Count}(x_i = v, C) + 1}{\text{Count}(C) + k} \quad (k = \text{number of possible values of the feature; } k = 2 \text{ for binary})$$

P(offer = Yes | Spam) = (4 + 1)/(5 + 2) = 5/7; P(meeting = No | Spam) = 5/7; P(offer = Yes | Not Spam) = (1 + 1)/(5 + 2) = 2/7; P(meeting = No | Not Spam) = 2/7
Score(Spam) = 0.5 (5/7)(5/7) = 25/98; Score(Not Spam) = 0.5 (2/7)(2/7) = 4/98 → P(Spam|X) = 25/29 = **86.21%** — still Spam; smoothing avoids zeros and moderates extreme estimates from small datasets.

12.6 Worked example B — 100 emails and the word "free"

	Spam	Not Spam
Total emails	40	60
Contains "free"	30	6
Doesn't contain "free"	10	54

P(Spam) = 40/100 = 0.4, P(NotSpam) = 0.6
P(Free|Spam) = 30/40 = 0.75, P(Free|NotSpam) = 6/60 = 0.1
P(Free) = (0.75)(0.4) + (0.1)(0.6) = 0.30 + 0.06 = 0.36
P(Spam|Free) = (0.75 × 0.4)/0.36 = 0.30/0.36 = **0.8333 = 83.33%** → classify as Spam

12.7 Worked example C — student pass/fail (categorical NB, with a zero!)

Student	Hours	Attendance	Result
1	Low	Low	Fail
2	Low	Low	Fail
3	Low	High	Fail
4	High	Low	Pass
5	High	High	Pass
6	High	High	Pass
7	High	High	Pass

New student: Hours = High, Attendance = High
P(Pass) = 4/7; P(Hours=High|Pass) = 4/4 = 1; P(Att=High|Pass) = 3/4
Score(Pass) = 4/7 × 1 × 3/4 = 3/7 = **0.4286**
P(Fail) = 3/7; P(Hours=High|Fail) = 0/3 = **0** (zero frequency!); P(Att=High|Fail) = 1/3
Score(Fail) = 0 → prediction **PASS**
With Laplace (k = 2): Pass: 4/7 × 5/6 × 4/6 = 0.317; Fail: 3/7 × 1/5 × 2/5 = 0.034 → still Pass (≈ 90%). The sklearn `CategoricalNB` code in the notes outputs `[1]` = PASS (it applies smoothing by default, alpha = 1).

12.8 Worked example D — species M vs H (10-mark question from slides)

No	Colour	Legs	Height	Smelly	Species
1	White	3	Short	Yes	M
2	Green	2	Tall	No	M
3	Green	3	Short	Yes	M
4	White	3	Short	Yes	M
5	Green	2	Short	No	H
6	White	2	Tall	No	H
7	White	2	Tall	No	H
8	White	2	Short	Yes	H

P(·|class)	M	H
Colour = Green	2/4	1/4
Legs = 2	1/4	4/4
Height = Tall	1/4	2/4
Smelly = No	1/4	3/4

New: (Green, 2, Tall, No). $P(M) = P(H) = 4/8 = 0.5$

$\text{Score}(M) = 0.5 \times 2/4 \times 1/4 \times 1/4 \times 1/4 = \mathbf{0.0039}$

$\text{Score}(H) = 0.5 \times 1/4 \times 1 \times 2/4 \times 3/4 = \mathbf{0.0469}$

$\Rightarrow$ **Species H** ($P \approx 0.0469/0.0508 = 92.3\%$)

See also the PlayTennis (→ No, 0.795) and stolen-car (→ Not stolen) examples in Chapter 8.7 — those are the two most commonly asked NB numericals.

12.9 Types of Naive Bayes

Type	Feature representation	Example
Gaussian NB	Continuous numerical features, modelled with a Gaussian (normal) distribution within each class	Classifying iris flowers by petal measurements
Multinomial NB	Counts, such as word frequencies	Classifying documents by word counts
Bernoulli NB	Binary features: present / absent	Emails by whether particular words occur
Categorical NB	Features with discrete categories	Weather: sunny / cloudy / rainy; Low / High

12.10 Advantages, limitations and comparisons

ADVANTAGES

- **Simple** to understand and implement
- **Fast** to train and predict
- Works well with **high-dimensional** data (thousands of words)
- Needs relatively **little training data**
- Often very effective for **text classification**: spam filtering, sentiment analysis, news categorization, document classification
- Produces **class probability** estimates

LIMITATIONS

- **Independence assumption** — biggest limitation ("machine" and "learning" are related, NB treats them as independent)
- **Correlated features** can count similar evidence twice
- **Zero-frequency problem** → needs Laplace smoothing
- Probability estimates may be **overconfident / poorly calibrated**
- Performance depends on whether the chosen likelihood model suits the data

Feature	Naive Bayes	Logistic Regression	SVM
Type / model	Classification; generative, based on Bayes' theorem	Classification; discriminative model	Classification; max-margin
Main assumption	Conditional independence	No NB independence assumption	—
Probability output	Yes	Yes	Not inherently
Training speed	Very fast	Fast	Can be slower
Text classification	Very effective, very common	Also widely used	Also effective
Small datasets	Often useful	Can work well	Good (small/medium)
Correlated features	Problematic	Less restrictive	—
Feature scaling	Usually not required	Often helpful	Important
Nonlinear boundary	Not via kernels	No (linear)	Yes, using kernels
Main variants / concept	Gaussian, Multinomial, Bernoulli	Binary, multiclass	Maximum margin

HOW TO WRITE "EXPLAIN THE NAIVE BAYES CLASSIFIER" (10M)

1. Definition + why "naive" (conditional independence).
2. Bayes' theorem with prior/likelihood/evidence/posterior labelled.
3. NB formula: $\hat{C} = \text{argmax } P(C) \prod P(x_i|C)$ (MAP; P(X) dropped).
4. A worked numerical (email or PlayTennis): priors table → likelihood table → scores → decision → normalize.
5. Zero-frequency + Laplace smoothing.
6. Types, advantages, limitations, applications.

LIKELY EXAM QUESTIONS — CH 12

1. Explain the Naive Bayes classifier with an example. (10M) ★★★
2. Why is it called "naive"? What is the zero-frequency problem and how is it solved? (4M)
3. Classify the given instance using NB (PlayTennis / email / stolen car / species). (8–10M) ★
4. How does Naive Bayes handle uncertainty? (4M)
5. List the types, advantages and limitations of NB. (5M)

60-SECOND RECAP — UNIT III CORE

Classification → category; **regression** → number. LR: $\hat{y} = b_0 + b_1x$; $b_1 = S_{xy}/S_{xx}$, $b_0 = \bar{y} - b_1\bar{x}$; hours/marks → $\hat{y} = 29 + 11x$, $R^2 = 0.9837$. NB: $\hat{C} = \text{argmax } P(C)\prod P(x_i|C)$; email → Spam 94.12% (86.21% with Laplace).

13 Safety Net: Logistic Regression, SVM, KNN

Not in the CR's list, but they are the other Unit III classification algorithms in your notes. One page each — enough to write a 5-mark answer if the paper surprises you.

13.1 Logistic Regression

Supervised algorithm for **classification** (Pass/Fail, Spam/Not, Disease/No, Fraud/Not, Buy/Not). It predicts the **probability** that an observation belongs to a class. **Why not linear regression?** Its outputs (−0.5, 1.5, 2.3...) are not valid probabilities (must be 0 ≤ P ≤ 1). So we pass the linear combination through the **sigmoid**:

$$z = b_0 + b_1x_1 + \ldots + b_nx_n \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

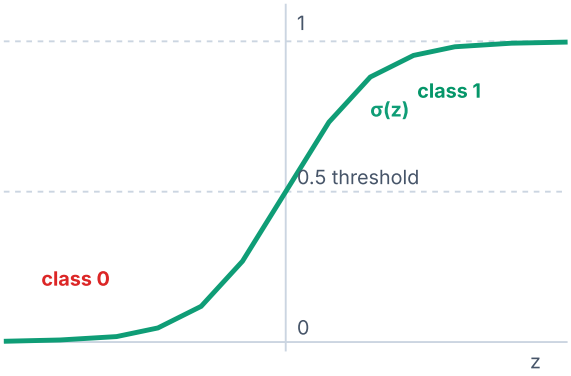


Fig 13.1 The S-shaped sigmoid squashes any z into (0, 1).

Steps: (1) linear combination z → (2) sigmoid → probability 0...1 → (3) threshold (usually 0.5): P ≥ 0.5 → y = 1 else 0 → (4) class. **Example:** hours = 5 → z = 1.2 → σ(1.2) = 0.768 > 0.5 → PASS (predict_proba might give [[0.05, 0.95]]). **Decision boundary:** P = 0.5 ⇔ z = 0 ⇔ $b_0 + b_1x = 0$ (a point); for 2 features a line, 3+ features a hyperplane. **Binary** (2 classes) vs **multiclass** (Iris: Setosa/Versicolor/Virginica). sklearn parameters: **C** = inverse regularization strength (smaller C → stronger regularization), **max_iter**, **solver** ("lbfgs").

Linear vs logistic: linear answers "How much?" (salary, house price, temperature); logistic answers "Which class?" (pass/fail, spam, fraud, disease).

13.2 Support Vector Machine (SVM)

Supervised algorithm mainly for classification (also regression — SVR). **Central idea:** find the decision boundary that separates the classes with the maximum possible margin from the data points.

- **Hyperplane:** $w_1x_1 + w_2x_2 + b = 0$ (e.g. $2x_1 + x_2 - 10 = 0$). 2 features → line, 3 → plane, >3 → hyperplane. General: $w \cdot x + b = 0$; classify by $\text{sign}(w \cdot x + b)$ with labels +1/−1.
- **Margin:** distance between the boundary and the closest points of each class; SVM maximizes it. Margin boundaries $w \cdot x + b = +1$ and -1; **margin width = $2/\|w\|$**. Distance of a point from the hyperplane = $|w \cdot x + b| / \|w\|$.
- **Support vectors:** the training points closest to the boundary (they satisfy $y_i(w \cdot x_i + b) = 1$); they "support"/determine the hyperplane — moving them moves the boundary, removing far points doesn't.

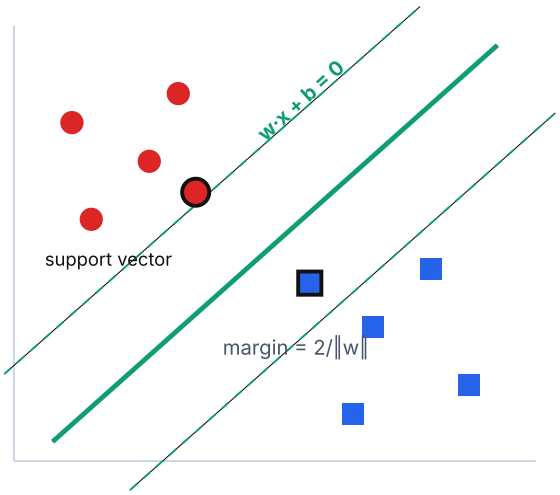


Fig 13.2 Maximum-margin hyperplane; outlined points are support vectors.

- **Hard margin:** assumes perfect separation: minimize $\frac{1}{2}\|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1$. **Soft margin:** allows some points inside the margin or misclassified (noise, outliers, overlap) — controlled by **C**: large C = try hard to classify all training points (small margin, complex boundary, may overfit); small C = allow mistakes for a wider margin (simpler, better generalization).
- **Kernel trick:** builds nonlinear boundaries without explicitly transforming the data. Kernels: linear, polynomial (degree), **RBF** $K(x, x') = \exp(-\gamma\|x - x'\|^2)$, sigmoid. **γ**: small → each point influences a large area → smoother boundary; large → small influence → complex boundary.
- **Feature scaling** is important (age 18–60 vs salary 20,000–2,00,000): use StandardScaler — `fit_transform` on train, `transform` on test (avoid data leakage).
- **Pros:** effective in high dimensions (text, image), maximum-margin principle, kernel trick, good for small/medium datasets. **Cons:** slow on very large data, needs scaling, sensitive to C/γ/kernel/degree, less interpretable. Application: network intrusion detection (normal vs attack).

13.3 K-Nearest Neighbors (KNN)

Supervised algorithm for classification *and* regression: a new point gets the class of its nearby points — "*tell me who your nearest neighbours are, and I will tell you your class*". **K** = number of neighbours considered; **nearest** = closest by distance; **neighbours** = those closest points.

$$\text{Euclidean } d = \sqrt{[(x_1-y_1)^2 + (x_2-y_2)^2 + \ldots]}$$
$$\text{Manhattan } d = \Sigma|x_i - y_i|$$

Algorithm: (1) compute the distance from the new point to every training point; (2) pick the K nearest; (3) **majority vote** (classification) or **average** (regression, e.g. houses ₹50, 55, 60 L → ₹55 L).

Choosing K: K = 1 → captures very local patterns but sensitive to noise/outliers → **overfitting**; very large K (50, 100) → very smooth boundary, ignores local patterns → **underfitting**; moderate K (≈5) balances. Choose K by cross-validation on the training set, not by test accuracy.

Lazy / instance-based learning: KNN learns no equation (no b_0, b_1); training just **stores** the data; the work happens at prediction (distances → K neighbours → vote). Needs **feature scaling** (salary would dominate age). Parameters: n_neighbors, weights ('uniform' or 'distance' — closer neighbours count more), metric (euclidean, manhattan, minkowski).

Pros: very simple, easy to implement, no real training, nonlinear boundaries, classification + regression. **Cons:**

NUMERICAL (ILLUSTRATIVE)

Student	Hours x_1	Attend. x_2 (/10)	Class	d to (6, 7)
A	2	4	Fail	5.00
B	3	5	Fail	3.61
C	6	8	Pass	1.00
D	7	7	Pass	1.00
E	8	9	Pass	2.83
F	4	3	Fail	4.47

New student (6, 7). K = 3 → C, D, E → Pass 3, Fail 0 → **Pass**. K = 5 → C, D, E, B, F → Pass 3, Fail 2 → **Pass** (majority vote).

Learner	Training	Prediction
Logistic regression	optimize → learn coefficients	fast (plug into equation)
KNN (lazy)	just store data	slow (distances, vote)

slow prediction, sensitive to scaling, irrelevant features and choice of K, curse of dimensionality. Uses: healthcare (similar patients), recommendation, image classification, pattern recognition, anomaly detection, student analytics.

ONE-LINE ANSWERS

Logistic = sigmoid of a linear score + threshold. SVM = maximum-margin hyperplane defined by support vectors; kernels for nonlinearity; C trades margin vs errors. KNN = vote of the K nearest points; lazy learner; scale your features; small K overfits, large K underfits.

14 Last-Minute Revision

Read this the night before and the morning of the exam. Everything here is explained in full in the earlier chapters.

14.1 Formula & rule sheet

Search (Unit I)	
Greedy best-first	$f(n) = h(n)$
A*	$f(n) = g(n) + h(n)$
Admissible	$h(n) \le h^*(n)$ (never overestimates)
Consistent	$h(n) \le c(n, a, n') + h(n')$
BFS / IDS time	$O(b^d)$; IDS space $O(bd)$
DFS	time $O(b^m)$, space $O(bm)$
Bidirectional	$O(b^{d/2})$
Effective branching factor	$N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$
Logic (Units I–II)	
Implication	$P \Rightarrow Q \equiv \neg P \vee Q$ (false only when P=T, Q=F)
Biconditional	$P \Leftrightarrow Q \equiv (P \Rightarrow Q) \wedge (Q \Rightarrow P)$
De Morgan	$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$; $\neg(P \vee Q) \equiv \neg P \wedge \neg Q$
Quantifier negation	$\neg\forall x\ P \equiv \exists x\ \neg P$; $\neg\exists x\ P \equiv \forall x\ \neg P$
Entailment	$KB \models \alpha \Leftrightarrow M(KB) \subseteq M(\alpha) \Leftrightarrow KB \wedge \neg\alpha$ unsat.
Modus Ponens	$\alpha \Rightarrow \beta,\ \alpha \vdash \beta$
Resolution	$P \vee Q,\ \neg Q \vee R \vdash P \vee R$
UI / EI	$\forall v\ \alpha \vdash SUBST(\{v/g\}, \alpha)$; $\exists v\ \alpha \vdash SUBST(\{v/k\}, \alpha)$, k new
Unify	$SUBST(\theta, p) = SUBST(\theta, q)$
Quantifier + connective	$\forall \ldots \Rightarrow \quad \exists \ldots \wedge$

Probability (Unit II)	
Conditional	$P(A\|B) = P(A \wedge B) / P(B)$
Product rule	$P(A \wedge B) = P(A\|B)P(B) = P(B\|A)P(A)$
Sum rule	$P(A \vee B) = P(A) + P(B) - P(A \wedge B)$
Bayes	$P(A\|B) = P(B\|A)P(A) / P(B)$
Total probability	$P(B) = \Sigma\ P(B\|A_i)P(A_i)$
MAP	$h_{MAP} = \text{argmax}\ P(D\|h)P(h)$
Bayes net joint	$P(x_{1..n}) = \prod P(x_i \mid \text{Parents}(X_i))$
Naive Bayes	$v_{NB} = \text{argmax}\ P(v) \prod P(a_i\|v)$
Laplace smoothing	$(\text{count} + 1) / (\text{Count}(C) + k)$
Regression (Unit III)	
Model	$\hat{y} = b_0 + b_1 x$; $e = y - \hat{y}$
Slope	$b_1 = S_{xy}/S_{xx} = \Sigma(x-\bar{x})(y-\bar{y})/\Sigma(x-\bar{x})^2$
Shortcuts	$S_{xy} = \Sigma xy - \Sigma x \Sigma y/n$; $S_{xx} = \Sigma x^2 - (\Sigma x)^2/n$
Intercept	$b_0 = \bar{y} - b_1\bar{x}$
Errors	$MSE = SSE/n$; $RMSE = \sqrt{MSE}$; $MAE = \Sigma\|e\|/n$
R^2	$1 - SSE/SST = \Sigma(\hat{y}-\bar{y})^2/\Sigma(y-\bar{y})^2$
Matrix form	$b = (X^TX)^{-1}X^Ty$
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$
SVM margin	$2/\|w\|$

14.2 Answers you must be able to reproduce from memory

Problem	Final answer
A* Arad → Bucharest (Romania)	Arad–Sibiu–Rimnicu Vilcea–Pitesti–Bucharest, cost **418** (greedy gives 450 via Fagaras)
8-puzzle h1, h2 for 7 2 4 / 5 _ 6 / 8 3 1	h1 = **8**, h2 = **18**
Crime KB	Criminal(West) in 2 FC iterations; θ = {x/West, y/M1, z/Nono}
Unify Q(a, g(x, a), f(y)) & Q(a, g(f(b), a), x)	{x/f(b), y/b}
Meningitis	P(M|S) = **0.0014**
Cancer, positive test	0.0078 vs 0.0298 → h_{MAP} = **¬cancer** (P(cancer|+) = 0.21)
Liver disease / alcoholic	**0.14**
PlayTennis (sunny, cool, high, strong)	0.0053 vs 0.0206 → **No** (0.795)
Stolen car (Red, SUV, Domestic)	0.024 vs 0.072 → **Not stolen**
Rain–Sprinkler–GrassWet	P(W) = **0.448**
Cloudy network	P(W) = **0.6471**; P(S|W) = 0.430 (slide: 0.438)
Find-S EnjoySport	⟨Sunny, Warm, ?, Strong, ?, ?⟩
Candidate-Elimination EnjoySport	S = {⟨Sunny, Warm, ?, Strong, ?, ?⟩}; G = {⟨Sunny, ?, ...⟩, ⟨?, Warm, ?, ...⟩}
Hours → marks regression	$\hat{y}$ = **29 + 11x**; $\hat{y}(6)$ = 95; R^2 = **0.9837**
Email NB (offer = Y, meeting = N)	Spam, **94.12%**; with Laplace **86.21%**

14.3 Top questions by priority

MUST-PREPARE (★★★)

1. PEAS for a given agent + environment properties table
2. Types of agents with diagrams
3. BFS/DFS/UCS/IDS comparison; trace a search on a graph
4. A* algorithm, admissible & consistent heuristics; trace on a map
5. Wumpus world: PEAS, properties, reasoning
6. PL → CNF; prove by resolution
7. English → FOL translations
8. Forward/backward chaining: crime / peanuts
9. Resolution refutation in FOL (John happy)
10. Bayes' theorem + cancer/meningitis MAP
11. Bayesian network definition + numerical
12. Classification vs regression
13. Linear regression with numerical example
14. Naive Bayes with numerical example

ALSO PREPARE (★★)

1. Four approaches to AI; Turing test
2. Rationality vs omniscience; learning and autonomy
3. Problem formulation (8-puzzle, vacuum, 8-queens)
4. Heuristics h1/h2, dominance, relaxed problems
5. Entailment, soundness, completeness
6. Forward vs backward chaining (PL & FOL)
7. Unification algorithm + MGU
8. Knowledge engineering (full adder)
9. Knowledge graphs: triples, ontology, open-world, SPARQL
10. Well-posed learning problems (T, P, E)
11. Types of ML; key tasks; ML process
12. Find-S and Candidate-Elimination numericals
13. Laplace smoothing; why "naive"

14.4 Night-before checklist

☐ I can draw: agent–environment loop, 5 agent architectures, Wumpus grid

☐ I know KG vs ontology, RDF triple, open-world assumption

☐ I can write the BFS / DFS / A* comparison table from memory

☐ I can trace A* on the Romania map and get 418

☐ I can write a truth table for ⇒ and ⇔

☐ I can convert a sentence to CNF and resolve to □

☐ I can translate "all / some / not all / no one" sentences into FOL

☐ I can unify two expressions and give the MGU

☐ I can do the crime example by FC and BC with a proof tree

☐ I can derive Bayes' theorem and solve the cancer MAP problem

☐ I can compute a joint probability and P(W) in a Bayes network

☐ I can run Naive Bayes on PlayTennis (0.0053 vs 0.0206)

☐ I can give T, P, E for three learning problems

☐ I can run Find-S and Candidate-Elimination on EnjoySport

☐ I can write the classification vs regression table (8+ rows)

☐ I can fit $\hat{y} = 29 + 11x$ from the hours/marks table and compute R^2

IN THE EXAM HALL

- Read all questions first; start with the one you know best to build momentum.
- Spend marks-proportional time: about 1.5 min per mark.
- If a numerical goes wrong, still write the formula, the method and the table — method marks add up.
- Leave 5 minutes to underline keywords and box final answers.

All the best for CIE-1!